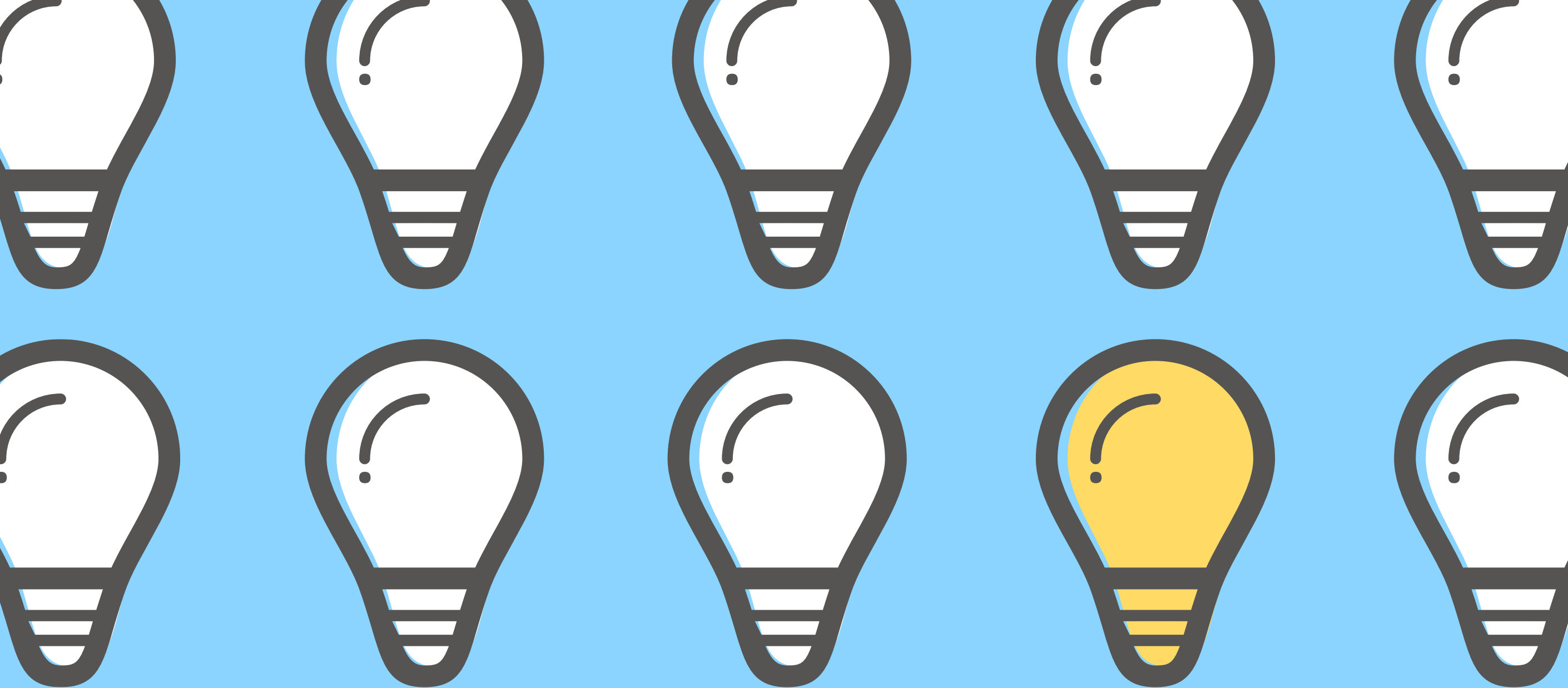




FLAVIO COPES

THE WEB PLATFORM API COMPLETE REFERENCE

A book for Frontend Developers

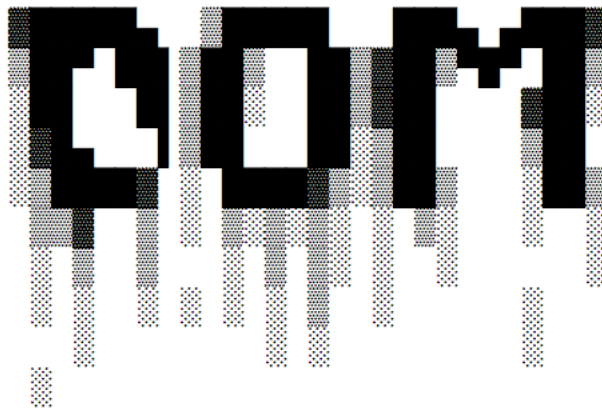
BY FLAVIO COPES
[HTTPS://FLAVIOCOPES.COM](https://flaviocopes.com)

UPDATED MAY 19, 2018

- [The DOM](#)
- [Progressive Web Apps](#)
- [Service Workers](#)
- [XHR](#)
- [Fetch API](#)
- [Channel Messaging API](#)
- [Cache API](#)
- [Push API](#)
- [Notifications API](#)
- [IndexedDB](#)
- [Selectors API](#)
- [Web Storage API](#)
- [Cookies](#)
- [History API](#)
- [Efficiently load JavaScript with defer and async](#)
- [The WebP Image Format](#)
- [SVG](#)
- [Data URLs](#)
- [CORS](#)
- [Web Workers](#)
- [requestAnimationFrame\(\)](#)
- [Console API](#)
- [WebSockets](#)
- [The Speech Synthesis API](#)

THE DOCUMENT OBJECT MODEL (DOM)

DOM stands for Document Object Model, a representation of an HTML document in nodes and objects. Browsers expose an API that you can use to interact with the DOM. That's how modern JavaScript frameworks work, they use the DOM API to tell the browser what to display on the page



- The Window object
 - Properties
 - Methods
- The Document object
- Types of Nodes
- Traversing the DOM
 - Getting the parent
 - Getting the children
 - Getting the siblings
- Editing the DOM

The DOM is the browser internal representation of a web page. When the browser retrieves your HTML from your server, the parser analyzes the structure of your code, and creates a model of it. Based on this model, the browser then renders the page on the screen.

Browsers expose an API that you can use to interact with the DOM. That's how modern JavaScript frameworks work, they use the DOM API to tell the browser what to display on the page.

In Single Page Applications, the DOM continuously changes to reflect what appears on the screen, and as a developer you can inspect it using the Browser Developer Tools.

The DOM is language-agnostic, and the de-facto standard to access the DOM is by using JavaScript, since it's the only language that browsers can run.

The DOM is standardized by WHATWG in the DOM Living Standard Spec (<https://dom.spec.whatwg.org/>).

With JavaScript you can interact with the DOM to:

- inspect the page structure
- access the page metadata and headers
- edit the CSS styling
- attach or remove event listeners
- edit any node in the page
- change any node attribute

and much more.

The main 2 objects provided by the DOM API, the ones you will interact the most with, are `document` and `window`.

The Window object

The `window` object represents the window that contains the DOM document.

`window.document` points to the `document` object loaded in the window.

Properties and methods of this object can be called without referencing `window` explicitly, because it represents the global object. So, the previous property `window.document` is usually called just `document`.

Properties

Here is a list of useful properties you will likely reference a lot:

- `console` points to the browser debugging console. Useful to print error messages or logging, using `console.log`, `console.error` and other tools (see the Browser DevTools article)
- `document` as already said, points to the `document` object, key to the DOM interactions you will perform
- `history` gives access to the **History API**
- `location` gives access to the Location interface (<https://developer.mozilla.org/en-US/docs/Web/API/Location>), from which you can determine the URL, the protocol, the hash and other useful information.
- `localStorage` is a reference to the Web Storage API `localStorage` object
- `sessionStorage` is a reference to the Web Storage API `sessionStorage` object

Methods

The `window` object also exposes useful methods:

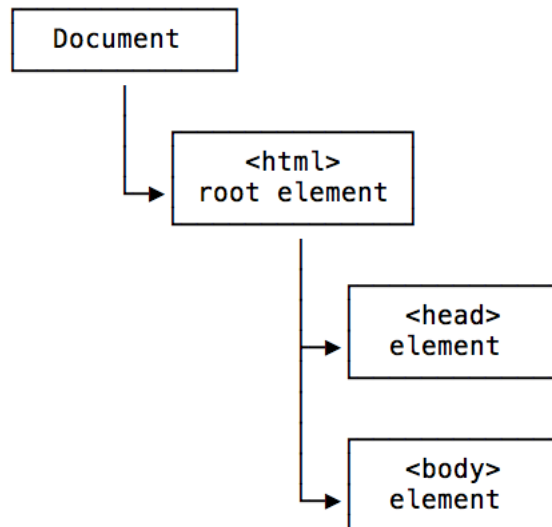
- `alert()` : which you can use to display alert dialogs
- `postMessage()` : used by the Channel Messaging API
- `requestAnimationFrame()` : used to perform animations in a way that's both performant and easy on the CPU
- `setInterval()` : call a function every `n` milliseconds, until the interval is cleared with `clearInterval()`
- `clearInterval()` : clears an interval created with `setInterval()`
- `setTimeout()` : execute a function after `n` milliseconds
- `setImmediate()` : execute a function as soon as the browser is ready
- `addEventListener()` : add an event listener to the document
- `removeEventListener()` : remove an event listener from the document

See the full reference of all the properties and methods of the `window` object at <https://developer.mozilla.org/en-US/docs/Web/API/Window>

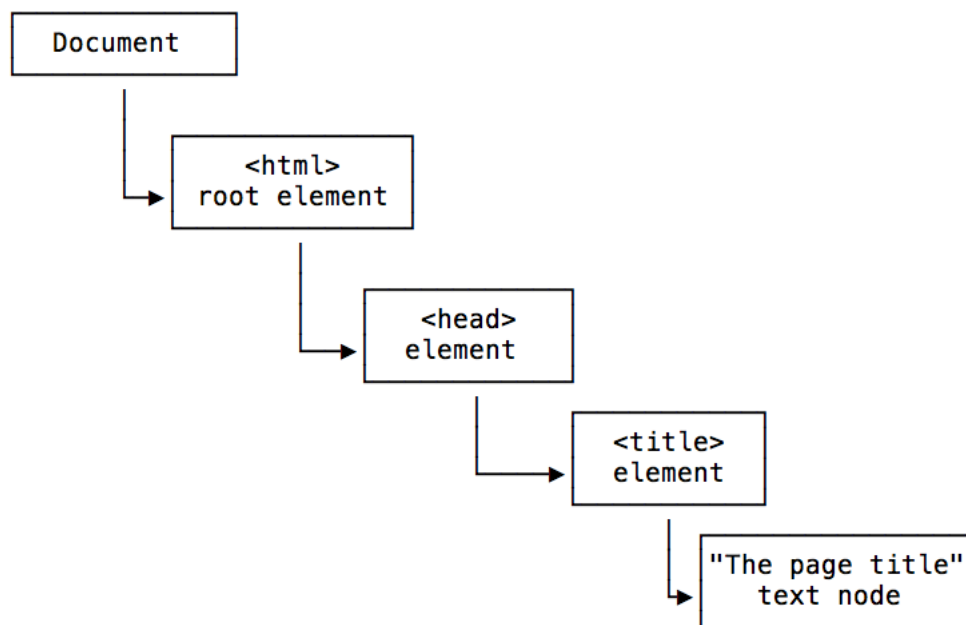
The Document object

The `document` object represents the DOM tree loaded in a window.

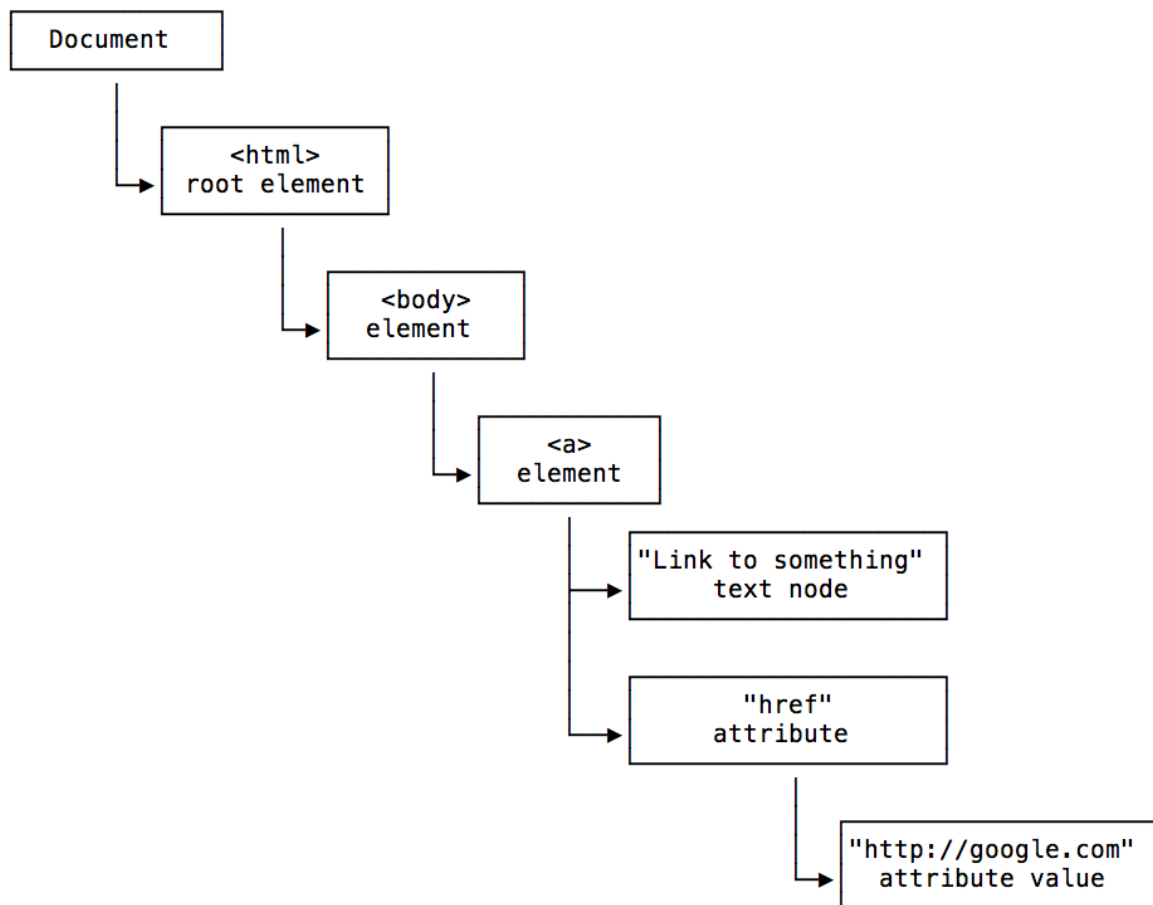
Here is a representation of a portion of the DOM pointing to the head and body tags:



Here is a representation of a portion of the DOM showing the head tag, containing a title tag with its value:



Here is a representation of a portion of the DOM showing the body tag, containing a link, with a value and the href attribute with its value:



The Document object can be accessed from `window.document`, and since `window` is the global object, you can use the shortcut `document` object directly from the browser console, or in your JavaScript code.

This Document object has a ton of properties and methods. The Selectors API methods are the ones you'll likely use the most:

- `document.getElementById()`
- `document.querySelector()`
- `document.querySelectorAll()`
- `document.getElementsByTagName()`
- `document.getElementsByClassName()`

You can get the document title using `document.title`, and the URL using `document.URL`. The referrer is available in `document.referrer`, the domain in `document.domain`.

From the `document` object you can get the body and head Element nodes (<https://developer.mozilla.org/en-US/docs/Web/API/Element>):

- `document.documentElement`: the Document node

- `document.body`: the `body` Element node
- `document.head`: the `head` Element node

```
> document.documentElement
< <html prefix="og: http://ogp.me/ns#" lang="en" class="gr__localhost">
  ▶ <head>...</head>
  ▶ <body data-gr-c-s-loaded="true">...</body>
  </html>
> document.head
< ▶ <head>...</head>
> document.body
< ▶ <body data-gr-c-s-loaded="true">...</body>
```

You can also get a list of all the element nodes of a particular type, like an `HTMLCollection` (<https://developer.mozilla.org/en-US/docs/Web/API/HTMLCollection>) of all the links using `document.links`, all the images using `document.images`, all the forms using `document.forms`.

The document cookies are accessible in `document.cookie`. The last modified date in `document.lastModified`.

You can do much more, even get *old school* and fill your scripts with `document.write()`, a method that was used a lot back in the early days of JavaScript to interact with the pages.

See the full reference of all the properties and methods of the `document` object at <https://developer.mozilla.org/en-US/docs/Web/API/Document>

Types of Nodes

There are different types of nodes, some of which you already saw in the example images above. The main ones you will see are:

- **Document**: the document Node, the start of the tree
- **Element**: an HTML tag
- **Attr**: an attribute of a tag
- **Text**: the text content of an Element or Attr Node
- **Comment**: an HTML comment
- **DocumentType**: the Doctype declaration

Traversing the DOM

The DOM is a tree of elements, with the Document node at the root, which points to the `html` Element node, which in turn points to its child element nodes `head` and `body`, and so on.

From each of those elements, you can navigate the DOM structure and move to different nodes.

Getting the parent

Every element has one and one single parent.

To get it, you can use `Node.parentNode` (<https://developer.mozilla.org/en-US/docs/Web/API/Node/parentNode>) or `Node.parentElement` (<https://developer.mozilla.org/en-US/docs/Web/API/Node/parentElement>) (where Node means a node in the DOM).

They are almost the same, except when ran on the `html` element: `parentNode` returns the parent of the specified node in the DOM tree, while `parentElement` returns the DOM node's parent Element, or null if the node either has no parent, or its parent isn't a DOM Element.

People most usually use `parentNode`.

Getting the children

To check if a Node has child nodes, use `Node.hasChildNodes()` which returns a boolean value.

To access all the Element Nodes children of a node, use `Node.childNodes`.

The DOM also exposes a `Node.children` method, but it will not just include Element nodes, but it includes also the white space between elements as Text nodes, which is not something you generally want.

The screenshot shows the Chrome DevTools interface. The DOM tree on the left highlights the `header.post-header` element. The breadcrumb path is `html.gr__localhost > body > div#wrapper > div#container > div.container > article.post-container > header.post-header`. The console on the right shows the following commands and results:

```
> $0.children
< ▶ HTMLCollection(3) [h1.post-title, h2.post-description, p.post-date]

> $0.childNodes
< ▶ NodeList(7) [text, h1.post-title, text, h2.post-description, text, p.post-date, text]

>
```

To get the first child Element Node, use `Node.firstElementChild`, and to get the last child Element Node, use `Node.lastElementChild`:

The screenshot shows the Chrome DevTools interface. The DOM tree on the left highlights the `article.post-container` element. The breadcrumb path is `html.gr__localhost > body > div#wrapper > div#container > div.container > article.post-container > header.post-header`. The console on the right shows the following commands and results:

```
> $0.lastElementChild
< ▶ <div class="post-content clearfix" itemprop="articleBody">...</div>

> $0.firstElementChild
< ▶ <header class="post-header">...</header>
```

The DOM also exposes `Node.firstChild` and `Node.lastChild`, with the difference that they do not “filter” the tree for Element nodes only, and they will also show empty Text nodes that indicate white space.

In short, to navigate children Element Nodes use

- `Node.childNodes`
- `Node.firstElementChild`
- `Node.lastElementChild`

Getting the siblings

In addition to getting the parent and the children, since the DOM is a tree you can also get the siblings of any Element Node.

You can do so using

- `Node.previousElementSibling`

- **`Node.nextElementSibling`**

The DOM also exposes `previousSibling` and `nextSibling`, but as their counterparts described above, they include white spaces as Text nodes, so you generally avoid them.

Editing the DOM

The DOM offers various methods to edit the nodes of the page and alter the document tree.

With

- `document.createElement()` : creates a new Element Node
- `document.createTextNode()` : creates a new Text Node

you can create new elements, and add them to the DOM elements you want, as children, by using `document.appendChild()`:

```
const div = document.createElement('div')
div.appendChild(document.createTextNode('Hello world!'))
```

- `first.removeChild(second)` removes the child node “second” from the node “first”.
- `document.insertBefore(newNode, existingNode)` inserts “newNode” as a sibling of “existingNode”, placing it before that in the DOM tree structure.
- `element.appendChild(newChild)` alters the tree under “element”, adding a new child Node “newChild” to it, after all the other children.
- `element.prepend(newChild)` alters the tree under “element”, adding a new child Node “newChild” to it, before other child elements. You can pass one or more child Nodes, or even a string which will be interpreted as a Text node.
- `element.replaceChild(existingChild, newChild)` alters the tree under “element”, replacing “existingChild” with a new Node “newChild”.
- `element.insertAdjacentElement(position, newElement)` inserts “newElement” in the DOM, positioned relatively to “element” depending on “position” parameter value. See the possible values (<https://developer.mozilla.org/en-US/docs/Web/API/Element/insertAdjacentElement>).
- `element.textContent = 'something'` changes the content of a Text node to “something”.

PROGRESSIVE WEB APPS

A Progressive Web App is an app that can provide additional features based on the device support, including offline capabilities, push notifications and almost native app look and speed, and local caching of resources

- Introduction
- What is a Progressive Web App
- Progressive Web Apps alternatives
 - Native Mobile Apps
 - Hybrid Apps
 - Apps built with React Native
- Progressive Web Apps features
 - Features
 - Benefits
 - Core concepts
- Service Workers
- The App Manifest
 - Example
- The App Shell
 - Caching

Introduction

Progressive Web Apps (PWA) are the latest trend of **mobile application development** using web technologies, at the time of writing (march 2018) work on Android and iOS devices with iOS 11.3 or higher, and macOS 10.13.4 or higher.

PWA is a term that identifies a bundle of techniques that have the goal of creating a better experience for web-based apps.

What is a Progressive Web App

A Progressive Web App is an app that *can* **provide additional features based on the device support**, providing offline capability, push notifications and almost native app look and speed, and local caching of resources.

This technique was originally introduced by Google in 2015, and proves to bring many advantages to both the developer and the users.

Developers have access to building **almost-first-class** applications using a web stack, which is always considerably easier and cheaper than building native applications, especially when considering the implications of building and maintaining cross-platform apps.

Devs can benefit from a **reduced installation friction**, at a time when having an app in the store does not actually bring anything in terms of discoverability for 99,99% of the apps, and Google search can provide the same benefits if not more.

A Progressive Web App is a website which is developed with certain technologies that make the mobile experience much more pleasant than a normal mobile-optimized website, to a point that it's almost working like a native app, as it offers the following features:

- Offline support
- Loads fast
- Is secure
- Is capable of emitting push notifications
- Has an immersive, full-screen user experience without the URL bar

Mobile platforms (Android at the time of writing, but it's not technically limited to that) offer an increasing support for Progressive Web Apps to the point of asking the user to **add the app to the home screen** when they detect a site a user is visiting is a PWA.

But first, a little clarification on the name. *Progressive Web App* can be a **confusing term**, and a good definition is *web apps that take advantage of modern browsers features (like web workers and the web app manifest) to let their mobile devices "upgrade" the app to the role of a first-class citizen app*.

Progressive Web Apps alternatives

How does a PWA stand compared to the alternatives when it comes to building a mobile experience?

Let's focus on the pros and cons of each, and let's see where PWAs are a good fit.

Native Mobile Apps

Native mobile apps are the most obvious way to build a mobile app. Objective-C or Swift on iOS, Java / Kotlin on Android and C# on Windows Phone.

Each platform has its own UI and UX conventions, and the native widgets provide the experience that the user expects. They can be deployed and distributed through the platform App Store.

The main pain point with native apps is that cross-platform development requires learning, mastering and keeping up to date with many different methodologies and best practices, so if for example you have a small team or even you're a solo developer building an app on 3 platforms, you need to spend a lot of time learning the technology but also the environment, manage different libraries, and use different workflows (for example, iCloud only works on iOS devices, there's no Android version).

Hybrid Apps

Hybrid applications are built using Web Technologies, but deployed to the App Store. In the middle sits a framework or some way to package the application so it's possible to send it for review to the traditional App Store.

Most common platforms are Phonegap, Xamarin, Ionic Framework, and many others, and usually what you see on the page is a WebView that essentially loads a local website.

The key aspect of Hybrid Apps is the **write once, run anywhere** concept, the different platform code is generated at build time, and you're building apps using JavaScript, HTML and CSS, which is amazing, and the device capabilities (microphone, camera, network, gps...) are exposed through JavaScript APIs.

The bad part of building hybrid apps is that unless you do a great job, you might settle on providing a common denominator, effectively creating an app that's sub-optimal on all platforms because the app is ignoring the platform-specific human-computer interaction guidelines.

Also, performance for complex views might suffer.

Apps built with React Native

React Native exposes the native controls of the mobile device through a JavaScript API, but you're effectively creating a native application, not embedding a website inside a WebView.

Their motto, to distinguish this approach from hybrid apps, is **learn once, write anywhere**, meaning that the approach is the same across platforms, but you're going to create completely separate apps in order to provide a great experience on each platform.

Performance is comparable to native apps, since what you build is essentially a native app, which is distributed through the App Store.

Progressive Web Apps features

In the last section you saw the main *competitors* of Progressive Web Apps. So how do PWAs stand compared to them, and what are their main features?

Remember, currently Progressive Web Apps are Android-only

Features

Progressive Web Apps have one thing that separates them completely from the above approaches: **they are not deployed to the app store..**

This is a key advantage, since the app store is beneficial if you have the reach and luck to be featured, which can make your app go viral, but unless you're in the 0,001% you're not going to get much benefits from having your little place on the App Store.

Progressive Web Apps are **discoverable using Search Engines**, and when a user gets to your site which has PWAs capabilities, **the browser in combination with the device asks the user if they want to install the app to the home screen**. This is huge because regular SEO can apply to your PWA, leading to much less reliance on paid acquisition.

Not being in the App Store means **you don't need the Apple or Google approval** to be in the users pockets, and you can release updates when you want, without having to go through the standard approval process which is typical of iOS apps.

PWAs are basically HTML5 applications / responsive websites on steroids, with some key technologies that were recently introduced that make some of the key features possible. If you remember the original iPhone came without the option to develop native apps, and developers

were told to develop HTML5 mobile apps, that could be installed to the home screen, but the tech back then was not ready for this.

Progressive Web Apps **run offline**.

The use of **service workers** allow the app to always have fresh content, and download it in the background, and provide support for **push notifications** to provide greater re-engagement opportunities.

Also, shareability makes for a much nicer experience for users that want to share your app, as they just need a URL.

Benefits

So why should users and developers care about Progressive Web Apps?

1. PWA are lighter. Native Apps can weight 200MB or more, while a PWA could be in the range of the KBs.
2. No native platform code
3. Lower the cost of acquisition (it's much more hard to convince a user to install an app than to visit a website to get the first-time experience)
4. Significant less effort is needed to build and release updates
5. Much more support for deep links than regular app-store apps

Core concepts

- **Responsive:** the UI adapts to the device screen size
- **App-like feel:** it doesn't feel like a website, but rather as an app as much as possible
- **Offline support:** it will use the device storage to provide offline experience
- **Installable:** the device browser prompts the user to install your app
- **Re-engaging:** push notifications help users re-discover your app once installed
- **Discoverable:** search engines and SEO optimization can provide a lot more users than the app store
- **Fresh:** the app updates itself and the content once online
- **Safe:** uses HTTPS
- **Progressive:** it will work on any device, even older one, even if with less features (e.g. just as a website, not installable)
- **Linkable:** easy to point to it, using URLs

Service Workers

Part of the Progressive Web App definition is that it must work offline.

Since the thing that allows the web app to work offline is the Service Worker, this implies that **Service Workers are a mandatory part of a Progressive Web App.**

WARNING: Service Workers are currently only supported by Chrome (Desktop and Android), Firefox and Opera. See <http://caniuse.com/#feat=serviceworkers> for updated data on the support.

TIP: Don't confuse Service Workers with Web Workers. They are a completely different thing.

A Service Worker is a JavaScript file that acts as a middleman between the web app and the network. Because of this it can provide cache services and speed the app rendering and improve the user experience.

Because of security reasons, only HTTPS sites can make use of Service Workers, and this is part of the reasons why a Progressive Web App must be served through HTTPS.

Service Workers are not available on the device the first time the user visits the app. What happens is that the first visit the web worker is installed, and then on subsequent visits to separate pages of the site will call this Service Worker.

Check out the complete guide to Service Workers

The App Manifest

The App Manifest is a JSON file that you can use to provide the device information about your Progressive Web App.

You add a link to the manifest in **all** your web site pages header:

```
<link rel="manifest" href="/manifest.webmanifest">
```

This file will tell the device how to set:

- The name and short name of the app
- The icons locations, in various sizes

- The starting URL, relative to the domain
- The default orientation
- The splash screen

Example

```
{
  "name": "The Weather App",
  "short_name": "Weather",
  "description": "Progressive Web App Example",
  "icons": [{
    "src": "images/icons/icon-128x128.png",
    "sizes": "128x128",
    "type": "image/png"
  }, {
    "src": "images/icons/icon-144x144.png",
    "sizes": "144x144",
    "type": "image/png"
  }, {
    "src": "images/icons/icon-152x152.png",
    "sizes": "152x152",
    "type": "image/png"
  }, {
    "src": "images/icons/icon-192x192.png",
    "sizes": "192x192",
    "type": "image/png"
  }, {
    "src": "images/icons/icon-256x256.png",
    "sizes": "256x256",
    "type": "image/png"
  }],
  "start_url": "/index.html?utm_source=app_manifest",
  "orientation": "portrait",
  "display": "standalone",
  "background_color": "#3E4EB8",
  "theme_color": "#2F3BA2"
}
```

The App Manifest is a W3C Working Draft, reachable at <https://www.w3.org/TR/appmanifest/>

The App Shell

The App Shell is not a technology but rather a **design concept** aimed at loading and rendering the web app container first, and the actual content shortly after, to give the user a nice app-like impression.

This is the equivalent of the Apple HIG (Human Interface Guidelines) suggestions to use a splash screen that resembles the user interface, to give a psychological hint that was found to lower the perception of the app taking a long time to load.

Caching

The App Shell is cached separately from the contents, and it's setup so that retrieving the shell building blocks from the cache takes very little time.

Find out more on the App Shell at <https://developers.google.com/web/updates/2015/11/app-shell>

SERVICE WORKERS

Service Workers are a key technology powering Progressive Web Applications on the mobile web. They allow caching of resources and push notifications, two of the main distinguishing features that up to now set native apps apart

- Introduction to Service Workers
- Background Processing
- Offline Support
 - Precache assets during installation
 - Caching network requests
- A Service Worker Lifecycle
 - Registration
 - Scope
 - Installation
 - Activation
- Updating a Service Worker
- Fetch Events
- Background Sync
- Push Events
- A note about console logs

Introduction to Service Workers

Service Workers are at the core of Progressive Web Apps, because they allow caching of resources and push notifications, two of the main distinguishing features that up to now set native apps apart.

A Service Worker is **programmable proxy** between your web page and the network, providing the ability to intercept and cache network requests, effectively **giving you the ability to create an offline-first experience for your app**.

It's a special kind of web worker, a JavaScript file associated with a web page which runs on a worker context, separate from the main thread, giving the benefit of being non-blocking - so computations can be done without sacrificing the UI responsiveness.

Being on a separate thread it has no DOM access, and no access to the Local Storage APIs and the XHR API as well, and it can only communicate back to the main thread using the **Channel Messaging API**.

Service Workers cooperate with other recent Web APIs:

- **Promises**
- **Fetch API**
- **Cache API**

And they are **only available on HTTPS** protocol pages, except for local requests, which do not need a secure connection for an easier testing.

Background Processing

Service Workers run independent of the application they are associated to, and they can receive messages when they are not active.

For example they can work:

- when your mobile application is **in the background**, not active
- when your mobile application is **closed**, so even not running in the background
- when **the browser is closed**, if the app is running in the browser

The main scenarios where Service Workers are very useful are:

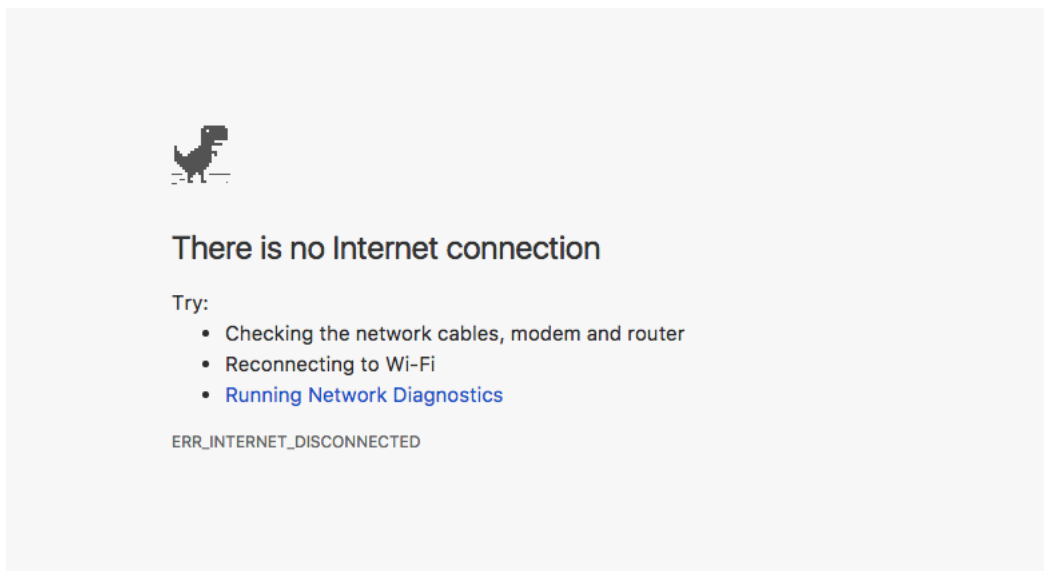
- they can be used as a **caching layer** to handle network requests, and cache content to be used when offline
- to allow **push notifications**

A Service Worker only runs when needed, and it's stopped when not used.

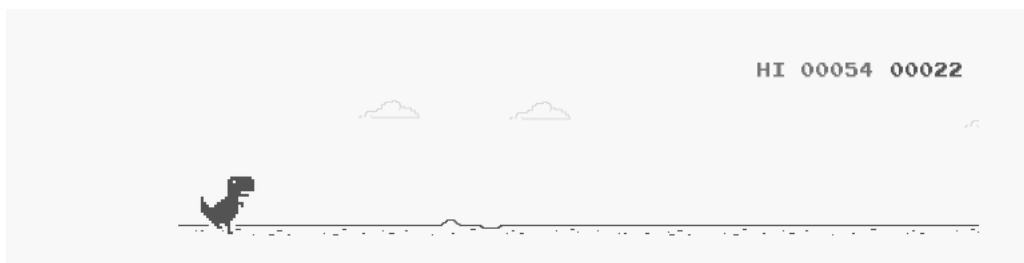
Offline Support

Traditionally the offline experience for web apps has been very poor. Without a network, often web mobile apps simply won't work, while native mobile apps have the ability to offer either a working version, or some kind of nice message.

This is not a nice message, but this is what web pages look like in Chrome without a network connection:



Possibly the only nice thing about this is that you get to play a free game by clicking the dinosaur, but it gets boring pretty quickly.



In the recent past the HTML5 AppCache already promised to allow web apps to cache resources and work offline, but its lack of flexibility and confusing behavior made it clear that it wasn't good enough for the job, failing its promises (and it's been discontinued (<https://html.spec.whatwg.org/multipage/offline.html#offline>)).

Service Workers are the new standard for offline caching.

Which kind of caching is possible?

Precache assets during installation

Assets that are reused throughout the application, like images, CSS, JavaScript files, can be installed the first time the app is opened.

This gives the base of what is called the **App Shell architecture**.

Caching network requests

Using the **Fetch API** we can edit the response coming from the server, determining if the server is not reachable and providing a response from the cache instead.

A Service Worker Lifecycle

A Service Worker goes through 3 steps to be fully working:

- Registration
- Installation
- Activation

Registration

Registration tells the browser where the server worker is, and it starts the installation in the background.

Example code to register a Service Worker placed in `worker.js`:

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('/worker.js')
      .then((registration) => {
        console.log('Service Worker registration completed with scope: ',
          registration.scope)
      }, (err) => {
        console.log('Service Worker registration failed', err)
      })
  })
} else {
  console.log('Service Workers not supported')
}
```

Even if this code is called multiple times, the browser will only perform the registration if the service worker is new, not registered previously, or if has been updated.

Scope

The `register()` call also accepts a scope parameter, which is a path that determines which part of your application can be controlled by the service worker.

It defaults to all files and subfolders contained in the folder that contains the service worker file, so if you put it in the root folder, it will have control over the entire app. In a subfolder, it will only control pages accessible under that route.

The example below registers the worker, by specifying the `/notifications/` folder scope.

```
navigator.serviceWorker.register('/worker.js', {  
  scope: '/notifications/'  
})
```

The `/` is important: in this case, the page `/notifications` won't trigger the Service Worker, while if the scope was

```
{  
  scope: '/notifications'  
}
```

it would have worked.

NOTE: The service worker cannot “up” itself from a folder: if its file is put under `/notifications`, it cannot control the `/` path or any other path that is not under `/notifications`.

Installation

If the browser determines that a service worker is outdated or has never been registered before, it will proceed to install it.

```
self.addEventListener('install', (event) => {  
  //...  
});
```

This is a good event to prepare the Service Worker to be used, by **initializing a cache**, and **cache the App Shell** and static assets using the **Cache API**.

Activation

The activation stage is the third step, once the service worker has been successfully registered and installed.

At this point, the service worker will be able to work with new page loads.

It cannot interact with pages already loaded, which means the service worker is only useful on the second time the user interacts with the app, or reloads one of the pages already open.

```
self.addEventListener('activate', (event) => {  
  //...  
});
```

A good use case for this event is to cleanup old caches and things associated with the old version but unused in the new version of the service worker.

Updating a Service Worker

To update a Service Worker you just need to change one byte into it, and when the register code is run, it will be updated.

Once a Service Worker is updated, it won't become available until all pages that were loaded with the old service worker attached are closed.

This ensures that nothing will break on the apps / pages already working.

Refreshing the page is not enough, as the old worker is still running and it's not been removed.

Fetch Events

A **fetch event** is fired when a resource is requested on the network.

This offers us the ability to **look in the cache** before making network requests.

For example the snippet below uses the **Cache API** to check if the request URL was already stored in the cached responses, and return the cached response if this is the case. Otherwise, it executes the fetch request and returns it.

```
self.addEventListener('fetch', (event) => {  
  event.respondWith(  
    caches.match(event.request)  
      .then((response) => {  
        if (response) { //entry found in cache  
          return response  
        }  
        return fetch(event.request)  
      })  
  })  
});
```

```
)  
)  
})
```

Background Sync

Background sync allows outgoing connections to be deferred until the user has a working network connection.

This is key to ensure a user can use the app offline, and take actions on it, and queue server-side updates for when there is a connection open, instead of showing an endless spinning wheel trying to get a signal.

```
navigator.serviceWorker.ready.then((swRegistration) => {  
  return swRegistration.sync.register('event1')  
});
```

This code listens for the event in the Service Worker:

```
self.addEventListener('sync', (event) => {  
  if (event.tag == 'event1') {  
    event.waitUntil(doSomething())  
  }  
})
```

`doSomething()` returns a promise. If it fails, another sync event will be scheduled to retry automatically, until it succeeds.

This also allows an app to update data from the server as soon as there is a working connection available.

Push Events

Service Workers enable web apps to provide native Push Notifications to users.

Push and Notifications are actually two different concepts and technologies, but combined to provide what we know as **Push Notifications**. Push provides the mechanism that allows a server to send information to a service worker, and Notifications are the way service workers can show information to the user.

Since Service Workers run even when the app is not running, they can listen for push events coming, and either provide user notifications, or update the state of the app.

Push events are initiated by a backend, through a browser push service, like the one provided by Firebase.

Here is an example of how the service worker can listen for incoming push events:

```
self.addEventListener('push', (event) => {
  console.log('Received a push event', event)

  const options = {
    title: 'I got a message for you!',
    body: 'Here is the body of the message',
    icon: '/img/icon-192x192.png',
    tag: 'tag-for-this-notification',
  }

  event.waitUntil(
    self.registration.showNotification(title, options)
  )
})
```

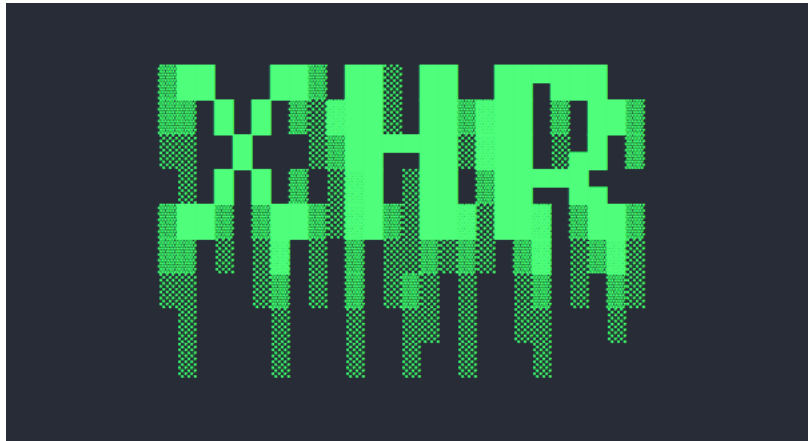
A note about console logs

If you have any console log statement (`console.log` and friends) in the Service Worker, make sure you turn on the `Preserve log` feature provided by the Chrome DevTools, or equivalent.

Otherwise, since the service worker acts before the page is loaded, and the console is cleared before loading the page, you won't see any log in the console.

XMLHttpRequest (XHR)

The introduction of XMLHttpRequest (XHR) in browsers have been a huge win for the Web Platform, in the mid 2000. Let's see how it works.



- Introduction
- An example XHR request
- Additional open() parameters
- onreadystatechange
- Aborting an XHR request
- Comparison with jQuery
- Comparison with Fetch
- Cross Domain Requests

Introduction

The introduction of XMLHttpRequest (XHR) in browsers have been a huge win for the Web Platform, in the mid 2000.

Things that now look normal, back in the day looked like coming from the future. I'm thinking about GMail or Google Maps, for example, all based in great part on XHR.

XHR was invented at Microsoft in the nineties, and became a de-facto standard as all browsers implemented it in the 2002-2006 period, and the W3C standardized XMLHttpRequest in 2006.

As it sometimes happen in the Web Platform, initially there were a few inconsistencies that made working with XHR quite different cross-browser.

Libraries like jQuery got a boost of popularity by providing an easy to use abstraction for developers, and in turn helped spread the usage of this technology.

An example XHR request

The following code creates an XMLHttpRequest (XHR) request object, and attaches a callback function that responds on the `onreadystatechange` event.

The xhr connection is set up to perform a GET request to `https://yoursite.com`, and it's started with the `send()` method:

```
const xhr = new XMLHttpRequest()
xhr.onreadystatechange = () => {
  if (xhr.readyState === 4) {
    xhr.status === 200 ? console.log(xhr.responseText) : console.error('error')
  }
}
xhr.open('GET', 'https://yoursite.com')
xhr.send()
```

Additional open() parameters

In the example above we just passed the method and the URL to the request.

We can specify the other HTTP methods of course (`get`, `post`, `head`, `put`, `delete`, `options`).

Other parameters let you specify a flag to make the request synchronous if set to false, and a set of credentials for HTTP authentication:

```
open(method, url, asynchronous, username, password)
```

onreadystatechange

The `onreadystatechange` is called multiple times during an XHR request. We explicitly ignore all the states other than `readyState === 4`, which means the request is done.

The states are

- 1 (OPENED): the request starts
- 2 (HEADERS_RECEIVED): the HTTP headers have been received
- 3 (LOADING): the response begins to download
- 4 (DONE): the response has been downloaded

Aborting an XHR request

An XHR request can be aborted by calling the `abort()` method on the `xhr` object.

Comparison with jQuery

With jQuery these lines can be translated to:

```
$.get('https://yoursite.com', data => {  
  console.log(data)  
}).fail((err) => {  
  console.error(err)  
})
```

Comparison with Fetch

With the Fetch API this is the equivalent code:

```
fetch('https://yoursite.com')  
  .then((data) => {  
    console.log(data)  
  })  
  .catch((err) => {  
    console.error(err)  
  })
```

Cross Domain Requests

Note that an XMLHttpRequest connection is subject to specific limits that are enforced for security reasons.

One of the most obvious is the enforcement of the same origin policy.

You cannot access resources on another server, *unless* the server explicitly supports this using CORS (Cross Origin Resource Sharing).

FETCH API

Learn all about the Fetch API, the modern approach to asynchronous network requests which uses Promises as a building block



- Introduction to the Fetch API
- Using Fetch
 - Catching errors
- Response Object
 - Metadata
 - headers
 - status
 - statusText
 - url
 - Body content
- Request Object
- Request headers
- POST Requests
- Fetch drawbacks
- How to cancel a fetch request

Introduction to the Fetch API

Since IE5 was released in 1998, we've had the option to make asynchronous network calls in the browser using XMLHttpRequest (XHR).

Quite a few years after this, GMail and other rich apps made heavy use of it, and made the approach so popular that it had to have a name: **AJAX**.

Working directly with the XMLHttpRequest has always been a pain and it was almost always abstracted by some library, in particular jQuery has its own helper functions built around it:

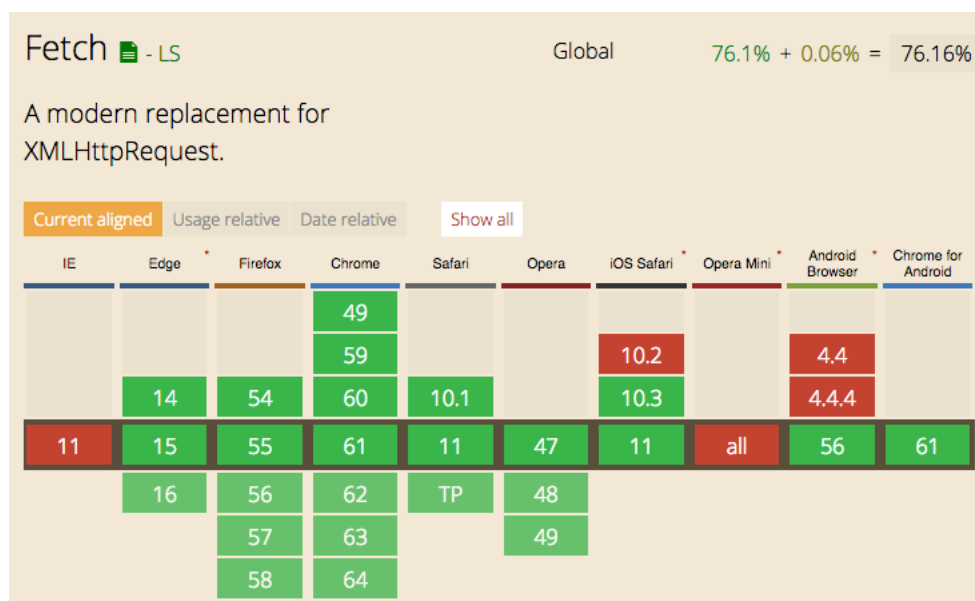
- `jQuery.ajax()`
- `jQuery.get()`
- `jQuery.post()`

and so on.

They had a huge impact on making this more accessible in particular with regards to making sure all worked on older browsers as well.

The **Fetch API**, has been standardized as a modern approach to asynchronous network requests, and uses **Promises** as a building block.

Fetch has a good support across the major browsers, except IE.



The polyfill <https://github.com/github/fetch> released by GitHub allows us to use `fetch` on any browser.

Using Fetch

Starting to use Fetch for GET requests is very simple:

```
fetch('/file.json')
```

and you're already using it: fetch is going to make an HTTP request to get the `file.json` resource on the same domain.

As you can see, the `fetch` function is available in the global `window` scope.

Now let's make this a bit more useful, let's actually see what the content of the file is:

```
fetch('./file.json')
  .then(response => response.json())
  .then(data => console.log(data))
```

Calling `fetch()` returns a promise. We can then wait for the promise to resolve by passing a handler with the `then()` method of the promise.

That handler receives the return value of the `fetch` promise, a **Response** object.

We'll see this object in details in the next section.

Catching errors

Since `fetch()` returns a promise, we can use the `catch` method of the promise to intercept any error occurring during the execution of the request, and the processing done in the `then` callbacks:

```
fetch('./file.json')
  .then(response => {
    //...
  })
  .catch(err => console.error(err))
```

Response Object

The Response Object returned by a `fetch()` call contains all the information about the request and the response of the network request.

Metadata

headers

Accessing the `headers` property on the `response` object gives you the ability to look into the HTTP headers returned by the request:

```
fetch('./file.json')
  .then(response => {
    console.log(response.headers.get('Content-Type'))
    console.log(response.headers.get('Date'))
  })
```

```
fetch('https://dog.ceo/api/breeds/image/random')
  .then(response => {
    console.log(response.headers.get('Content-Type'))
    console.log(response.headers.get('Date'))
  })
< ▶ Promise {<pending>}
application/json
Sun, 22 Apr 2018 05:05:00 GMT
```

status

This property is an integer number representing the HTTP response status.

- 101, 204, 205, or 304 is a null body status
- 200 to 299, inclusive, is an OK status (success)
- 301, 302, 303, 307, or 308 is a redirect

```
fetch('./file.json')
  .then(response => console.log(response.status))
```

statusText

`statusText` is a property representing the status message of the response. If the request is successful, the status is `OK`.

```
fetch('./file.json')
  .then(response => console.log(response.statusText))
```

url

`url` represents the full URL of the property that we fetched.

```
fetch('./file.json')
  .then(response => console.log(response.url))
```

Body content

A response has a body, accessible using the `text()` or `json()` methods, which return a promise.

```
fetch('./file.json')
  .then(response => response.text())
  .then(body => console.log(body))
```

```
fetch('./file.json')
  .then(response => response.json())
  .then(body => console.log(body))
```

```
fetch('https://dog.ceo/api/breeds/image/random')
  .then(response => response.text())
  .then(body => console.log(body))
< ▶ Promise {<pending>}
{"status":"success","message":"https://images.dog.ceo/breeds/terrier-scottish/n02097298_8392.jpg"}
>
fetch('https://dog.ceo/api/breeds/image/random')
  .then(response => response.json())
  .then(body => console.log(body))
< ▶ Promise {<pending>}
▶ {status: "success", message: "https://images.dog.ceo/breeds/dhole/n02115913_257.jpg"}
```

The same can be written using the ES2017 async functions:

```
(async () => {
  const response = await fetch('./file.json')
  const data = await response.json()
  console.log(data)
})()
```

Request Object

The Request object represents a resource request, and it's usually created using the `new Request()` API.

Example:

```
const req = new Request('/api/todos')
```

The Request object offers several read-only properties to inspect the resource request details, including

- `method`: the request's method (GET, POST, etc.)
- `url`: the URL of the request.
- `headers`: the associated Headers object of the request
- `referrer`: the referrer of the request
- `cache`: the cache mode of the request (e.g., default, reload, no-cache).

And exposes several methods including `json()`, `text()` and `formData()` to process the body of the request.

The full API can be found at <https://developer.mozilla.org/docs/Web/API/Request>

Request headers

Being able to set the HTTP request header is essential, and `fetch` gives us the ability to do this using the Headers object:

```
const headers = new Headers();
headers.append('Content-Type', 'application/json')
```

or more simply

```
const headers = new Headers({
  'Content-Type': 'application/json'
})
```

To attach the headers to the request, we use the Request object, and pass it to `fetch()` instead of simply passing the URL.

Instead of:

```
fetch('./file.json')
```

we do

```
const request = new Request('./file.json', {
  headers: new Headers({
    'Content-Type': 'application/json'
  })
})
```

```
})  
fetch(request)
```

The Headers object is not limited to setting value, but we can also query it:

```
headers.has('Content-Type');  
headers.get('Content-Type');
```

and we can delete a header that was previously set:

```
headers.delete('X-My-Custom-Header');
```

POST Requests

Fetch also allows to use any other HTTP method in your request: POST, PUT, DELETE or OPTIONS.

Specify the method in the method property of the request, and pass additional parameters in the header and in the request body:

Example of a POST request:

```
const options = {  
  method: 'post',  
  headers: {  
    "Content-type": "application/x-www-form-urlencoded; charset=UTF-8"  
  },  
  body: 'foo=bar&test=1'  
}  
  
fetch(url, options)  
.catch((err) => {  
  console.error('Request failed', err)  
})
```

Fetch drawbacks

While it's a great improvement over XHR, especially considering its Service Workers integration, Fetch currently has **no way to abort a request** once it's done. With Fetch it's also **hard to measure upload progress**.

If you need those things in your app, the Axios JavaScript library might be a better fit.

How to cancel a fetch request

For a few years after `fetch` was introduced, there was no way to abort a request once opened.

Now we can, thanks to the introduction of `AbortController` and `AbortSignal`, a generic API to notify **abort** events

You integrate this API by passing a signal as a fetch parameter:

```
const controller = new AbortController()
const signal = controller.signal

fetch('./file.json', { signal })
```

You can set a timeout that fires an abort event 5 seconds after the fetch request has started, to cancel it:

```
setTimeout(() => controller.abort(), 5 * 1000)
```

Conveniently, if the fetch already returned, calling `abort()` won't cause any error.

When an abort signal occurs, fetch will reject the promise with a `DOMException` named `AbortError`:

```
fetch('./file.json', { signal })
  .then(response => response.text())
  .then(text => console.log(text))
  .catch(err => {
    if (err.name === 'AbortError') {
      console.error('Fetch aborted')
    } else {
      console.error('Another error', err)
    }
  })
```

THE CHANNEL MESSAGING API

The Channel Messaging API allows iframes and workers to communicate with the main document thread, by passing messages

- Introduction to Channel Messaging API
 - How it works
- An example with an iframe
- An example with a Service Worker

Introduction to Channel Messaging API

Given two scripts running in the same document, but in a different context, the Channel Messaging API allows them to communicate by passing messages through a channel.

This use case involves communication between

- the document and an iframe
- two iframes
- two documents

How it works

Calling `new MessageChannel()` a message channel is initialized.

The channel has 2 properties, called

- `port1`
- `port2`

Those properties are a `MessagePort` object. `port1` is the port used by the part that created the channel, and `port2` is the port used by the channel receiver (by the way, the channel is bidirectional, so the receiver can send back messages as well).

Sending the message is done through the

```
otherWindow.postMessage()
```

method, where `otherWindow` is the other browsing context.

It accepts a message, an origin and the port.

A message can be a JavaScript value like strings, numbers, and some data structures are supported, namely

- File
- Blob
- FileList
- ArrayBuffer

“Origin” is a URI (e.g. `https://example.org`). You can use `'*` to allow less strict checking, or specify a domain, or specify `'/'` to set a same-domain target, without needing to specify which domain is it.

The other browsing context listens for the message using `MessagePort.onmessage`, and it can respond back by using `MessagePort.postMessage`.

A channel can be closed by invoking `MessagePort.close`.

Let’s see a practical example in the next lesson.

An example with an iframe

Here’s an example of a communication happening between a document and an iframe embedded into it.

The main document defines an `iframe` and a `span` where we’ll print a message that’s sent from the `iframe` document. As soon as the `iframe` document is loaded, we send it a message on the `channel` we created.

```
<!DOCTYPE html>
<html>
  <body>
    <iframe src="iframe.html" width="500" height="500"></iframe>
```

```

    <span></span>
  </body>
  <script>
    const channel = new MessageChannel()
    const display = document.querySelector('span')
    const iframe = document.querySelector('iframe')

    iframe.addEventListener('load', () => {
      iframe.contentWindow.postMessage('Hey', '*', [channel.port2])
    }, false)

    channel.port1.onmessage = (e) => {
      para.innerHTML = e.data
    }
  </script>
</html>

```

The iframe page source is even simpler:

```

<!DOCTYPE html>
<html>
  <script>
    window.addEventListener("message", (event) => {
      if (event.origin !== "http://example.org:8080") {
        return
      }

      // process

      // send a message back
      event.ports[0].postMessage('Message back from the iframe')
    }, false)
  </script>
</html>

```

As you can see we don't even need to initialize a channel, because the `window.onmessage` handler is automatically run when the message is received from the container page.

`e` is the event that's sent, and is composed by the following properties:

- `data`: the object that's been sent from the other window
- `origin`: the origin URI of the window that sent the message
- `source`: the window object that sent the message

Always verify the origin of the message sender.

`e.ports[0]` is the way we reference `port2` in the iframe, because `ports` is an array, and the port was added as the first element.

An example with a Service Worker

A Service Worker is an event-driven worker, a JavaScript file associated with web page. Check out the Service Workers guide to know more about them.

What's important to know is that Service Workers are isolated from the main thread, and we must communicate with them using messages.

This is how a script attached to the main document will handle sending messages to the Service Worker:

```
// `worker` is the service worker already instantiated

const messageChannel = new MessageChannel()
messageChannel.port1.addEventListener('message', (event) => {
  console.log(event.data)
})
worker.postMessage(data, [messageChannel.port2])
```

In the Service Worker code, we add an event listener for the `message` event:

```
self.addEventListener('message', (event) => {
  console.log(event.data)
})
```

And it can send messages back by posting a message to `messageChannel.port2`, with

```
self.addEventListener('message', (event) => {
  event.ports[0].postMessage(data)
})
```

More on the inner workings of Service Workers in the Service Workers guide.

CACHE API

The Cache API is part of the Service Worker specification, and is a great way to have more power on resources caching.

- Introduction
- Detect if the Cache API is available
- Initialize a cache
- Add items to the cache
 - `cache.add()`
 - `cache.addAll()`
 - Manually fetch and add
- Retrieve an item from the cache
- Get all the items in a cache
- Get all the available caches
- Remove an item from the cache
- Delete a cache

Introduction

The Cache API is part of the Service Worker specification, and is a great way to have more power on resources caching.

It allows you to **cache URL-addressable resources**, which means assets, web pages, HTTP APIs responses.

It's **not** meant to cache individual chunks of data, which is the task of the **IndexedDB API**.

It's currently available in Chrome ≥ 40 , Firefox ≥ 39 , and Opera ≥ 27 .

Internet Explorer and Safari still do not support it, while Edge has support for that is in preview release.

Mobile support is good on Android, supported on the Android Webview and in Chrome for Android, while on iOS it's only available to Opera Mobile and Firefox Mobile users.

Detect if the Cache API is available

The Cache API is exposed through the `caches` object. To detect if the API is implemented in the browser, just check for its existence using:

```
if ('caches' in window) {  
  //ok  
}
```

Initialize a cache

Use the `caches.open` API, which returns a promise with a **cache object** ready to be used:

```
caches.open('mycache').then((cache) => {  
  // you can start using the cache  
})
```

If the cache does not exist yet, `caches.open` creates it.

Add items to the cache

The `cache` object exposes two methods to add items to the cache: `add` and `addAll`.

`cache.add()`

`add` accepts a single URL, and when called it fetches the resource and caches it.

```
caches.open('mycache').then((cache) => {  
  cache.add('/api/todos')  
})
```

To allow more control on the fetch, instead of a string you can pass a **Request** object, part of the Fetch API specification:

```
caches.open('mycache').then((cache) => {  
  const options = {  
    // the options  
  }  
  cache.add(new Request('/api/todos', options))  
})
```

`cache.addAll()`

`addAll` accepts an array, and returns a promise when all the resources have been cached.

```
 caches.open('mycache').then((cache) => {
   cache.addAll(['/api/todos', '/api/todos/today']).then(() => {
     //all requests were cached
   })
 })
```

Manually fetch and add

`cache.add()` automatically fetches a resource, and caches it.

The Cache API offers a more granular control on this via `cache.put()`. You are responsible for fetching the resource and then telling the Cache API to store a response:

```
const url = '/api/todos'
fetch(url).then((res) => {
  return caches.open('mycache').then((cache) => {
    return cache.put(url, res)
  })
})
```

Retrieve an item from the cache

`cache.match()` returns a **Response** object which contains all the information about the request and the response of the network request

```
 caches.open('mycache').then((cache) => {
   cache.match('/api/todos').then((res) => {
     //res is the Response Object
   })
 })
```

Get all the items in a cache

```
 caches.open('mycache').then((cache) => {
   cache.keys().then((cachedItems) => {
     //
   })
 })
```

cachedItems is an array of **Request** objects, which contain the URL of the resource in the `url` property.

Get all the available caches

The `caches.keys()` method lists the keys of every cache available.

```
caches.keys().then((keys) => {  
  // keys is an array with the list of keys  
})
```

Remove an item from the cache

Given a `cache` object, its `delete()` method removes a cached resource from it.

```
caches.open('mycache').then((cache) => {  
  cache.delete('/api/todos')  
})
```

Delete a cache

The `caches.delete()` method accepts a cache identifier and when executed it wipes the cache and its cached items from the system.

```
caches.delete('mycache').then(() => {  
  // deleted successfully  
})
```

THE PUSH API

The Push API allows a web app to receive messages pushed by a server, even if the web app is not currently open in the browser or not running on the device.



- What is the Push API
- What can you do with it
- How it works
 - Overview

- Getting the user's permission
 - Check if Service Workers are supported
 - Check if the Push API is supported
 - Register a Service Worker
 - Request permission from the user
 - Subscribe the user and get the PushSubscription object
 - Send the PushSubscription object to your server
- How the Server side works
 - Registering a new client subscription
 - Sending a Push message
 - In the real world...
- Receive a Push event
 - Displaying a notification

What is the Push API

The Push API is a recent addition to the browser APIs, and it's currently supported by Chrome (Desktop and Mobile), Firefox and Opera since 2016. See more about the current state of browsers support at <https://caniuse.com/#feat=push-api>

IE, Edge do not support it yet, and Safari has its own implementation (<https://developer.apple.com/notifications/safari-push-notifications/>) .

Since Chrome and Firefox support it, approximately 60% of the users browsing on the desktop have access to it, so it's quite *safe* to use.

What can you do with it

You can send messages to your users, pushing them from the server to the client, even when the user is not browsing the site.

This lets you deliver notifications and content updates, giving you the ability to have a more engaged audience.

This is huge because one of the missing pillars of the mobile web, compared to native apps, was the ability to receive notifications, along with offline support.

How it works

Overview

When a user visits your web app, you can trigger a panel asking permission to send updates. A Service Worker is installed, and operating in the background listens for a Push Event.

Push and Notifications are a separate concept and API, sometimes mixed because of the push notifications term used in iOS. Basically, the Notifications API is invoked when a push event is received using the Push API.

Your **server** sends the notification to the client, and the Service Worker, if given permission, receives a **push event**. The Service Worker reacts to this event by **triggering a notification**.

Getting the user's permission

The first step in working with the Push API is getting the user's permission to receive data from you.

Many sites implement this panel badly, showing it on the first page load. The user is not yet convinced your content is good, and they will deny the permission. Do it wisely.

There are 6 steps:

1. Check if Service Workers are supported
2. Check if the Push API is supported
3. Register a Service Worker
4. Request permission from the user
5. Subscribe the user and get the PushSubscription object
6. Send the PushSubscription object to your server

Check if Service Workers are supported

```
if (!('serviceWorker' in navigator)) {  
  // Service Workers are not supported. Return  
  return  
}
```

Check if the Push API is supported

```
if (!('PushManager' in window)) {  
  // The Push API is not supported. Return  
  return  
}
```

Register a Service Worker

This code register the Service Worker located in the `worker.js` file placed in the domain root:

```
window.addEventListener('load', () => {  
  navigator.serviceWorker.register('/worker.js')  
    .then((registration) => {  
      console.log('Service Worker registration completed with scope: ',  
        registration.scope)  
    }, (err) => {  
      console.log('Service Worker registration failed', err)  
    })  
})
```

To know more about how Service Workers work in details, check out the [Service Workers guide](#).

Request permission from the user

Now that the Service worker is registered, you can request the permission.

The API to do this changed over time, and it went from accepting a callback function as a parameter to returning a Promise, breaking the backward and forward compatibility, and we need to do **both** as we don't know which approach is implemented by the user's browser.

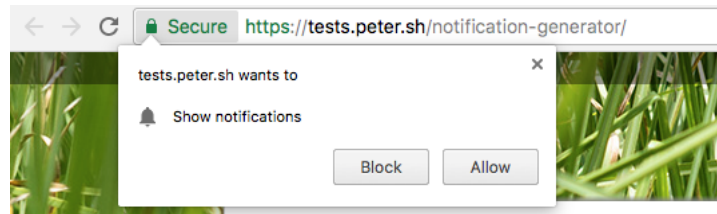
The code is the following, calling `Notification.requestPermission()`.

```
const askPermission = () => {  
  return new Promise((resolve, reject) => {  
    const permissionResult = Notification.requestPermission((result) => {  
      resolve(result)  
    })  
    if (permissionResult) {  
      permissionResult.then(resolve, reject)  
    }  
  })  
  .then((permissionResult) => {  
    if (permissionResult !== 'granted') {  
      throw new Error('Permission denied')  
    }  
  })  
}
```

```
}  
}
```

The `permissionResult` value is a string, that can have the value of: - granted - default - denied

This code causes the browser to show the permission dialogue:



If the user clicks Block, you won't be able to ask for the user's permission any more, unless they manually go and unblock the site in an advanced settings panel in the browser (very unlikely to happen).

Subscribe the user and get the PushSubscription object

If the user gave us permission, we can subscribe it and by calling `registration.pushManager.subscribe()`.

```
const APP_SERVER_KEY = 'XXX'  
  
window.addEventListener('load', () => {  
  navigator.serviceWorker.register('/worker.js')  
  .then((registration) => {  
    askPermission().then(() => {  
      const options = {  
        userVisibleOnly: true,  
        applicationServerKey: urlBase64ToUint8Array(APP_SERVER_KEY)  
      }  
      return registration.pushManager.subscribe(options)  
    }).then((pushSubscription) => {  
      // we got the pushSubscription object  
    })  
  }, (err) => {  
    console.log('Service Worker registration failed', err)  
  })  
})
```

`APP_SERVER_KEY` is a string - called *Application Server Key* or *VAPID key* - that identifies the applications's public key, part of a public / private key pair.

It will be used as part of the validation that for security reasons occurs to make sure you (and only you, not someone else) can send a push message back to the user.

Send the PushSubscription object to your server

In the previous snippet we got the `pushSubscription` object, which contains all we need to send a push message to the user. We need to send this information to our server, so we're able to send notifications later on.

We first create a JSON representation of the object

```
const subscription = JSON.stringify(pushSubscription)
```

and we can post it to our server using the Fetch API:

```
const sendToServer = (subscription) => {  
  return fetch('/api/subscription', {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json'  
    },  
    body: JSON.stringify(subscription)  
  })  
  .then((res) => {  
    if (!res.ok) {  
      throw new Error('An error occurred')  
    }  
    return res.json()  
  })  
  .then((resData) => {  
    if (!(resData.data && resData.data.success)) {  
      throw new Error('An error occurred')  
    }  
  })  
}  
  
sendToServer(subscription)
```

Server-side, the `/api/subscription` endpoint receives the POST request and can store the subscription information into its storage.

How the Server side works

So far we only talked about the client-side part: getting a user's permission to be notified in the future.

What about the server? What should it do, and how should it interact with the client?

These server-side examples uses Express.js (<http://expressjs.com/>) as the base HTTP framework, but you can write a server-side Push API handler in any language or framework

Registering a new client subscription

When the client sends a new subscription, remember we used the `/api/subscription` HTTP POST endpoint, sending the `PushSubscription` object details in JSON format, in the body.

We initialize Express.js:

```
const express = require('express')
const app = express()
```

This utility function makes sure the request is valid, has a body and an endpoint property, otherwise it returns an error to the client:

```
const isValidSaveRequest = (req, res) => {
  if (!req.body || !req.body.endpoint) {
    res.status(400)
    res.setHeader('Content-Type', 'application/json')
    res.send(JSON.stringify({
      error: {
        id: 'no-endpoint',
        message: 'Subscription must have an endpoint'
      }
    }))
    return false
  }
  return true
}
```

The next utility function saves the subscription to the database, returning a promise resolved when the insertion completed (or failed). The `insertToDatabase` function is a placeholder, we're not going into those details here:

```
const saveSubscriptionToDatabase = (subscription) => {
  return new Promise((resolve, reject) => {
    insertToDatabase(subscription, (err, id) => {
      if (err) {
        reject(err)
        return
      }

      resolve(id)
    })
  })
}
```

```
    })  
  })  
}
```

We use those functions in the POST request handler below. We check if the request is valid, then we save the request and then we return a `data.success: true` response back to the client, or an error:

```
app.post('/api/subscription', (req, res) => {  
  if (!isValidSaveRequest(req, res)) {  
    return  
  }  
  
  saveSubscriptionToDatabase(req, res.body)  
  .then((subscriptionId) => {  
    res.setHeader('Content-Type', 'application/json')  
    res.send(JSON.stringify({ data: { success: true } }))  
  })  
  .catch((err) => {  
    res.status(500)  
    res.setHeader('Content-Type', 'application/json')  
    res.send(JSON.stringify({  
      error: {  
        id: 'unable-to-save-subscription',  
        message: 'Subscription received but failed to save it'  
      }  
    }))  
  })  
})  
  
app.listen(3000, () => {  
  console.log('App listening on port 3000')  
})
```

Sending a Push message

Now that the server has registered the client in its list, we can send it Push messages. Let's see how that works by creating an example code snippet that fetches all subscriptions and sends a Push message to all of them at the same time.

We use a library because the **Web Push protocol**

(<https://developers.google.com/web/fundamentals/push-notifications/web-push-protocol>) is **complex**, and a lib allows us to abstract away a lot of low level code that makes sure we can work safely and correctly handle any edge case.

This example uses the `web-push` Node.js library (<https://github.com/web-push-libs/web-push>) to handle sending the Push message

We first initialize the `web-push` lib, and we generate a tuple of private and public keys, and set them as the VAPID details:

```
const webpush = require('web-push')
const vapidKeys = webpush.generateVAPIDKeys()

const PUBLIC_KEY = 'XXX'
const PRIVATE_KEY = 'YYY'

const vapidKeys = {
  publicKey: PUBLIC_KEY,
  privateKey: PRIVATE_KEY
}

webpush.setVapidDetails(
  'mailto:my@email.com',
  vapidKeys.publicKey,
  vapidKeys.privateKey
)
```

Then we set up a `triggerPush()` method, responsible for sending the push event to a client. It just calls `webpush.sendNotification()` and catches any error. If the return error HTTP status code is 410 (<https://developer.mozilla.org/docs/Web/HTTP/Status/410>), which means **gone**, we delete that subscriber from the database.

```
const triggerPush = (subscription, dataToSend) => {
  return webpush.sendNotification(subscription, dataToSend)
    .catch((err) => {
      if (err.statusCode === 410) {
        return deleteSubscriptionFromDatabase(subscription._id)
      } else {
        console.log('Subscription is no longer valid: ', err)
      }
    })
}
```

We don't implement getting the subscriptions from the database, but we leave it as a stub:

```
const getSubscriptionsFromDatabase = () => {
  //stub
}
```

The meat of the code is the callback of the POST request to the `/api/push` endpoint:

```
app.post('/api/push', (req, res) => {
  return getSubscriptionsFromDatabase()
    .then((subscriptions) => {
      let promiseChain = Promise.resolve()
      for (let i = 0; i < subscriptions.length; i++) {
```

```

    const subscription = subscriptions[i]
    promiseChain = promiseChain.then(() => {
      return triggerPush(subscription, dataToSend)
    })
  }
  return promiseChain
})
.then(() => {
  res.setHeader('Content-Type', 'application/json')
  res.send(JSON.stringify({ data: { success: true } }))
})
.catch((err) => {
  res.status(500)
  res.setHeader('Content-Type', 'application/json')
  res.send(JSON.stringify({
    error: {
      id: 'unable-to-send-messages',
      message: `Failed to send the push ${err.message}`
    }
  }))
})
})
})

```

What the above code does is: it gets all the subscriptions from the database, then it iterates on them, and it calls the `triggerPush()` function we explained before.

Once the subscriptions are done, we return a successful JSON response, unless an error occurred and we return a 500 error.

In the real world...

It's unlikely that you'll set up your own Push server unless you have a very special use case, or you just want to learn the tech or you like to DIY. Instead, you usually want to use platforms such as OneSignal (<https://onesignal.com>) which transparently handle Push events to all kind of platforms, Safari and iOS included, for free.

Receive a Push event

When a Push event is sent from the server, how does the client get it?

It's a normal JavaScript event listener, on the `push` event, which runs inside a Service Worker:

```

self.addEventListener('push', (event) => {
  // data is available in event.data
})

```

`event.data` contains the `PushMessageData`

(<https://developer.mozilla.org/docs/Web/API/PushMessageData>) object which exposes methods to retrieve the push data sent by the server, in the format you want:

- **`arrayBuffer()`** : as an `ArrayBuffer` object
- **`blob()`**: as a `Blob` object
- **`json()`**: parsed as json
- **`text()`**: plain text

You'll normally use `event.data.json()`.

Displaying a notification

Here we intersect a bit with the Notifications API, but for a good reason, as one of the main use cases of the Push API is to display notifications.

Inside our `push` event listener in the Service Worker, we need to display the notification to the user, and to tell the event to wait until the browser has shown it before the function can terminate. We extend the event lifetime until the browser has done displaying the notification (until the promise has been resolved), otherwise the Service Worker could be stopped in the middle of your processing:

```
self.addEventListener('push', (event) => {  
  const promiseChain = self.registration.showNotification('Hey!')  
  event.waitUntil(promiseChain)  
})
```

More on notifications in the Notifications API Guide.

NOTIFICATION API

The Notifications API is responsible for showing the user system notifications. It's the interface that browsers expose to the developer to allow showing messages to the user, with their permission, even if the web site is not open in the browser

- Introduction to the Notification API
- Permissions
- Create a notification
 - Add a body
 - Add an image
- Close a notification

Introduction to the Notification API

The **Notifications API** is the interface that browsers expose to the developer to allow **showing messages to the user**, with their permission, even if the web site / web app is not open in the browser.

Those messages are consistent and native, which means that the receiving person is used to the UI and UX of them, being system-wide and not specific to your site.

In combination with the Push API this technology can be a successful way to increase user engagement and to enhance the capabilities of your app.

The Notifications API interacts heavily with Service Workers, as they are required for Push Notifications. You can use the Notifications API without Push, but its use cases are limited.

```
if (window.Notification && Notification.permission !== "denied") {  
  Notification.requestPermission((status) => {  
    // status is "granted", if accepted by user  
    var n = new Notification('Title', {  
      body: 'I am the body text!',  
      icon: '/path/to/icon.png' // optional  
    })  
  })  
}
```

```
}  
  })  
}
```

```
n.close()
```

Permissions

To show a notification to the user, you must have permission to do so.

The `Notification.requestPermission()` method call requests this permission.

You can call

```
Notification.requestPermission()
```

in this very simple form, and it will show a permission granting panel - unless permission was already granted before.

To do something when the user interacts (allows or denies), you can attach a processing function to it:

```
const process = (permission) => {  
  if (permission === "granted") {  
    // ok we can show the permission  
  }  
}  
  
Notification.requestPermission((permission) => {  
  process(permission)  
}).then((permission) => {  
  process(permission)  
})
```

See how we pass in a callback and also we expect a promise. This is because of different implementations of `Notification.requestPermission()` made in the past, which we now must support as we don't know in advance which version is running in the browser. So to keep things in a single location I extracted the permission processing in the `process()` function.

In both cases that function is passed a `permission` string which can have one of these values:

- `granted`: the user accepted, we **can** show a permission
- `denied`: the user denied, we **can't** show any permission

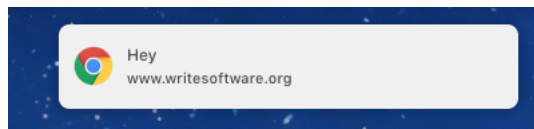
Those values can also be retrieved checking the `Notification.permission` property, which - if the user already granted permissions - evaluates to `granted` or `denied`, but if you haven't called `Notification.requestPermission()` yet, it will resolve to `default`.

Create a notification

The `Notification` object exposed by the `window` object in the browser allows you to create a notification and to customize its appearance.

Here is the simplest example, that works after you asked for permissions:

```
Notification.requestPermission()  
new Notification('Hey')
```

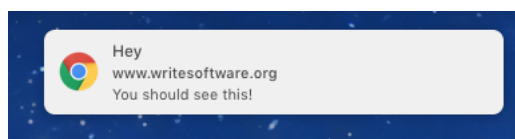


You have a few options to customize the notification.

Add a body

First, you can add a body, which is usually shown as a single line:

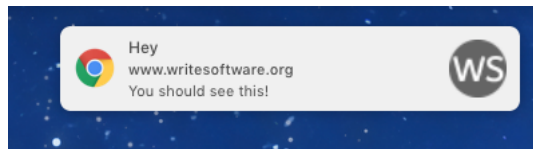
```
new Notification('Hey', {  
  body: 'You should see this!'  
})
```



Add an image

You can add an icon property:

```
new Notification('Hey', {  
  body: 'You should see this!'  
  icon: '/user/themes/writesoftware/favicon.ico'  
})
```



More customization options, with platform-specific properties, can be found at <https://developer.mozilla.org/docs/Web/API/Notification>

Close a notification

You might want to close a notification once you opened it.

To do so, create a reference to the notification you open:

```
const n = new Notification('Hey')
```

and then you can close it later, using:

```
n.close()
```

or with a timeout:

```
setTimeout(n.close(), 1 * 1000)
```

INDEXEDDB

IndexedDB is one of the storage capabilities introduced into browsers over the years. Here's an introduction to IndexedDB, the Database of the Web supported by all modern Browsers

- Introduction to IndexedDB
- Create an IndexedDB Database
 - How to **create a database**
- Create an Object Store
 - How to create an object store or add a new one
 - Indexes
 - Check if a store exists
- Deleting from IndexedDB
 - Delete a database
 - Delete an object store
 - To delete data in an object store use a transaction
- Add an item to the database
- Getting items from a store
 - Getting a specific item from a store using `get()`
 - Getting all the items using `getAll()`
 - Iterating on all the items using a cursor via `openCursor()`
 - Iterating on a subset of the items using bounds and cursors

Introduction to IndexedDB

IndexedDB is one of the storage capabilities introduced into browsers over the years. It's a key/value store (a noSQL database) considered to be **the definitive solution for storing data in browsers**.

It's an asynchronous API, which means that performing costly operations won't block the UI thread providing a sloppy experience to users. It can store an indefinite amount of data,

although once over a certain threshold the user is prompted to give the site higher limits.

It's supported on all modern browsers (<http://caniuse.com/#feat=indexeddb>) .

It supports transactions, versioning and gives good performance.

Inside the browser we can also use:

- **Cookies:** can host a very small amount of strings
- **Web Storage** (or DOM Storage), a term that commonly identifies localStorage and sessionStorage, two key/value stores. sessionStorage, does not retain data, which is cleared when the session ends, while localStorage keeps the data across sessions

Local/session storage have the disadvantage of being capped at a small (and inconsistent) size, with browsers implementation offering from 2MB to 10MB of space per site.

In the past we also had **Web SQL**, a wrapper around SQLite, but now this is **deprecated** and unsupported on some modern browsers, it's never been a recognized standard and so it should not be used, although 83% of users have this technology on their devices according to Can I Use (<http://caniuse.com/#feat=sql-storage>) .

While you can technically create multiple databases per site, you generally **create one single database**, and inside that database you can create **multiple object stores**.

A database is **private to a domain**, so any other site cannot access another website IndexedDB stores.

Each store usually contains a set of *things*, which can be

- strings
- numbers
- objects
- arrays
- dates

For example you might have a store that contains posts, another that contains comments.

A store contains a number of items which have a unique key, which represents the way by which an object can be identified.

You can alter those stores using transactions, by performing add, edit and delete operations, and iterating over the items they contain.

Since the advent of Promises in ES2015, and the subsequent move of APIs to using promises, the IndexedDB API seems a bit *old school*.

While there's nothing wrong in it, in all the examples that I'll explain I'll use the IndexedDB Promised Library (<https://github.com/jakearchibald/idb>) by Jake Archibald, which is a tiny layer on top of the IndexedDB API to make it easier to use.

This library is also used on all the examples on the Google Developers website regarding IndexedDB

Create an IndexedDB Database

Include the `idb` lib using:

```
yarn add idb
```

And then include it in your page, either using Webpack or Browserify or any other build system, or simply:

```
<script src="./node_modules/idb/lib/idb.js"></script>
```

And we're ready to go.

Before using the IndexedDB API, always make sure you check for support in the browser, even though it's widely available, you never know which browser the user is using:

```
((() => {  
  'use strict'  
  
  if (!('indexedDB' in window)) {  
    console.warn('IndexedDB not supported')  
    return  
  }  
  
  // ...IndexedDB code  
})())
```

How to create a database

Using `idb.open()`:

```
const name = 'mydbname'
const version = 1 //versions start at 1
idb.open(name, version, upgradeDb => {})
```

The first 2 parameters are self-explanatory. The third param, which is optional, is a callback **called only if the version number is higher than the current installed database version**. In the callback function body you can upgrade the structure (stores and indexes) of the db.

We use the name `upgradeDB` for the callback to identify this is the time to update the database if needed.

Create an Object Store

How to create an object store or add a new one

An **object store** is created or updated in this callback, using the `db.createObjectStore('storeName', options)` syntax:

```
const dbPromise = idb.open('mydb', 1, (upgradeDB) => {
  upgradeDB.createObjectStore('store1')
})
.then(db => console.log('success'))
```

If you installed a previous version, the callback allows you to perform a the migration:

```
const dbPromise = idb.open('keyval-store', 3, (upgradeDB) => {
  switch (upgradeDB.oldVersion) {
    case 0: // no db created before
      // a store introduced in version 1
      upgradeDB.createObjectStore('store1')
    case 1:
      // a new store in version 2
      upgradeDB.createObjectStore('store2', { keyPath: 'name' })
  }
})
.then(db => console.log('success'))
```

`createObjectStore()` as you can see in case 1 accepts a second parameter that indicates the index key of the database. This is very useful when you store objects: `put()` calls don't need a second parameter, but can just take the value (an object) and the key will be mapped to the object property that has that name.

The index gives you a way to retrieve a value later by that specific key, and it must be unique (every item must have a different key)

A key can be set to auto increment, so you don't need to keep track of it on the client code. If you don't specify a key, IndexedDB will create it transparently for us:

```
upgradeDb.createObjectStore('notes', { autoIncrement: true })
```

but you can specify a specific field of object value to auto increment as well:

```
upgradeDb.createObjectStore('notes', {  
  keyPath: 'id',  
  autoIncrement: true  
})
```

As a general rule, use auto increment if your values do not contain a unique key already (for example, an email address for users).

Indexes

An index is a way to retrieve data from the object store. It's defined along with the database creation in the `idb.open()` callback in this way:

```
const dbPromise = idb.open('dogsdb', 1, (upgradeDB) => {  
  const dogs = upgradeDB.createObjectStore('dogs')  
  dogs.createIndex('name', 'name', { unique: false })  
})
```

The `unique` option determines if the index value should be unique, and no duplicate values are allowed to be added.

You can access an object store already created using the `upgradeDb.transaction.objectStore()` method:

```
const dbPromise = idb.open('dogsdb', 1, (upgradeDB) => {  
  const dogs = upgradeDB.transaction.objectStore('dogs')
```

```
dogs.createIndex('name', 'name', { unique: false })
})
```

Check if a store exists

You can check if an object store already exists by calling the `objectStoreNames()` method:

```
if (!upgradeDb.objectStoreNames.contains('store3')) {
  upgradeDb.createObjectStore('store3')
}
```

Deleting from IndexedDB

Deleting the database, an object store and data

Delete a database

```
idb.delete('mydb')
.then(() => console.log('done'))
```

Delete an object store

An object store can only be deleted in the callback when opening a db, and that callback is only called if you specify a version higher than the one currently installed:

```
const dbPromise = idb.open('dogsdb', 2, (upgradeDB) => {
  upgradeDB.deleteObjectStore('old_store')
})
```

To delete data in an object store use a transaction

```
const key = 232

dbPromise.then((db) => {
  const tx = db.transaction('store', 'readwrite')
  const store = tx.objectStore('store')
  store.delete(key)
  return tx.complete
})
.then(() => {
```

```
    console.log('Item deleted')
  })
```

Add an item to the database

You can use the `put` method of the object store, but first we need a reference to it, which we can get from `upgradeDB.createObjectStore()` when we create it.

When using `put`, the value is the first argument, the key is the second. This is because if you specify `keyPath` when creating the object store, you don't need to enter the key name on every `put()` request, you can just write the value.

This populates `store0` as soon as we create it:

```
idb.open('mydb', 1, (upgradeDB) => {
  keyValStore = upgradeDB.createObjectStore('store0')
  keyValStore.put('Hello world!', 'Hello')
})
```

To add items later down the road, you need to create a **transaction**, that ensures database integrity (if an operation fails, all the operations in the transaction are rolled back and the state goes back to a known state).

For that, use a reference to the `dbPromise` object we got when calling `idb.open()`, and run:

```
dbPromise.then((db) => {
  const val = 'hey!'
  const key = 'Hello again'

  const tx = db.transaction('store1', 'readwrite')
  tx.objectStore('store1').put(val, key)
  return tx.complete
})
.then(() => {
  console.log('Transaction complete')
})
.catch(() => {
  console.log('Transaction failed')
})
```

The IndexedDB API offers the `add()` method as well, but since `put()` allows us to both add and update, it's simpler to just use it.

Getting items from a store

Getting a specific item from a store using `get()`

```
dbPromise.then(db => db.transaction('objs')
    .objectStore('objs')
    .get(123456))
    .then(obj => console.log(obj))
```

Getting all the items using `getAll()`

```
dbPromise.then(db => db.transaction('store1')
    .objectStore('store1')
    .getAll())
    .then(objects => console.log(objects))
```

Iterating on all the items using a cursor via `openCursor()`

```
dbPromise.then((db) => {
    const tx = db.transaction('store', 'readonly')
    const store = tx.objectStore('store')
    return store.openCursor()
})
    .then(function logItems(cursor) {
        if (!cursor) { return }
        console.log('cursor is at: ', cursor.key)
        for (const field in cursor.value) {
            console.log(cursor.value[field])
        }
        return cursor.continue().then(logItems)
    })
    .then(() => {
        console.log('done!')
    })
```

Iterating on a subset of the items using bounds and cursors

```
const searchItems = (lower, upper) => {
    if (lower === '' && upper === '') { return }

    let range
    if (lower !== '' && upper !== '') {
        range = IDBKeyRange.bound(lower, upper)
    }
}
```

```

    } else if (lower === '') {
      range = IDBKeyRange.upperBound(upper)
    } else {
      range = IDBKeyRange.lowerBound(lower)
    }

    dbPromise.then((db) => {
      const tx = db.transaction(['dogs'], 'readonly')
      const store = tx.objectStore('dogs')
      const index = store.index('age')
      return index.openCursor(range)
    })
    .then(function showRange(cursor) {
      if (!cursor) { return }
      console.log('cursor is at:', cursor.key)
      for (const field in cursor.value) {
        console.log(cursor.value[field])
      }
      return cursor.continue().then(showRange)
    })
    .then(() => {
      console.log('done!')
    })
  })
}

searchDogsBetweenAges(3, 10)

```

THE SELECTORS API

Access DOM elements using `querySelector` and `querySelectorAll`. They accept any CSS selector, so you are no longer limited by selecting elements by ``id``

- Introduction
- The Selectors API
- Basic jQuery to DOM API examples
 - Select by `id`
 - Select by `class`
 - Select by tag name
- More advanced jQuery to DOM API examples
 - Select multiple items
 - Select by HTML attribute value
 - Select by CSS pseudo class
 - Select the descendants of an element

Introduction

jQuery and other DOM libraries got a huge popularity boost in the past, among with other features they provided, thanks to an easy way to select elements on a page.

Traditionally browsers provided one single way to select a DOM element, and that was by its `id` attribute, with `getElementById()` (<https://developer.mozilla.org/en-US/docs/Web/API/Document/getElementById>), a method offered by the `document` object.

The Selectors API

Since 2013 the Selectors API (<https://www.w3.org/TR/selectors-api/>), the DOM allows you to use two more useful methods:

- `document.querySelector()`

- `document.querySelectorAll()`

They can be safely used, as [caniuse.com](https://caniuse.com/#feat=queryselector) tells us (<https://caniuse.com/#feat=queryselector>), and they are even fully supported on IE9 in addition to all the other modern browsers, so there is no reason to avoid them, unless you need to support IE8 (which has partial support) and below.

They accept any CSS selector, so you are no longer limited by selecting elements by `id`.

- `document.querySelector()` returns a single element, the first found
- `document.querySelectorAll()` returns all the elements, wrapped in a `NodeList` object.

Those are all valid selectors:

- `document.querySelector('#test')`
- `document.querySelector('.my-class')`
- `document.querySelector('#test .my-class')`
- `document.querySelector('a: hover')`

Basic jQuery to DOM API examples

Here below is a translation of the popular jQuery API into native DOM API calls.

Select by `id`

```
$('#test')  
document.querySelector('#test')
```

We use `querySelector` since an `id` is unique in the page

Select by `class`

```
$('.test')  
document.querySelectorAll('.test')
```

Select by tag name

```
$('div')  
document.querySelectorAll('div')
```

More advanced jQuery to DOM API examples

Select multiple items

```
$('div, span')  
document.querySelectorAll('div, span')
```

Select by HTML attribute value

```
$('[data-example="test"]')  
document.querySelectorAll('[data-example="test"]')
```

Select by CSS pseudo class

```
$(':nth-child(4n)')  
document.querySelectorAll(':nth-child(4n)')
```

Select the descendants of an element

For example all `li` elements under `#test`:

```
$('#test li')  
document.querySelectorAll('#test li')
```

THE WEB STORAGE API

The Web Storage API provides a way to store data in the browser. It defines two storage mechanisms which are very important: Session Storage and Local Storage, part of the set of storage options available on the Web Platform



- Introduction
- How to access the storage
- Methods
 - `setItem(key, value)`
 - `getItem(key)`
 - `removeItem(key)`
 - `key(n)`
 - `clear()`
- Storage size limits
 - Desktop
 - Mobile

- Going over quota
- Developer Tools
 - Chrome
 - Firefox
 - Safari

Introduction

The Web Storage API defines two storage mechanisms which are very important: **Session Storage** and **Local Storage**.

They are part of the set of storage options available on the Web Platform, which includes:

- Cookies
- IndexedDB
- The Cache API

Application Cache is deprecated, and Web SQL is not implemented in Firefox, Edge and IE.

Both Session Storage and Local Storage provide a private area for your data. Any data you store cannot be accessed by other websites.

Session Storage maintains the data stored into it for the duration of the page session. If multiple windows or tabs visit the same site, they will have two different Session Storage instances.

When a tab/window is closed, the Session Storage for that particular tab/window is cleared.

Session storage is meant to allow the scenario of handling different processes happening on the same site independently, something not possible with cookies for example, which are shared in all sessions.

Local Storage instead persists the data until it's explicitly removed, either by you or by the user. It's never cleaned up automatically, and it's shared in all the sessions that access a site.

Both Local Storage and Session Storage are **protocol specific**: data stored when the page is accessed using `http` is not available when the page is served with `https`, and vice versa.

Web Storage is only accessible in the browser. It's not sent to the server like cookies do.

How to access the storage

Both Local and Session Storage are available on the `window` object, so you can access them using `sessionStorage` and `localStorage`.

Their set of properties and methods is exactly the same, because they return the same object, a Storage (<https://developer.mozilla.org/en-US/docs/Web/API/Storage>) object.

The Storage Object has a single property, `length`, which is the number of data items stored into it.

Methods

setItem(key, value)

`setItem()` adds an item to the storage. Accepts a string as key, and a string as a value:

```
localStorage.setItem('username', 'flaviocopes')
localStorage.setItem('id', '123')
```

If you pass any value that's not a string, it will be converted to string:

```
localStorage.setItem('test', 123) //stored as the '123' string
localStorage.setItem('test', {test: 1}) //stored as "[object Object]"
```

getItem(key)

`getItem()` is the way to retrieve a string value from the storage, by using the key string that was used to store it:

```
localStorage.getItem('username') // 'flaviocopes'
localStorage.getItem('id') // '123'
```

removeItem(key)

`removeItem()` removes the item identified by `key` from the storage, returning nothing (an `undefined` value):

```
localStorage.removeItem('id')
```

key(n)

Every item you store has an index number. So the first time you use `setItem()`, that item can be referenced using `key(0)`. The next with `key(1)` and so on.

If you reference a number that does not point to a storage item, it returns `null`.

Every time you remove an item with `removeItem()`, the index consolidates:

```
localStorage.setItem('a', 'a')
localStorage.setItem('b', 'b')
localStorage.key(0) //"a"
localStorage.key(1) //"b"
localStorage.removeItem('b')
localStorage.key(1) //null

localStorage.setItem('b', 'b')
localStorage.setItem('c', 'c')
localStorage.key(1) //"b"
localStorage.removeItem('b')
localStorage.key(1) //"c"
```

clear()

`clear()` removes everything from the storage object you are manipulating:

```
localStorage.setItem('a', 'a')
localStorage.setItem('b', 'b')
localStorage.length //2
localStorage.clear()
localStorage.length //0
```

Storage size limits

Through the Storage API you can store a lot more data than you would be able with cookies.

The amount of storage available on Web might differ by storage type (local or session), browser, and by device type. A research

(<https://www.html5rocks.com/en/tutorials/offline/quota-research/>) by html5rocks.com points out those limits:

Desktop

- Chrome, IE, Firefox: 10MB
- Safari: 5MB for local storage, unlimited session storage

Mobile

- Chrome, Firefox: 10MB
- iOS Safari and WebView: 5MB for local storage, session storage unlimited unless in iOS6 and iOS7 where it's 5MB
- Android Browser: 2MB local storage, unlimited session storage

Going over quota

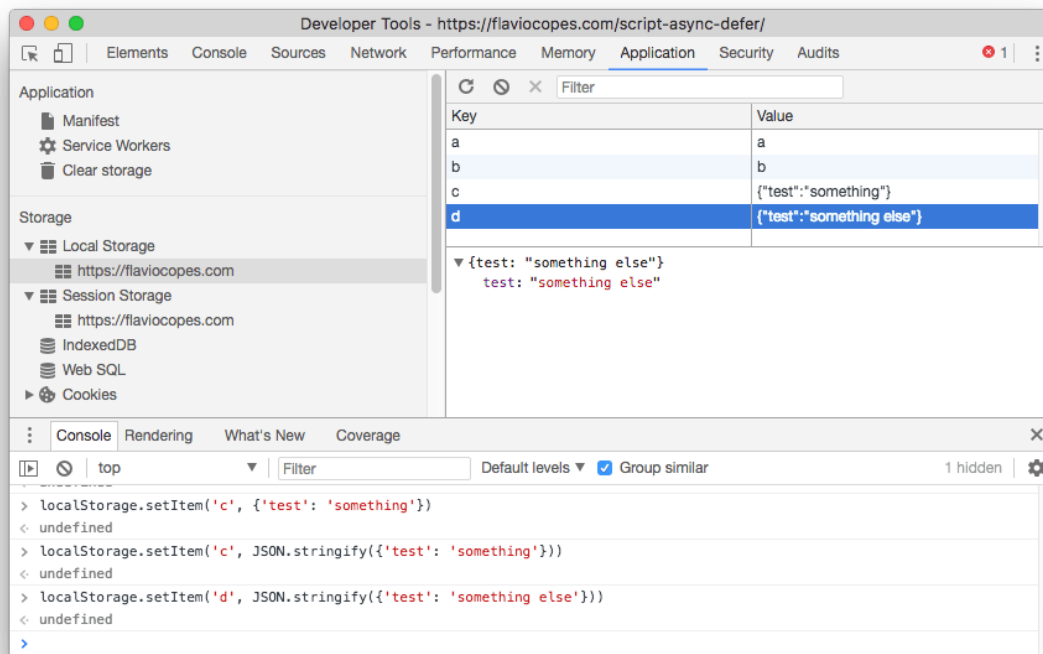
You need to handle quota errors, especially if you store lots of data. You can do so with a try/catch:

```
try {
  localStorage.setItem('key', 'value')
} catch(domException) {
  if (['QuotaExceededError',
    'NS_ERROR_DOM_QUOTA_REACHED'].includes(domException.name)) {
    // handle quota limit exceeded error
  }
}
```

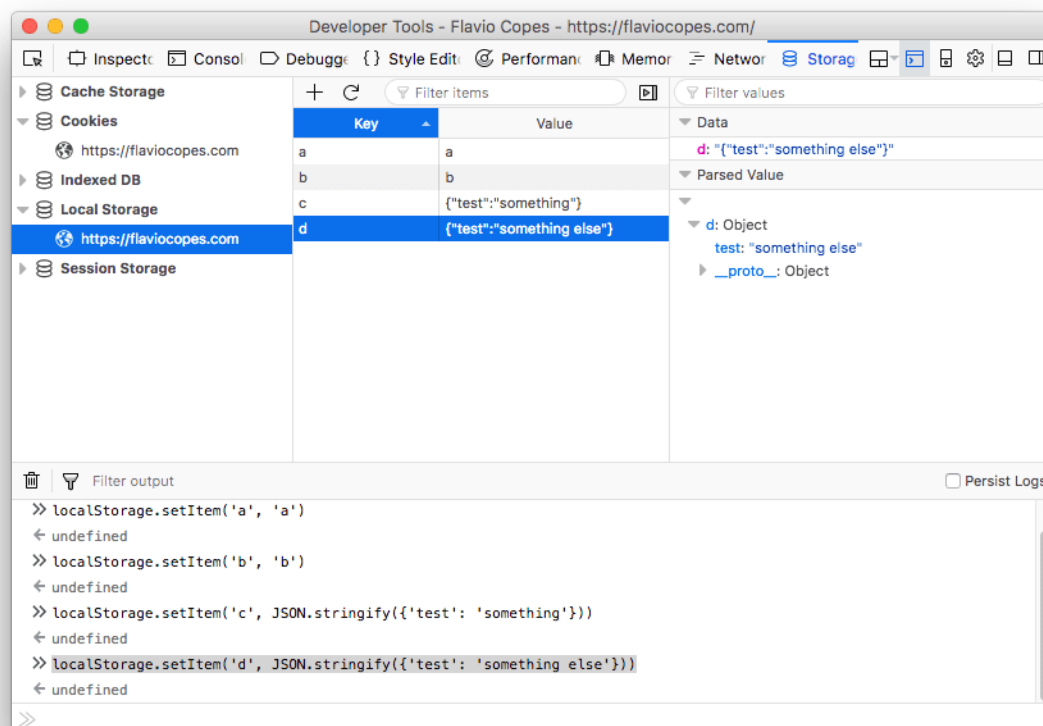
Developer Tools

The DevTools of the major browsers all offer a nice interface to inspect and manipulate the data stored in the Local and Session Storage.

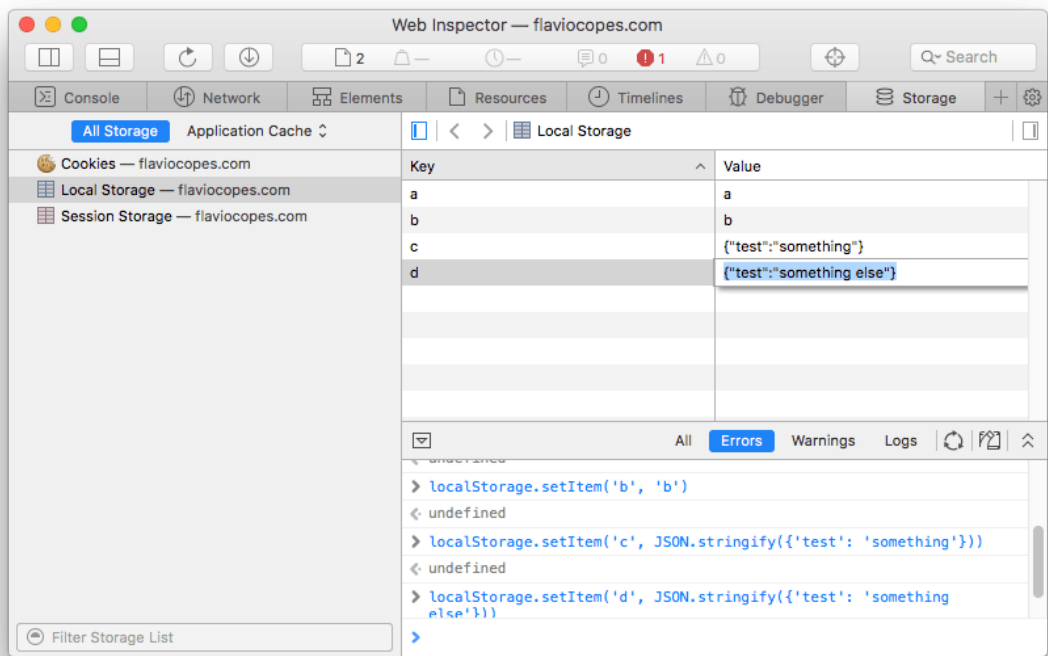
Chrome



Firefox

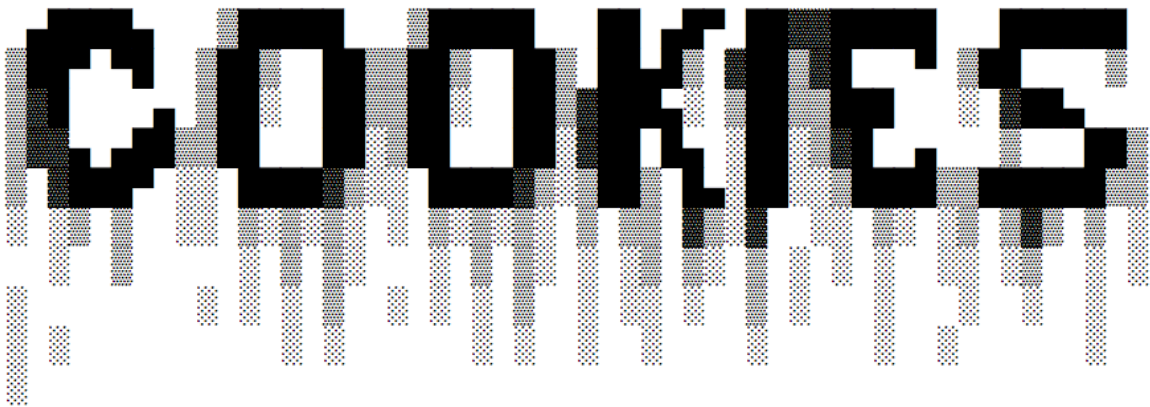


Safari



COOKIES

Cookies are a fundamental part of the Web, as they allow sessions and in general to recognize the users during the navigation



- Introduction
- Restrictions of cookies
- Set cookies
- Set a cookie expiration date
- Set a cookie path
- Set a cookie domain
- Cookie Security
 - Secure
 - HttpOnly
 - SameSite
- Update a cookie value or parameter
- Delete a cookie
 - Access the cookies values
- Check if a cookie exists
- Abstractions libraries
- Use cookies server-side

- Inspect cookies with the Browser DevTools
 - Chrome
 - Firefox
 - Safari
- Alternatives to cookies

Introduction

By using Cookies we can exchange information between the server and the browser to provide a way to customize a user session, and for servers to recognize the user between requests.

HTTP is stateless, which means all request origins to a server are exactly the same and a server cannot determine if a request comes from a client that already did a request before, or it's a new one.

Cookies are sent by the browser to the server when an HTTP request starts, and they are sent back from the server, which can edit their content.

Cookies are essentially used to store a session id.

In the past cookies were used to store various types of data, since there was no alternative. But nowadays with the Web Storage API (Local Storage and Session Storage) and IndexedDB, we have much better alternatives.

Especially because cookies have a very low limit in the data they can hold, since they are sent back-and-forth for every HTTP request to our server - including requests for assets like images or CSS / JavaScript files.

Cookies have a long history, they had their first version in 1994, and over time they were standardized in multiple RFC revisions.

RFC stands for Request for Comments, the way standards are defined by the Internet Engineering Task Force (IETF), the entity responsible for setting standards for the Internet

The latest specification for Cookies is defined in the RFC 6265 (<https://tools.ietf.org/html/rfc6265>) , which is dated 2011.

Restrictions of cookies

- Cookies can only store **4KB of data**
- Cookies are **private to the domain**. A site can only read the cookies it set, not other domains cookies
- You can have up to 20 limits of cookies per domain (but the exact number depends on the specific browser implementation)
- Cookies are limited in their total number (but the exact number depends on the specific browser implementation). If this number is exceeded, new cookies replace the older ones.

Cookies can be set or read server side, or client side.

In the client side, cookies are exposed by the document object as `document.cookie`

Set cookies

The simplest example to set a cookie is:

```
document.cookie = 'foo=bar'
```

This will add a new cookie to the existing ones (it does not overwrite existing cookies)

The cookie value should be url encoded with `encodeURIComponent()`

(https://developer.mozilla.org/en-US/docs/JavaScript/Reference/Global_Objects/encodeURIComponent), to make sure it does not contain any whitespace, comma or semicolon which are not valid in cookie values.

Set a cookie expiration date

If you don't set anything else, the cookie will expire when the browser is closed. To prevent so, add an expiration date, expressed in the UTC format (`Mon, 26 Mar 2018 17:04:05 UTC`)

```
document.cookie = 'foo=bar; expires=Mon, 26 Mar 2018 17:04:05 UTC'
```

A simple JavaScript snippet to set a cookie that expires in 24 hours is:

```
const date = new Date()  
date.setHours(date.getHours() + 5)  
document.cookie = 'foo=bar; expires=' + date.toUTCString()
```

Alternatively you can use the `max-age` parameter to set an expiration expressed in number of seconds:

```
document.cookie = 'foo=bar; max-age=3600' //expires in 60 minutes  
document.cookie = 'foo=bar; max-age=31536000' //expires in 1 year
```

Set a cookie path

The `path` parameter specifies a document location for the cookie, so it's assigned to a specific path, and sent to the server only if the path matches the current document location, or a parent:

```
document.cookie = 'foo=bar; path="/dashboard"'
```

this cookie is sent on `/dashboard`, `/dashboard/today` and other sub-urls of `/dashboard/`, but not on `/posts` for example.

If you don't set a path, it defaults to the current document location. This means that to apply a global cookie from an inner page, you need to specify `path="/"`.

Set a cookie domain

The `domain` can be used to specify a subdomain for your cookie.

```
document.cookie = 'foo=bar; domain="mysite.com";'
```

If not set, it defaults to the host portion even if using a subdomain (if on `subdomain.mydomain.com`, by default it's set to `mydomain.com`). Domain cookies are included in subdomains.

Cookie Security

Secure

Adding the `secure` parameter makes sure the cookie can only be transmitted securely over HTTPS, and it will not be sent over unencrypted HTTP connections:

```
document.cookie = 'foo=bar; Secure;'
```

Note that this does not make cookies secure in any way - always avoid adding sensitive information to cookies

HttpOnly

One useful parameter is `HttpOnly`, which makes cookies inaccessible via the `document.cookie` API, so they are only editable by the server:

```
document.cookie = 'foo=bar; Secure; HttpOnly'
```

SameSite

`SameSite`, still experimental and only supported by Chrome and Firefox

(<https://caniuse.com/#feat=same-site-cookie-attribute>, lets servers require that a cookie is not sent on cross-site requests, but only on resources that have the cookie domain as the origin, which should be a great help towards reducing the risk of CSRF (Cross Site Request Forgery) attacks.

Update a cookie value or parameter

To update the value of a cookie, just assign a new value to the cookie name:

```
document.cookie = 'foo=bar2'
```

Similar to updating the value, to update the expiration date, reassign the value with a new `expires` or `max-age` property:

```
document.cookie = 'foo=bar; max-age=31536000' //expires in 1 year
```

Just remember to also add any additional parameters you added in the first place, like `path` or `domain`.

Delete a cookie

To delete a cookie, unset its value and pass a date in the past:

```
document.cookie = 'foo=; expires=Thu, 01 Jan 1970 00:00:00 UTC;'
```

(and again, with all the parameters you used to set it)

Access the cookies values

To access a cookie, lookup `document.cookie`:

```
const cookies = document.cookie
```

This will return a string with all the cookies set for the page, semicolon separated:

```
'foo1=bar1; foo2=bar2; foo3=bar3'
```

Check if a cookie exists

```
//ES5
if (document.cookie.split(';').filter((item) => {
  return item.indexOf('foo=') >= 0
}).length) {
  //foo exists
}

//ES7
if (document.cookie.split(';').filter((item) => {
  return item.includes('foo=')
}).length) {
  //foo exists
}
```

Abstractions libraries

There are a number of different libraries that will provide a friendlier API to manage cookies.

One of them is <https://github.com/js-cookie/js-cookie>, which supports up to IE7, and has a lot of stars on GitHub (which is always good).

Some examples of its usage:

```
Cookies.set('name', 'value')
Cookies.set('name', 'value', {
  expires: 7,
```

```

    path: '',
    domain: 'subdomain.site.com',
    secure: true
  })

Cookies.get('name') // => 'value'
Cookies.remove('name')

//JSON
Cookies.set('name', { foo: 'bar' })
Cookies.getJSON('name') // => { foo: 'bar' }

```

Use that or the native Cookies API?

It all comes down to adding more kilobytes to download for each user, so it's your choice.

Use cookies server-side

Every environment used to build an HTTP server allows you to interact with cookies, because cookies are a pillar of the Modern Web, and not much could be built without them.

PHP has `$_COOKIE` Go has cookies facilities in the `net/http` standard library

and so on.

Let's do an example with Node.js

When using Express.js, you can create cookies using the `res.cookie`

(<http://expressjs.com/en/api.html#res.cookie>) API:

```

res.cookie('foo1', '1bar', { domain: '.example.com', path: '/admin', secure: true })
res.cookie('foo2', 'bar2', { expires: new Date(Date.now() + 900000), httpOnly: true })
res.cookie('foo3', 'bar3', { maxAge: 900000, httpOnly: true })

//takes care of serializing JSON
res.cookie('foo4', { items: [1,2,3] }, { maxAge: 900000 })

```

To parse cookies, a good choice is to use the <https://github.com/expressjs/cookie-parser> middleware. Every Request object will have cookies information in the `req.cookie` (<http://expressjs.com/en/api.html#req.cookies>) property:

```

req.cookies.foo //bar
req.cookies.foo1 //bar1

```

If you create your cookies using `signed: true`:

```
res.cookie('foo5', 'bar5', { signed: true })
```

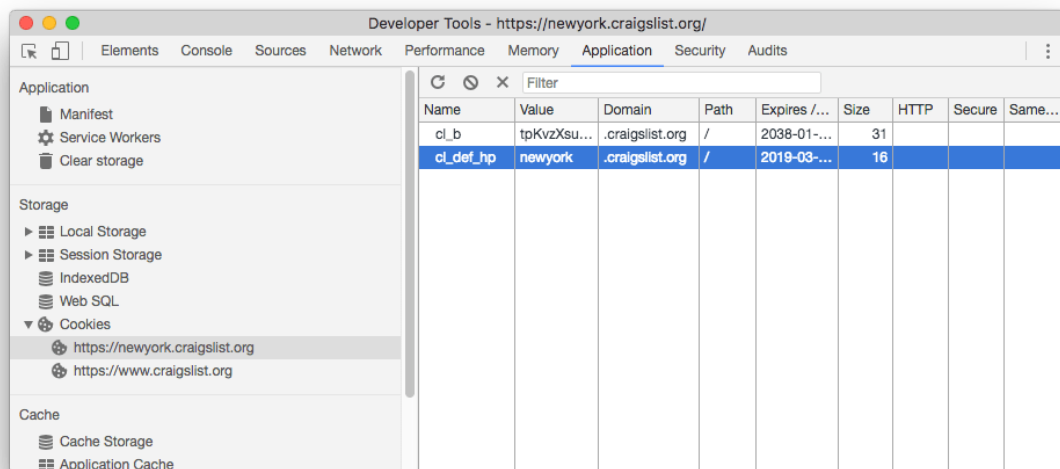
they will be available in the `req.signedCookies` object instead. Signed cookies will be completely unreadable in the frontend, but transparently encoded/decoded on the server side.

<https://github.com/expressjs/session> and <https://github.com/expressjs/cookie-session> are two different middleware options to build cookie-based authentication, which one to use depends on your needs.

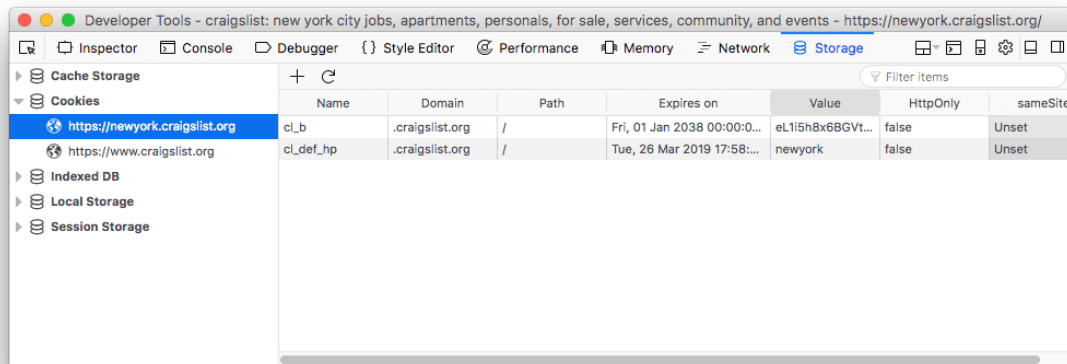
Inspect cookies with the Browser DevTools

All browsers in their DevTools provide an interface to inspect and edit cookies.

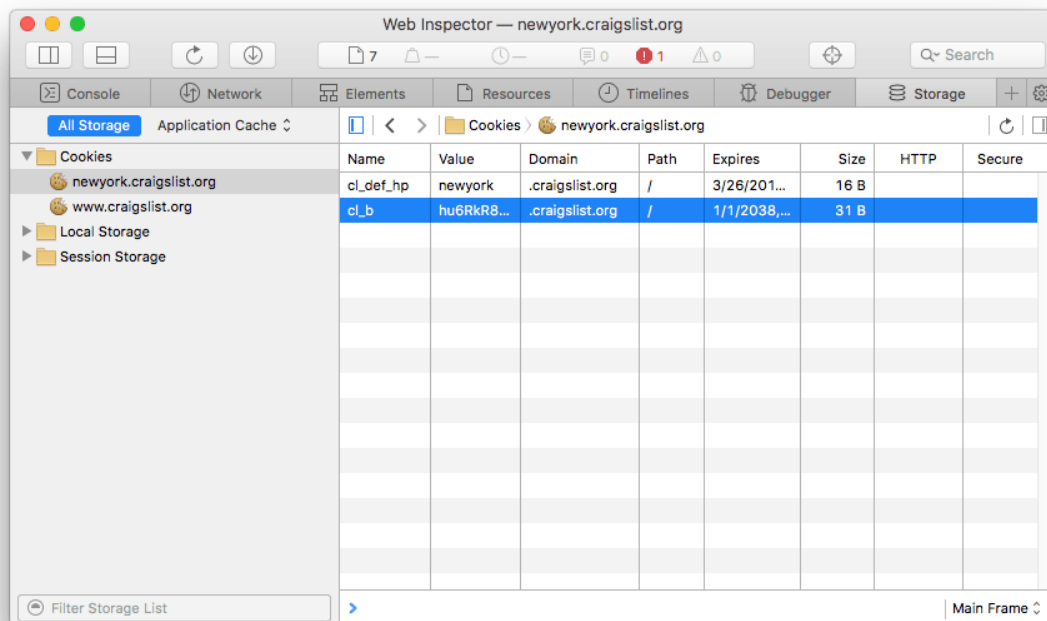
Chrome



Firefox



Safari



Alternatives to cookies

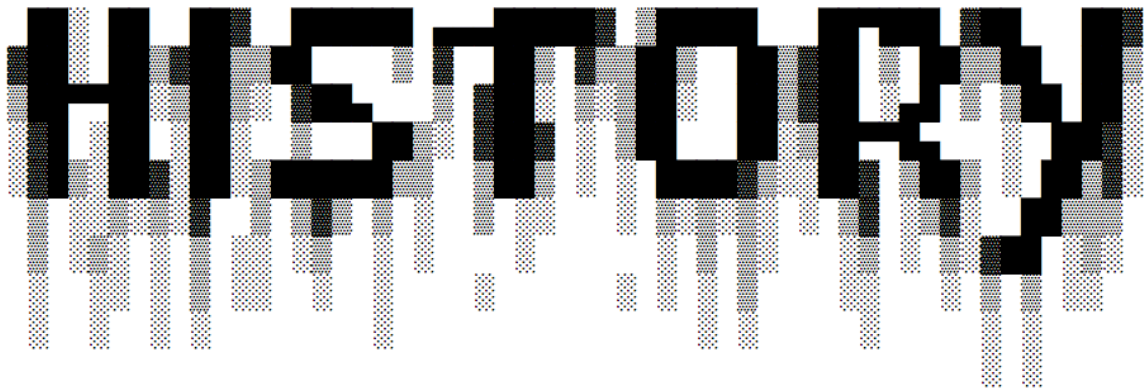
Are cookies the only way to build authentication and sessions on the Web?

No! There is a technology that recently got popular, called **JSON Web Tokens (JWT)**, which is a **Token-based Authentication**.

Article about JWT coming soon!

HISTORY API

The History API is the way browsers let you interact with the address bar and the navigation history



- Introduction
- Access the History API
- Navigate the history
- Add an entry to the history
- Modify history entries
- Access the current history entry state
- The `onpopstate` event

Introduction

The History API lets you **interact with the browser history, trigger the browser navigation methods** and **change the address bar content**.

It's especially useful in combination with modern Single Page Applications, on which you never make a server-side request for new pages, but instead the page is always the same: just the internal content changes.

A **modern JavaScript application** running in the browser that does not interact with the History API, either explicitly or at the framework level, is going to be a poor experience to the user, since **the back and forward buttons break**.

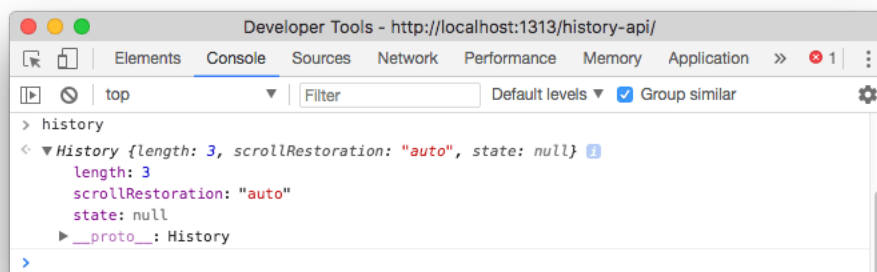
Also, when navigating the app, **the view changes but the address bar does not**.

And also **the reload button breaks**: reloading the page, since there is no deep linking, is going to make the browser show a different page

The History API was introduced in HTML5 and is now supported by all modern browsers (<https://caniuse.com/#feat=history>). IE supports it since version 10, and if you need to support IE9 and older, use the History.js library (<https://github.com/browserstate/history.js/>).

Access the History API

The History API is available on the `window` object, so you can call it like this: `window.history` or simply `history`, since `window` is the global object.



Navigate the history

Let's start with the simplest thing you can do with the History API.

Go back to the previous page:

```
history.back()
```

this goes to the previous entry in the session history. You can forward to the next page using

```
history.forward()
```

This is exactly just like using the browser back and forward buttons.

`go()` lets you navigate back or forward multiple levels deep. For example

```
history.go(-1) //equivalent to history.back()
history.go(-2) //equivalent to calling history.back() twice
history.go(1) //equivalent to history.forward()
history.go(3) //equivalent to calling history.forward() 3 times
```

To know how many entries there are in the history, you can call

```
history.length
```

Add an entry to the history

Using `pushState()` you can create a new history entry programmatically. You pass 3 parameters.

The first is an object which can contain anything (there is a size limit however, and the object needs to be serializable).

The second parameter is currently unused by major browsers, so you generally pass an empty string.

The third parameter is a URL associated to the new state. Note that the URL needs to belong to the same origin domain of the current URL.

```
const state = { foo: "bar" }
history.pushState(state, '', '/foo');
```

Calling this won't change the content of the page, and does not cause any browser action like changing `window.location` would.

Modify history entries

While `pushState()` lets you add a new state to the history, **`replaceState()`** allows you to edit the current history state.

```
history.pushState({}, '', '/posts');
const state = { post: "first" }
```

```
history.pushState(state, '', '/post/first');  
const state = { post: "second" }  
history.replaceState(state, '', '/post/second');
```

If you now call

```
history.back()
```

the browser goes straight to `/posts`, since `/post/first` was **replaced** by `/post/second`

Access the current history entry state

Accessing the property

```
history.state
```

returns the current state object (the first parameter passed to `pushState` or `replaceState`).

The `onpopstate` event

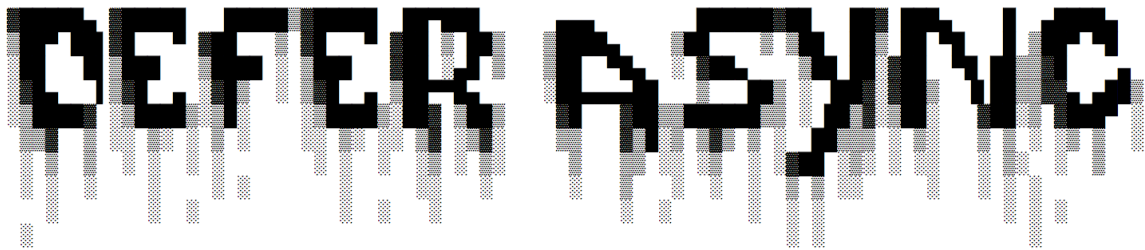
This event is called on `window` every time the active history state changes, with the current state as the callback parameter:

```
window.onpopstate = (event) => {  
  console.log(event.state)  
}
```

will log the new state object (the first parameter passed to `pushState` or `replaceState`) every time you call `history.back()`, `history.forward()` or `history.go()`.

EFFICIENTLY LOAD JAVASCRIPT WITH DEFER AND ASYNC

When loading a script on an HTML page, you need to be careful not to harm the loading performance of the page. Depending on where and how you add your scripts to an HTML page will influence the loading time



- The position matters
- Async and Defer
- Performance comparison
 - No defer or async, in the head
 - No defer or async, in the body
 - With async, in the head
 - With defer, in the head
- Blocking parsing
- Blocking rendering
- domInteractive
- Keeping things in order
- TL;DR, tell me what's the best

When loading a script on an HTML page, you need to be careful not to harm the loading performance of the page.

A script is traditionally included in the page in this way:

```
<script src="script.js"></script>
```

whenever the HTML parser finds this line, a request will be made to fetch the script, and the script is executed.

Once this process is done, the parsing can resume, and the rest of the HTML can be analyzed.

As you can imagine, this operation can have a huge impact on the loading time of the page.

If the script takes a little longer to load than expected, for example if the network is a bit slow or if you're on a mobile device and the connection is a bit sloppy, the visitor will likely see a blank page until the script is loaded and executed.

The position matters

When you first learn HTML, you're told script tags live in the `<head>` tag:

```
<html>
  <head>
    <title>Title</title>
    <script src="script.js"></script>
  </head>
  <body>
    ...
  </body>
</html>
```

As I told earlier, when the parser finds this line, it goes to fetch the script and executes it. *Then*, after it's done with this task, it goes on to parse the body.

This is bad because there is a lot of delay introduced. A very common solution to this issue is to put the `script` tag to the bottom of the page, just before the closing `</body>` tag.

Doing so, the script is loaded and executed after all the page is already parsed and loaded, which is a **huge improvement** over the `head` alternative.

This is the best thing you can do, if you need to support older browsers that do not support two relatively recent features of HTML: `async` and `defer`.

Async and Defer

Both `async` and `defer` are boolean attributes. Their usage is similar:

```
<script async src="script.js"></script>
```

```
<script defer src="script.js"></script>
```

if you specify both, `async` takes precedence on modern browsers, while older browsers that support `defer` but not `async` will fallback to `defer`.

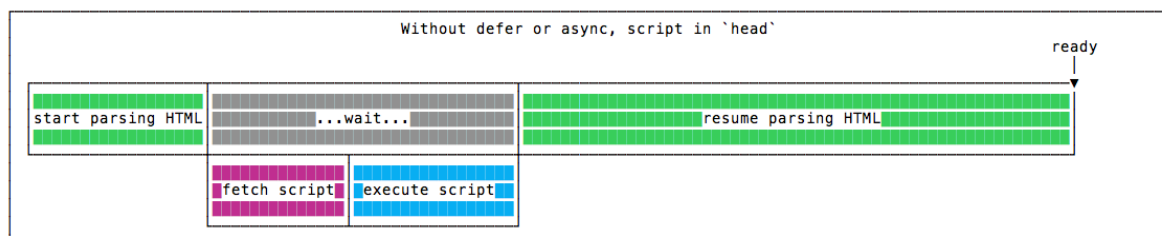
For the support table, check [caniuse.com](https://caniuse.com/#feat=script-async) for `async` <https://caniuse.com/#feat=script-async> and for `defer` <https://caniuse.com/#feat=script-defer>

These attributes make only sense when using the script in the `head` portion of the page, and they are useless if you put the script in the `body` footer like we saw above.

Performance comparison

No defer or async, in the head

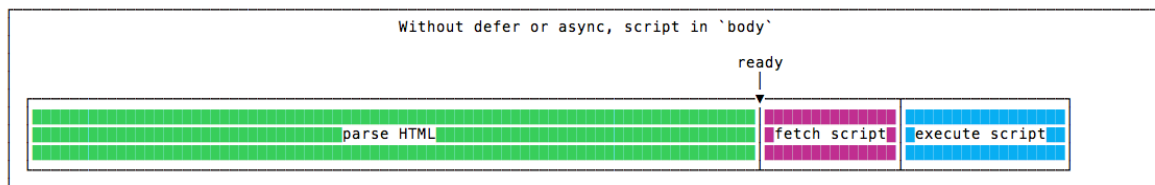
Here's how a page loads a script without neither `defer` or `async`, put in the `head` portion of the page:



The parsing is paused until the script is fetched, and executed. Once this is done, parsing resumes.

No defer or async, in the body

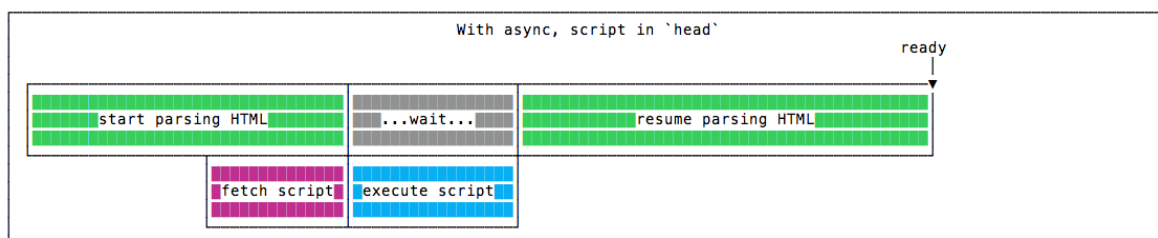
Here's how a page loads a script without neither `defer` or `async`, put at the end of the `body` tag, just before it closes:



The parsing is done without any pauses, and when it finishes, the script is fetched, and executed. Parsing is done before the script is even downloaded, so the page appears to the user way before the previous example.

With async, in the head

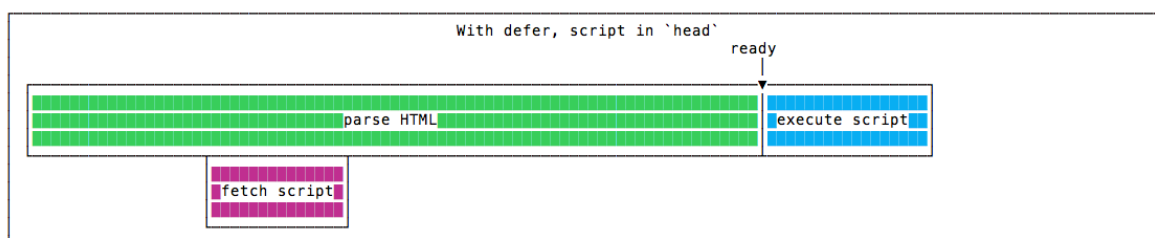
Here's how a page loads a script with `async`, put in the `head` tag:



The script is fetched asynchronously, and when it's ready the HTML parsing is paused to execute the script, then it's resumed.

With defer, in the head

Here's how a page loads a script with `defer`, put in the `head` tag:



The script is fetched asynchronously, and it's executed only after the HTML parsing is done.

Parsing finishes just like when we put the script at the end of the `body` tag, but overall the script execution finishes well before, because the script has been downloaded in parallel with the HTML parsing.

So this is the winning solution in terms of speed 🏆

Blocking parsing

`async` blocks the parsing of the page while `defer` does not.

Blocking rendering

Neither `async` nor `defer` guarantee anything on blocking rendering. This is up to you and your script (for example, making sure your scripts run after the `onLoad`) event.

domInteractive

Scripts marked `defer` are executed right after the `domInteractive` event, which happens after the HTML is loaded, parsed and the DOM is built.

CSS and images at this point are still to be parsed and loaded.

Once this is done, the browser will emit the `domComplete` event, and then `onLoad` .

`domInteractive` is important because its timing is recognized as a measure of perceived loading speed. See the MDN (<https://developer.mozilla.org/en-US/docs/Web/API/PerformanceTiming/domInteractive>) for more.

Keeping things in order

Another case pro `defer` : scripts marked `async` are executed in casual order, when they become available. Scripts marked `defer` are executed (after parsing completes) in the order which they are defined in the markup.

TL;DR, tell me what's the best

The best thing to do to speed up your page loading when using scripts is to put them in the `head` , and add a `defer` attribute to your `script` tag:

```
<script defer src="script.js"></script>
```

This is the scenario that triggers the faster `domInteractive` event.

Considering the pros of `defer`, it seems a better choice over `async` in a variety of scenarios.

Unless you are fine with delaying the first render of the page, making sure that when the page is parsed the JavaScript you want is already executed.

WEBP

WebP is an Open Source image format developed at Google, which promises to generate images smaller in size compared to JPG and PNG formats, while generating better looking images



- Introduction
- How much smaller?
- Generating WebP images
- Browsers support
- How can you use WebP today?

Introduction

WebP is an Open Source image format developed at Google, which promises to generate images smaller in size compared to JPG and PNG formats, while generating better looking images.

WebP supports **transparency**, like PNG and GIF images.

WebP supports **animations**, like GIF images

And, using WebP you can set the quality ratio of your images, so you decide if you want to get better quality or a smaller size (like it happens in JPG images).

So **WebP can do all GIF, JPG and PNG images can do**, in a single format, and generate **smaller images**. Sounds like a deal.

If you want to compare how images look in the various formats, here's a great gallery by Google (<https://developers.google.com/speed/webp/gallery>) .

WebP is not *new*, it's been around for several years now.

How much smaller?

The premise of generating smaller images is very interesting, especially when you consider that most of a web page size is determined by the amount and size of the image assets the user should download.

Google published a comparative study (https://developers.google.com/speed/webp/docs/c_study) on the results of 1 million images with this result:

WebP achieves overall higher compression than either JPEG or JPEG 2000. Gains in file size minimization are particularly high for smaller images which are the most common ones found on the web.

You should experiment with the kind of images you intend to serve, and form your opinion based on that.

In my tests, lossless compression compared to PNG generates WebP images 50% smaller. PNG reaches that file sizes only when using lossy compression.

Generating WebP images

All modern graphical and image editing tools let you export to `.webp` files.

Command-line tools also exist to convert images to WebP directly. Google provides a set of tools (<https://developers.google.com/speed/webp/download>) for this.

`cwebp` is the main command line utility to convert any image to `.webp` , use it with

```
cwebp image.png -o image.webp
```

Check out all the options using `cwebp --longhelp` .

Browsers support

WebP is currently supported by

- Chrome
- Opera
- Opera Mini
- UC Browser
- Samsung Internet

However, only Chrome for Desktop and Opera 19+ implement all the features of WebP, which are:

- lossy compression
- lossless compression
- transparency
- animation

Other browsers only implement a subset, and Firefox, Safari, Edge and IE do not support WebP at all, and there are no signs of WebP being implemented any time soon in those browsers.

But Chrome alone is a good portion of the web market, so if we can serve those users an optimized image, to speed up serving them and consume less bandwidth, it's great. But check if it actually reduces your images size.

Check with your JPG/PNG image optimization tooling results, and see if adding an additional layer of complexity introduced by WebP is useful or not.

How can you use WebP today?

There are a few different ways to do so.

You can use a server-level mechanism that serves WebP images instead of JPG and PNG when the `HTTP_ACCEPT` request header contains `image/webp`.

The first is the most convenient, as completely transparent to you and to your web pages.

Another option is to use Modernizr and check the `Modernizr.webp` setting.

If you don't need to support Internet Explorer, a very convenient way is to use the `<picture>` tag, which is now supported (<https://caniuse.com/#search=picture>) by all the major browsers except Opera Mini and IE (all versions).

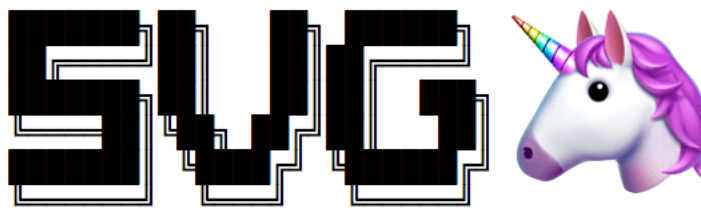
The `<picture>` tag is generally used for responsive images, but we can use it for WebP too, as this tutorial from HTML5 Rocks (<https://www.html5rocks.com/en/tutorials/responsive/picture-element/#toc-file-type>) explains.

You can specify a list of images, and they will be used in order, so in the next example, browsers that support WebP will use the first image, and fallback to JPG if not:

```
<picture>
  <source type="image/webp" srcset="image.webp">
  
</picture>
```

SVG

SVG is an awesome and incredibly powerful image format. This tutorial gives you an overview of SVG by explaining all you need to know in a simple way



- Introduction
- The advantages of SVG
- Your first SVG image
- Using SVG
- SVG Elements
 - `text`
 - `circle`
 - `rect`
 - `line`
 - `path`
 - `textPath`
 - `polygon`
 - `g`
- SVG viewport and viewBox
- Inserting SVG in Web Pages
 - With an `img` tag
 - With the CSS `background-image` property

- Inline in the HTML
- With an `object`, `iframe` or `embed` tag
- Inline SVG using a Data URL
- Styling elements
- Interacting with a SVG with CSS or JavaScript
 - CSS inside SVG
 - JavaScript inside SVG
 - JavaScript outside the SVG
 - CSS outside the SVG
- SVG vs Canvas API
- SVG Symbols
- Validate an SVG
- Should I include the `xmlns` attribute?
- Should I worry about browser support?

Introduction

Despite being standardized in the early 2000s, SVG (a shorthand for *Scalable Vector Graphics*) is a hot topic these days.

SVG has been penalized for quite a few years by the poor browser support (most notably IE).

I found this quote from a 2011 book: “at the time of writing, direct embedding of SVG into HTML works only in the very newest browsers”. 7 years ago, this is now a thing of the past, and we can use SVG images safely.

Today we can use SVG images safely, unless you have a lot of users with IE8 and below, or with older Android devices. In this case, fallbacks exist.

IE	Edge *	Firefox	Chrome	Safari	iOS Safari *	Opera Mini *	Chrome for Android	UC Browser for Android	Samsung Internet
			49						
			63		10.2				
		58	64		10.3				4
11	16	59	65	11	11.2	all	64	11.8	6.2
	17	60	66	11.1	11.3				
		61	67	TP					
			68						

Some part of the success of SVG is due to the variety of screen displays we must support, at different resolutions and sizes. A perfect task for SVG.

Also, the rapid decline of Flash in the last few years led to a renewed interest in SVG, which is great for a lot of things that Flash did in the past.

SVG is a **vector** image file format. This makes them very different than image format such as PNG, GIF or JPG, which are raster image file formats.

The advantages of SVG

SVG images, thanks to being vector images, can **infinitely scale** and not have any issue in image quality degradation. How so? Because SVG images are built using **XML markup**, and the browser prints them by plotting each point and line, rather than filling some space with pre-defined pixels. This ensures SVG images can adapt to different screen sizes and resolutions, even ones that have yet to be invented.

Thanks to being defined in XML, SVG images are much more **flexible** than JPG or PNG images, and** we can use CSS and JavaScript to interact with them**. SVG images can even *contain* CSS and JavaScript.

SVG images can render vector-style images a lot smaller than other formats, and are mainly used on logos and illustrations. Another huge use case is icons. Once domain of Icon Fonts like FontAwesome, now designers prefer using SVG images because they are smaller and they allow to have multi-color icons.

SVG is easy to animate, which is a very cool topic.

SVG provides some image editing effects, like masking and clipping, applying filters, and more.

SVG are just text, and as such it can be efficiently compressed using GZip.

Your first SVG image

SVG images are defined using XML. This means that SVG will look very familiar if you are proficient in HTML, except rather than having tags that are suited for document construction (like `p`, `article`, `footer`, `aside`) in SVG we have the building blocks of vector images: `path`, `rect`, `line` and so on.

This is an example SVG image:

```
<svg width="10" height="10">
  <rect x="0" y="0" width="10" height="10" fill="blue" />
</svg>
```

Notice how it's very easy to read and understand how the image will look like: it's a simple blue rectangle of 10x10 pixels (the default unit).

Most of the times you won't have to edit the SVG code, but you will use tools like Sketch or Figma or any other vector graphics tool to create the image, and export it as SVG.

The current version of SVG is 1.1, and SVG 2.0 is under development.

Using SVG

SVG images can be displayed by the browser by including them in a `img` tag:

```

```

just like you would do for other pixel-based image formats:

```




```

In addition, pretty uniquely, SVG they can be directly included in the HTML page:

```
<!DOCTYPE html>
<html>
  <head>
    <title>A page</title>
  </head>
  <body>
    <svg width="10" height="10">
      <rect x="0" y="0" width="10" height="10" fill="blue" />
    </svg>
  </body>
</html>
```

Please note that HTML5 and XHTML require a different syntax for inline SVG images. Luckily XHTML is a thing of the past, as it was more complex than necessary, but it's worth knowing in case you still need to work on XHTML pages.

The **ability to inline SVG in HTML** makes this format a **unicorn** in the scene, as other images can't do this, and must be fetched by opening a separate request for each one.

SVG Elements

In the example above you saw the usage of the `rect` element. SVG has a lot of different elements.

The most used ones are

- `text` : creates a text element
- `circle` : creates a circle
- `rect` : creates a rectangle
- `line` : creates a line
- `path` : create a path between two points
- `textPath` : create a path between two points, and a linked text element
- `polygon` : allows to create any kind of polygon
- `g` : groups separate elements

Coordinates start at 0,0 at the top-left of the drawing area, and extend from left to right for `x`, from top to bottom for `y`.

The images you see reflect the code shown above. Using the Browser DevTools you can inspect and change them.

text

The `text` element adds text. The text can be selected using the mouse. `x` and `y` define the starting point of the text

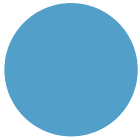
```
<svg>
  <text x="5" y="30">A nice rectangle</text>
</svg>
```

A nice rectangle

circle

Define a circle. `cx` and `cy` are the center coordinates, and `r` is the radius. `fill` is a common attribute and represents the figure color.

```
<svg>
  <circle cx="50" cy="50" r="50" fill="#529fca" />
</svg>
```



rect

Defines a rectangle. `x`, `y` are the starting coordinates, `width` and `height` are self-explanatory.

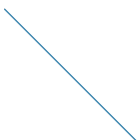
```
<svg>
  <rect x="0" y="0" width="100" height="100" fill="#529fca" />
</svg>
```



line

`x1` and `y1` define the starting coordinates. `x2` and `y2` define the ending coordinates. `stroke` is a common attribute and represents the line color.

```
<svg>
  <line x1="0" y1="0" x2="100" y2="100" stroke="#529fca" />
</svg>
```



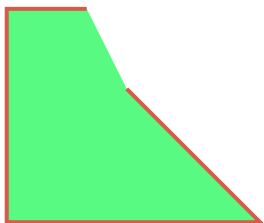
path

A path is a sequence of lines and curves. It's the most powerful tool to draw using SVG, and as such it's the most complex.

`d` contains the directions commands. These commands start with the command name, and a set of coordinates:

- `M` means Move, it accepts a set of coordinates `x, y`
- `L` means Line, it accepts a set of coordinates `x, y` to draw the line to
- `H` is an Horizontal Line, it only accept an `x` coordinate
- `V` is a Vertical Line, it only accept an `y` coordinate
- `Z` means Close Path, puts a line back to the start
- `A` means Arch, it needs a whole tutorial on its own
- `Q` is a quadratic Bezier curve, again it needs a whole tutorial on its own

```
<svg height="300" width="300">
  <path d="M 100 100 L 200 200 H 10 V 40 H 70"
        fill="#59fa81" stroke="#d85b49" stroke-width="3" />
</svg>
```



textPath

Adds a text along the shape of a path element.

Wow such a nice SVG tut

polygon

Draw any random polygon with `polygon`. `points` represents a set of x,y coordinates the polygon should link:

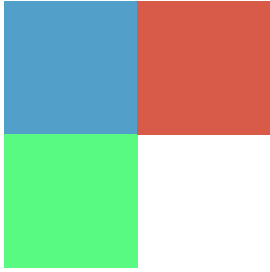
```
<svg>
  <polygon points="9.9, 1.1, 3.3, 21.78, 19.8, 8.58, 0, 8.58, 16.5, 21.78" />
</svg>
```



g

Using the `g` element you can group multiple elements:

```
<svg width="200" height="200">
  <rect x="0" y="0" width="100" height="100" fill="#529fca" />
  <g id="my-group">
    <rect x="0" y="100" width="100" height="100" fill="#59fa81" />
    <rect x="100" y="0" width="100" height="100" fill="#59fa81" />
  </g>
</svg>
```



SVG viewport and viewBox

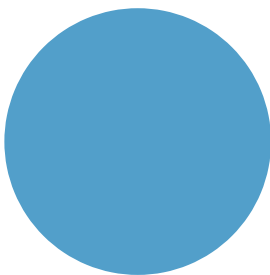
The size of an SVG relative to its container is set by the `width` and `height` attributes of the `svg` element. Those units default to pixels, but you can use any other usual unit like `%` or `em`. This is the **viewport**.

Generally “container” means the browser window, but a `svg` element can contain other `svg` elements, in that case the container is the parent `svg`.

An important attribute is `viewBox`. It lets you define a new coordinates system inside the SVG canvas.

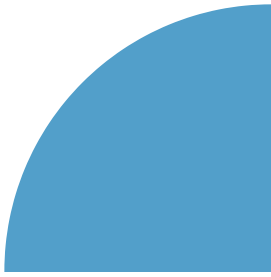
Say you have a simple circle, in a 200x200px SVG:

```
<svg width="200" height="200">
  <circle cx="100" cy="100" r="100" fill="#529fca" />
</svg>
```



By specifying a **viewBox** you can choose to **only show a portion** of this SVG. For example you can start at point 0, 0 and only show a 100x100px canvas:

```
<svg width="200" height="200" viewBox="0 0 100 100">
  <circle cx="100" cy="100" r="100" fill="#529fca" />
</svg>
```



starting at 100, 100 you will see another portion, the bottom right half of the circle:

```
<svg width="200" height="200" viewBox="100 100 100 100">
  <circle cx="100" cy="100" r="100" fill="#529fca" />
</svg>
```



A great way to visualize this is to imagine Google Maps being a gigantic SVG image, and your browser is a `viewBox` as big as the window size. When you move around, the `viewBox` changes its starting point (x, y) coordinates, and when you resize the window, you change the width and height of the `viewBox`.

Inserting SVG in Web Pages

There are various ways to add SVG to a webpage.

The most common ones are:

- with an `img` tag
- with the CSS `background-image` property
- inline in the HTML
- with an `object`, `iframe` or `embed` tag

See all these examples live on Glitch: <https://flavio-svg-loading-ways.glitch.me/>

With an `img` tag

```

```

With the CSS background-image property

```
<style>
.svg-background {
  background-image: url(flag.svg);
  height: 200px;
  width: 300px;
}
</style>
<div class="svg-background"></div>
```

Inline in the HTML

```
<svg width="300" height="200" viewBox="0 0 300 200"
  version="1.1" xmlns="http://www.w3.org/2000/svg"
  xmlns:xlink="http://www.w3.org/1999/xlink">
  <title>Italian Flag</title>
  <desc>By Flavio Copes https://flaviocopes.com</desc>
  <g id="flag">
    <rect fill="green" x="0" y="0" width="100" height="200"></rect>
    <rect fill="white" x="100" y="0" width="100" height="200"></rect>
    <rect fill="red" x="200" y="0" width="100" height="200"></rect>
  </g>
</svg>
```

With an object, iframe or embed tag

```
<object data="flag.svg" type="image/svg+xml"></object>

<iframe src="flag.svg" frameborder="0"></iframe>

<embed src="flag.svg" type="" />
```

Using `embed` you have the option to get the SVG document from the parent document using

```
document.getElementById('my-svg-embed').getSVGDocument()
```

and from inside the SVG you can reference the parent document with:

```
window.parent.document
```

Inline SVG using a Data URL

You can use any of the above examples combined with Data URLs to inline the SVG in the HTML:

```


<object data="data:image/svg+xml;<DATA>" type="image/svg+xml"></object>

<iframe data="data:image/svg+xml;<DATA>" frameborder="0"></iframe>
```

and in CSS too:

```
.svg-background {
  background-image: url("data:image/svg+xml;<DATA>");
}
```

Just change `<DATA>` with the appropriate Data URL (https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Data_URIs).

Styling elements

Any SVG element can accept a `style` attribute, just like HTML tags. Not all CSS properties work as you would expect, due to the SVG nature. For example to change the color of a text element, use `fill` instead of `color`.

```
<svg>
  <text x="5" y="30" style="fill: green">A nice text</text>
</svg>

<svg>
  <text x="5" y="70" style="fill: green; font-family: Courier New">
    A nice text
  </text>
</svg>
```

You can use `fill` as an element attribute as well, as you saw before:

```
<svg>
  <text x="5" y="70" fill="green">A nice text</text>
</svg>
```

Other common properties are

- `fill-opacity`, background color opacity
- `stroke`, defines the border color
- `stroke-width`, sets the width of the border

CSS can target SVG elements like you would target HTML tags:

```
rect {
  fill: red;
}
circle {
  fill: blue;
}
```

Interacting with a SVG with CSS or JavaScript

SVG images can be styled using CSS, or scripted with JavaScript, in those cases:

- **when the SVG is inlined in the HTML**
- when the image is loaded through `object`, `embed` or `iframe` tags

but (⚠️ depending on the browser implementation) they must be loaded from the same domain (and protocol), due to the same-origin policy.

`iframe` needs to be explicitly sized, otherwise the content is cropped, while `object` and `embed` resize to fit their content.

If the SVG is loaded using a `img` tag, or through CSS as a background, independently of the origin:

- CSS and JavaScript cannot interact with it
- JavaScript contained in the SVG is disabled
- External resources like images, stylesheets, scripts, fonts cannot be loaded

In details

Feature	Inline SVG	object / embed / iframe	img
Can interact with the user	✅	✅	✅
Supports animations	✅	✅	✅
Can run its own JavaScript	✅	✅	👎
Can be scripted from outside	✅	👎	👎

Inline SVG images are definitely the most powerful and flexible, and it's the only way to perform certain operations with SVG.

If you want to do any interaction with the SVG with your scripts, it must be loaded inline in the HTML.

Loading an SVG in an `img`, `object` or `embed` works if you don't need to interact with it, just show it in the page, and it's especially convenient if you reuse SVG images in different pages, or the SVG size is quite big.

CSS inside SVG

Add the CSS in a CDATA:

```
<svg>
  <style>
    <![CDATA[
      #my-rect { fill: blue; }
    ]]>
  </style>
  <rect id="my-rect" x="0" y="0" width="10" height="10" />
</svg>
```

An SVG file can also include an external style sheet:

```
<?xml version="1.0" standalone="no"?>
<?xml-stylesheet type="text/css" href="style.css"?>
<svg xmlns="http://www.w3.org/2000/svg" version="1.1"
  width=".." height=".." viewBox="..">
  <rect id="my-rect" x="0" y="0" width="10" height="10" />
</svg>
```

JavaScript inside SVG

You can put the JavaScript first, and wrap in in a `load` event to execute it when the page is fully loaded and the SVG is inserted in the DOM:

```
<svg>
  <script>
    <![CDATA[
      window.addEventListener("load", () => {
        //...
      }, false)
    ]]>
  </script>
```

```
<rect x="0" y="0" width="10" height="10" fill="blue" />
</svg>
```

or you can avoid adding an event listener if you put the JS at the end of the other SVG code, to make sure the JavaScript runs when the SVG is present in the page:

```
<svg>
  <rect x="0" y="0" width="10" height="10" fill="blue" />
  <script>
    <![CDATA[
      //...
    ]]>
  </script>
</svg>
```

SVG elements, just like html tags, can have `id` and `class` attributes, so we can use the Selectors API to reference them:

```
<svg>
  <rect x="0" y="0" width="10" height="10" fill="blue"
    id="my-rect" class="a-rect" />
  <script>
    <![CDATA[
      console.log(document.getElementsByTagName('rect'))
      console.log(document.getElementById('my-rect'))
      console.log(document.querySelector('.a-rect'))
      console.log(document.querySelectorAll('.a-rect'))
    ]]>
  </script>
</svg>
```

Check out this Glitch <https://flaviocopes-svg-script.glitch.me/> for an example of this functionality.

JavaScript outside the SVG

If you can interact with the SVG (the SVG is inline in the HTML), you can change any SVG attribute using JavaScript, for example:

```
document.getElementById("my-svg-rect").setAttribute("fill", "black")
```

or really do any other DOM manipulation you want.

CSS outside the SVG

You can change any styling of the SVG image using CSS.

SVG attributes can be easily overwritten in CSS, and they have a lower priority over CSS. They do not behave like inline CSS, which has higher priority.

```
<style>
  #my-rect {
    fill: red
  }
</style>
<svg>
  <rect x="0" y="0" width="10" height="10" fill="blue"
        id="my-rect" />
</svg>
```

SVG vs Canvas API

The Canvas API is a great addition to the Web Platform, and it has similar browser support as SVG. The main (and big) difference with SVG is that Canvas is not vector based, but rather pixel based, so

- it has the same scaling issues as pixel-based PNG, JPG and GIF image formats
- it makes it impossible to edit a Canvas image using CSS or JavaScript like you can do with SVG

SVG Symbols

Symbols let you define an SVG image once, and reuse it in multiple places. This is a great help if you need to reuse an image, and maybe just change a bit some of its properties.

You do so by adding a `symbol` element and assigning an `id` attribute:

```
<svg class="hidden">
  <symbol id="rectangle" viewBox="0 0 20 20">
    <rect x="0" y="0" width="300" height="300" fill="rgb(255,159,0)" />
  </symbol>
</svg>
```

```
<svg>
  <use xlink:href="#rectangle" href="#rectangle" />
</svg>

<svg>
```

```
<use xlink:href="#rectangle" href="#rectangle" />
</svg>
```

(`xlink:href` is for Safari support, even if it's a deprecated attribute)

This starts to give an idea of the power of SVG.

If you want to style those 2 rectangles differently, for example using a different color for each? You can use CSS Variables.

```
<svg class="hidden">
  <symbol id="rectangle" viewBox="0 0 20 20">
    <rect x="0" y="0" width="300" height="300" fill="var(--color)" />
  </symbol>
</svg>
```

```
<svg class="blue">
  <use xlink:href="#rectangle" href="#rectangle" />
</svg>

<svg class="red">
  <use xlink:href="#rectangle" href="#rectangle" />
</svg>

<style>
svg.red {
  --color: red;
}
svg.blue {
  --color: blue;
}
</style>
```

See my Glitch playground (<https://flavio-svg-symbols.glitch.me/>) on SVG symbols.

Validate an SVG

An SVG file, being XML, can be written in an invalid format, and some services or apps might not accept an invalid SVG file.

SVG can be validated using the W3C Validator (<https://validator.w3.org>).

Should I include the `xmlns` attribute?

Sometimes an `svg` is defined as

```
<svg>
  ...
</svg>
```

sometimes as

```
<svg version="1.1" xmlns="http://www.w3.org/2000/svg">
  ...
</svg>
```

This second form is XHTML. It can also be used with HTML5 (documents with `<!DOCTYPE html>`) but in this case the first form is simpler.

Should I worry about browser support?

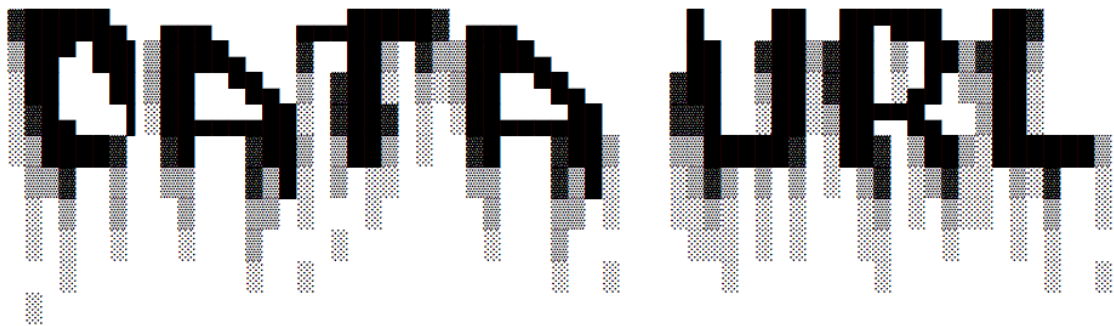
In 2018 SVG is supported by the vast majority of user's browsers.

You can still check for missing support using libraries like Modernizr (<https://modernizr.com/>), and provide a fallback:

```
if (!Modernizr.svg) {
  $(".my-svg").attr("src", "images/logo.png");
}
```

DATA URLS

A Data URL is a URI scheme that provides a way to inline data in a document, and it's commonly used to embed images in HTML and CSS



- Introduction
- How does a Data URL look
- Browser support
- Security

Introduction

A Data URL is a URI scheme that provides a way to inline data in an HTML document.

Say you want to embed a small image. You could go the usual way, upload it to a folder and use the `img` tag to make the browser reference it from the network:

```

```

or you can encode it in a special format, called Data URL, which makes it possible to embed the image directly in the HTML document, so the browser does not have to make a separate request to get it.

Data URLs might save some time for small files, but for bigger files there are downsides in the increased HTML file size, and they are especially a problem if the image reloads on all your pages, as you can't take advantage of the browser caching capabilities.

Also, an image encoded as Data URL is generally a bit bigger than the same image in binary format.

They aren't much practical if your images are frequently edited, since every time the image is changed, it must be encoded again.

That said, they have their place on the Web Platform.

How does a Data URL look

A Data URL is a string that starts with `data:` followed by the MIME type format. For example a PNG image has mime type `image/png`.

This is followed by a comma and then by the actual data.

Text is usually transferred in plain text, while binary data is usually base64 encoded.

Here is an example of how such Data URL will look like:

```

```

And here is a small version of the banner image of this article encoded in a link

Here is how a base64 encoded Data URL looks like. Notice it starts with `data:image/png;base64:`

```

```

And here is a small version of the banner image of this article base64 encoded in a link
[\(data:image/png;base64,iVBORw0KGgoAAAANSUheEUgAAAJYAAABjCMAAAABOgAI6AAADAFBMVEX///](data:image/png;base64,iVBORw0KGgoAAAANSUheEUgAAAJYAAABjCMAAAABOgAI6AAADAFBMVEX///)
.

This site has a very nice Data URL generator: <https://dopiaza.org/tools/datauri/index.php> which you can use to transform any image sitting in your desktop to a Data URL.

Data URLs can be used anywhere a URL can be used, as you saw you can use it for links, but it's also common to use them in CSS:

```
.main {  
  background-image url('data:image/png;base64,iVBORw0KGgoAA...');  
}
```

Browser support

They are supported in all modern browsers (<https://caniuse.com/#feat=datauri>) .

Security

Data URLs can encode any kind of information, not just images, and so they come with their set of security implications.

From Wikipedia (https://en.wikipedia.org/wiki/Data_URI_scheme) :

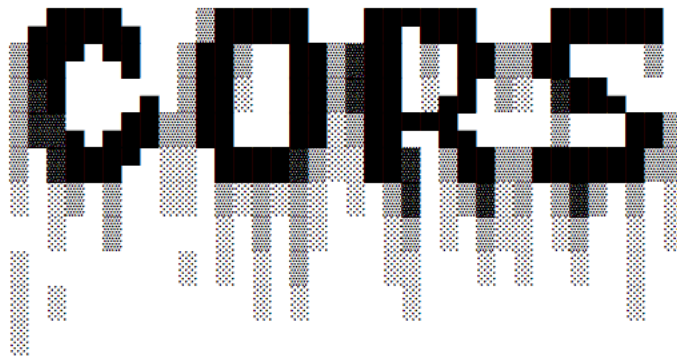
The data URI can be utilized to construct attack pages that attempt to obtain usernames and passwords from unsuspecting web users. It can also be used to get around cross-site scripting (XSS) restrictions, embedding the attack payload fully inside the address bar, and hosted via URL shortening services rather than needing a full website that is controlled by a third party.

Check this article from the Mozilla Firefox Blog (<https://blog.mozilla.org/security/2017/11/27/blocking-top-level-navigations-data-urls-firefox-59/>) for more information on how Data URLs can be used by malicious users to do bad things, and how the Firefox browser mitigates such risks.

Data URLs are defined in RFC 2397 (<https://tools.ietf.org/html/rfc2397>) .

CORS

An introduction to Cross-Origin Resource Sharing, the way to let clients and servers communicate even if they are not on the same domain



A JavaScript application running in the browser can usually only access HTTP resources on the same domain (origin) that serves it.

Loading images or scripts/styles always works, but XHR and Fetch calls to another server will fail, unless that server implements a way to allow that connection.

This way is called CORS, **Cross-Origin Resource Sharing**.

Also loading Web Fonts using `@font-face` has same-origin policy by default, and other less popular things (like WebGL textures and `drawImage` resources loaded in the Canvas API).

One very important thing that needs CORS is **ES Modules**, recently introduced in modern browsers.

If you don't set up a CORS policy **on the server** that allows to serve 3rd part origins, the request will fail.

Fetch example:

```
> fetch('https://google.com')
< ▶ Promise {<pending>}
✖ Failed to load https://google.com/: Redirect from https://google.com/ to https://www.google.it/?gfe_rd=cr&dcr=0&ei=TiDHWtehBcPCXprvpIgF' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. Origin 'https://flaviocopes.com' is therefore not allowed access. If an opaque response serves your needs, set the request's mode to 'no-cors' to fetch the resource with CORS disabled.
✖ Uncaught (in promise) TypeError: Failed to fetch
```

XHR example:

```
> const xhr = new XMLHttpRequest()
  xhr.onreadystatechange = () => {
    if (xhr.readyState === 4) {
      xhr.status === 200 ? console.log(xhr.responseText) : console.error('error')
    }
  }
  xhr.open('GET', 'https://google.com')
  xhr.send()
< undefined
✖ Failed to load https://google.com/: Redirect from 'https://google.com/' to 'https://www.google.it/?gfe_rd=cr&dcr=0&ei=3yDHWvnPN9LCXsWxp9AN' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. Origin 'http://localhost:1313' is therefore not allowed access.
✖ ▶ error
▶ XHR finished loading: GET "https://google.com/".
```

A Cross-Origin resource fails if it's:

- to a different **domain**
- to a different **subdomain**
- to a different **port**
- to a different **protocol**

and it's there for your security, to prevent malicious users to exploit the Web Platform.

But if you control both the server and the client, you have all the good reasons to allow them to talk to each other.

How?

It depends on your server-side stack.

Browser support

Pretty good (basically all except IE<10):

IE	Edge *	Firefox	Chrome	Safari	Opera	iOS Safari *	Opera Mini *	Android *	Blackberry	Opera Mobile *	Chrome Android
6		54	60	³ 8	45	³ 8.4		¹ 4			
7	12	55	61	³ 9	46	³ 9.2		¹ 4.1			
² 8	13	56	62	³ 9.1	47	³ 9.3		¹ 4.3			
² 9	14	57	63	³ 10	48	³ 10.2		4.4		12	
¹ 10	15	58	64	³ 10.1	49	³ 10.3		4.4.4	¹ 7	12.1	
11	16	59	65	³ 11	50	³ 11.2	all	62	10	37	64
	17	60	66	³ 11.1	51	³ 11.3					
		61	67	³ TP	52						
			68								

Example with Express

If you are using Node.js and Express as a framework, use the CORS middleware package (<https://github.com/expressjs/cors>) .

Here's a simple implementation of an Express Node.js server:

```
const express = require('express')
const app = express()

app.get('/without-cors', (req, res, next) => {
  res.json({msg: '😞 no CORS, no party!'})
})

const server = app.listen(3000, () => {
  console.log("Listening on port %s", server.address().port)
})
```

If you hit `/without-cors` with a fetch request from a different origin, it's going to raise the CORS issue.

All you need to do to make things work out is to require the `cors` package linked above, and pass it in as a middleware function to an endpoint request handler:

```
const express = require('express')
const cors = require('cors')
const app = express()

app.get('/with-cors', cors(), (req, res, next) => {
  res.json({msg: 'WHOA! with CORS it works! 🎉 🏆'})
})
```

```
}D
```

```
/* the rest of the app */
```

I made a simple Glitch example. Here is the client (<https://flavio-cors-client.glitch.me/>) working, and here's its code: <https://glitch.com/edit/#!/flavio-cors-client>.

This is the Node.js Express server: <https://glitch.com/edit/#!/flaviocopes-cors-example-express>

Note how the request that fails because it does not handle the CORS headings correctly is still received, as you can see in the Network panel, where you find the message the server sent:

Name	× Headers Preview Response Timing
 flavio-cors-client api.glitch.com/proj...	▼ {msg: "😞 no CORS, no party!"} msg: "😞 no CORS, no party!"
 without-cors flaviocopes-cors-e...	
 with-cors flaviocopes-cors-e...	
3 / 11 requests 1.2 KB / ...	

Allow only specific origins

This example has a problem however: ANY request will be accepted by the server as cross-origin.

As you can see in the Network panel, the request that passed has a response header `access-control-allow-origin: *`:

Name	× Headers Preview Response Timing
 flavio-cors-client api.glitch.com/proj...	Referrer Policy: no-referrer-when-downgrade
 without-cors flaviocopes-cors-e...	▼ Response Headers access-control-allow-origin: * content-length: 45 content-type: application/json; charset=utf-8 date: Fri, 06 Apr 2018 16:10:18 GMT etag: W/"2d-0UnEPhCoz1V3cgJ99D+m3n/b0p4" status: 200 x-powered-by: Express
 with-cors flaviocopes-cors-e...	
3 / 11 requests 1.2 KB / ...	

You need to configure the server to only allow one origin to serve, and block all the others.

Using the same `cors` Node library, here's how you would do it:

```
const cors = require('cors')

const corsOptions = {
  origin: 'https://yourdomain.com',
}

app.get('/products/:id', cors(corsOptions), (req, res, next) => {
  //...
})
```

You can serve more as well:

```
const whitelist = ['http://example1.com', 'http://example2.com']
const corsOptions = {
  origin: function (origin, callback) {
    if (whitelist.indexOf(origin) !== -1) {
      callback(null, true)
    } else {
      callback(new Error('Not allowed by CORS'))
    }
  }
}
```

Preflight

There are some requests that are handled in a “simple” way. All `GET` requests belong to this group.

Also *some* `POST` and `HEAD` requests do as well.

`POST` requests are also in this group, if they satisfy the requirement of using a Content-Type of

- `application/x-www-form-urlencoded`
- `multipart/form-data`
- `text/plain`

All other requests must run through a pre-approval phase, called preflight. The browser does this to determine if it has the permission to perform an action, by issuing an `OPTIONS` request.

A preflight request contains a few headers that the server will use to check permissions (irrelevant fields omitted):

```
OPTIONS /the/resource/you/request
Access-Control-Request-Method: POST
Access-Control-Request-Headers: origin, x-requested-with, accept
Origin: https://your-origin.com
```

The server will respond with something like this(irrelevant fields omitted):

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://your-origin.com
Access-Control-Allow-Methods: POST, GET, OPTIONS, DELETE
```

We checked for POST, but the server tells us we can also issue other HTTP request types for that particular resource.

Following the Node.js Express example above, the server must also handle the OPTIONS request:

```
var express = require('express')
var cors = require('cors')
var app = express()

//allow OPTIONS on just one resource
app.options('/the/resource/you/request', cors())

//allow OPTIONS on all resources
app.options('*', cors())
```

WEB WORKERS

Learn the way to run JavaScript code in the background using Web Workers



- Introduction
- Browser support for Web Workers
- Create a Web Worker
- Communication with a Web Worker
 - Using `postMessage` in the Web Worker object
 - Send back messages
 - Multiple event listeners
 - Using the Channel Messaging API
- Web Worker Lifecycle
- Loading libraries in a Web Worker
- APIs available in Web Workers

Introduction

JavaScript is single threaded. Nothing can run in parallel at the same time.

This is great because we don't need to worry about a whole set of issues that would happen with concurrent programming.

With this limitation, JavaScript code is forced to be efficient from the start, otherwise the user would have a bad experience. Expensive operations should be asynchronous to avoid blocking the thread.

As the needs of JavaScript applications grew, this started to become a problem in some scenarios.

Web Workers introduce the possibility of parallel execution inside the browser.

They have quite a few limitations:

- no access to the DOM: the Window object and the Document object
- they can communicate back with the main JavaScript program using messaging
- they need to be loaded from the same origin (domain, port and protocol)
- they don't work if you serve the page using the file protocol (`file://`)

The global scope of a Web Worker, instead of Window which is in the main thread, is a `WorkerGlobalScope` (<https://developer.mozilla.org/en-US/docs/Web/API/WorkerGlobalScope>) object.

Browser support for Web Workers

Pretty good!

IE	Edge *	Firefox	Chrome	Safari	Opera	iOS Safari *	Opera Mini *	Android *
		53	59	7.1	44	8		3
6		54	60	8	45	8.4		4
7	12	55	61	9	46	9.2		4.1
8	13	56	62	9.1	47	9.3		4.3
9	14	57	63	10	48	10.2		4.4
10	15	58	64	10.1	49	10.3		4.4.4
11	16	59	65	11	50	11.2	all	62
	17	60	66	11.1	51	11.3		
	18	61	67	TP	52			
			68					

You can check for Web Workers support using

```
if (typeof Worker !== 'undefined') {
}
```

Create a Web Worker

You create a Web Worker by initializing a Worker object, loading a JavaScript file from the same origin:

```
const worker = new Worker('worker.js')
```

Communication with a Web Worker

There are two main ways to communicate to a Web Worker:

- the `postMessage` API offered by the Web Worker object
- the Channel Messaging API

Using `postMessage` in the Web Worker object

You can send messages using `postMessage` on the `Worker` object.

Important: a message is transferred, not shared.

main.js

```
const worker = new Worker('worker.js')
worker.postMessage('hello')
```

worker.js

```
worker.onmessage = (e) => {
  console.log(e.data)
}

worker.onerror = (e) => {
  console.error(e.message)
}
```

Send back messages

A worker can send back messages to the function that created it. using

`worker.postMessage()` :

worker.js

```
worker.onmessage = (e) => {
  console.log(e.data)
  worker.postMessage('hey')
}

worker.onerror = (e) => {
  console.error(e.message)
}
```

main.js

```
const worker = new Worker('worker.js')
worker.postMessage('hello')

worker.onmessage = (e) => {
  console.log(e.data)
}
```

Multiple event listeners

If you want to setup multiple listeners for the `message` event, instead of using `onmessage` create an event listener (applies to the `error` event as well):

worker.js

```
worker.addEventListener('message', (e) => {
  console.log(e.data)
  worker.postMessage('hey')
}, false)

worker.addEventListener('message', (e) => {
  console.log(`I'm curious and I'm listening too`)
}, false)

worker.addEventListener('error', (e) => {
  console.log(e.message)
}, false)
```

main.js

```
const worker = new Worker('worker.js')
worker.postMessage('hello')

worker.addEventListener('message', (e) => {
  console.log(e.data)
}, false)
```

Using the Channel Messaging API

Since Workers are isolated from the main thread, they have their own, to communicate to them we need a special API: the Channel Messaging API.

main.js

```
const worker = new Worker('worker.js')
const messageChannel = new MessageChannel();
messageChannel.port1.addEventListener('message', (e) => {
  console.log(e.data)
})
worker.postMessage(data, [messageChannel.port2])
```

worker.js

```
self.addEventListener('message', (e) => {  
  console.log(e.data)  
})
```

A Web Worker can send messages back by posting a message to `messageChannel.port2`, like this:

```
self.addEventListener('message', (event) => {  
  event.ports[0].postMessage(data)  
})
```

Web Worker Lifecycle

Web Workers are launched and if they do not stay in listening mode for messages through `worker.onmessage` or by adding an event listener, they will be shut down as soon as their code is run through completion.

A Web Worker can be stopped using its `terminate()` method from the main thread, and inside the worker itself using the global method `close()`:

main.js

```
const worker = new Worker('worker.js')  
worker.postMessage('hello')  
worker.terminate()
```

worker.js

```
worker.onmessage = (e) => {  
  console.log(e.data)  
  close()  
}  
  
worker.onerror = (e) => {  
  console.error(e.message)  
}
```

Loading libraries in a Web Worker

Web Workers can use the `importScripts()` global function defined in their global scope:

```
importScripts('../utils/file.js', '../something.js')
```

APIs available in Web Workers

As said before, the DOM is not reachable by a Web Worker, so you cannot interact with the `window` and `document` objects. Also `parent` is unavailable.

You can however use many other APIs, which include:

- the XHR API
- the Fetch API
- the Broadcast Channel API
- the FileReader API
- IndexedDB
- the Notifications API
- Promises
- Service Workers
- the Channel Messaging API
- the Cache API
- the Console API (`console.log()` and friends)
- the JavaScript Timers (`setTimeout`, `setInterval`...)
- the CustomEvents API: `addEventListener()` and `removeEventListener()`
- the current URL, which you can access through the `location` property in read mode
- WebSockets
- WebGL
- SVG Animations

REQUESTANIMATIONFRAME()

Learn the API to perform animations and schedule event in a predictable way

`requestAnimationFrame()` is a relatively recent browser API. It gives a more predictable way to hook into the browser render cycle.

It's currently supported by all modern browsers (and IE 10+)

IE	Edge *	Firefox	Chrome	Safari	Opera	iOS Safari *	Opera Mini *	Android *
		55	59	7.1	44	8		5
6		54	60	8	45	8.4		4
7	12	55	61	9	46	9.2		4.1
8	13	56	62	9.1	47	9.3		4.3
9	14	57	63	10	48	10.2		4.4
10	15	58	64	10.1	49	10.3		4.4.4
11	16	59	65	11	50	11.2	all	62
	17	60	66	11.1	51	11.3		
		61	67	TP	52			
			68					

It's not an API specific to animations, but that's where it is used the most.

JavaScript has an event loop. It continuously runs to execute JavaScript.

In the past, animations were performed using `setTimeout()` or `setInterval()`. You perform a little bit of an animation, and you call `setTimeout()` to repeat again this code in a few milliseconds from now:

```
const performAnimation = () => {  
  //...  
  setTimeout(performAnimation, 1000 / 60)
```

```
}
setTimeout(performAnimation, 1000 / 60)
```

or

```
const performAnimation = () => {
  //...
}
setInterval(performAnimation, 1000 / 60)
```

You can stop an animation by getting the timeout or interval reference, and clearing it:

```
let timer

const performAnimation = () => {
  //...
  timer = setTimeout(performAnimation, 1000 / 60)
}

timer = setTimeout(performAnimation, 1000 / 60)

//...

clearTimeout(timer)
```

The 1000 / 60 interval between the `performAnimation()` calls is determined by the monitor refresh rate, which is in most of the cases 60 Hz (60 repaints per second), because it's useless to perform a repaint if the monitor cannot show it due to its limitations. It leads to ~16.6ms of time we have at our disposal to display every single frame.

The problem with this approach is that even though we specify this precision accurately, the browser might be busy performing other operations, and our `setTimeout` calls might not make it in time for the repaint, and it's going to be delayed to the next cycle.

This is bad because we lose one frame, and in the next the animation is performed 2 times, causing the eye to notice the clunky animation.

Check this example on Glitch of an animation built using of `setTimeout()` (<https://flavio-settimeout-animation.glitch.me/>) .

`requestAnimationFrame` is the standard way to perform animations, and it works in a very different way even though the code looks very similar to the `setTimeout/setInterval` code:

```
let request
```

```
const performAnimation = () => {
  request = requestAnimationFrame(performAnimation)
  //animate something
}

requestAnimationFrame(performAnimation)

//...

cancelAnimationFrame(request) //stop the animation
```

This example on Glitch of an animation built using of requestAnimationFrame() (<https://flavio-requestanimationframe-example.glitch.me/>) shows how

Optimization

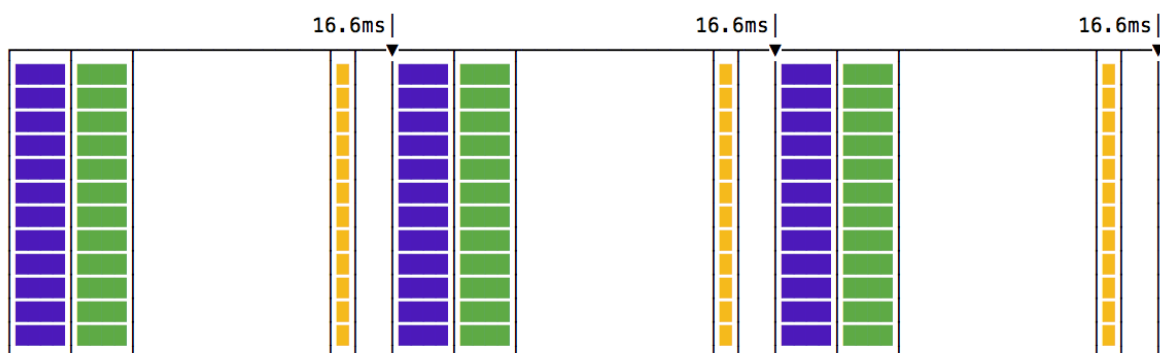
requestAnimationFrame() since its introduction was very CPU friendly, causing animations to stop if the current window or tab is not visible.

At the time requestAnimationFrame() was introduced, setTimeout/setInterval did run even if the tab was hidden, but now since this approach proved to be successful also to battery savings, browsers also implemented throttling for those events, allowing max 1 execution per each second.

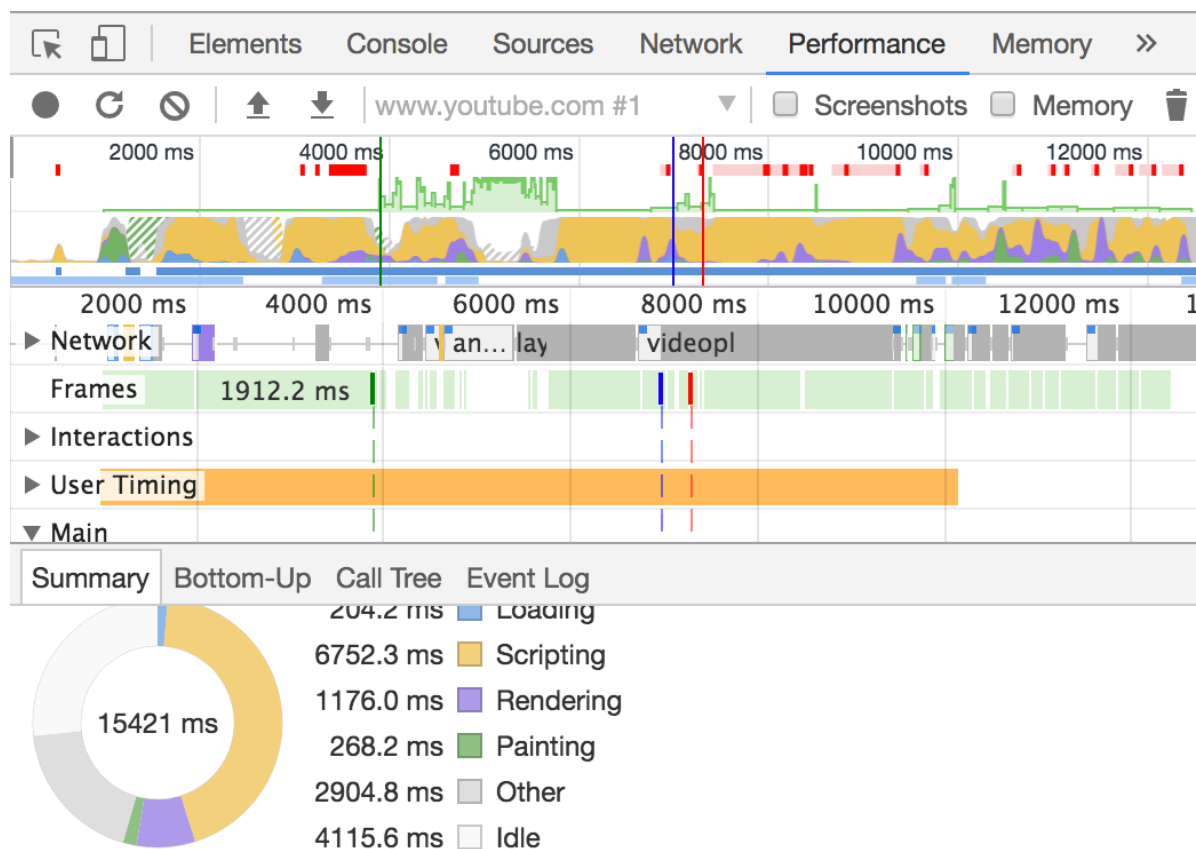
Using requestAnimationFrame the browser can further optimize the resource consumption and make the animations smoother.

Timeline examples

This is the perfect timeline if you use setTimeout or setInterval:

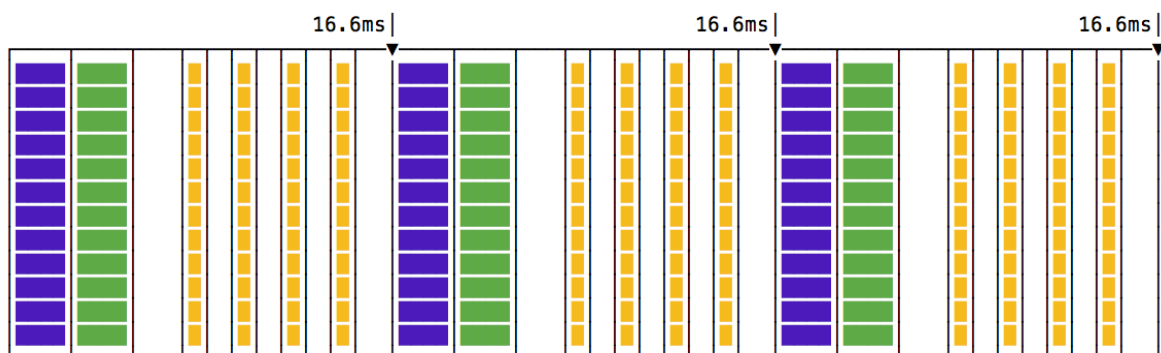


you have a set of paint (green) and render (purple) events, and your code is in the yellow box - by the way, these are the colors used in the Browser DevTools as well to represent the timeline:



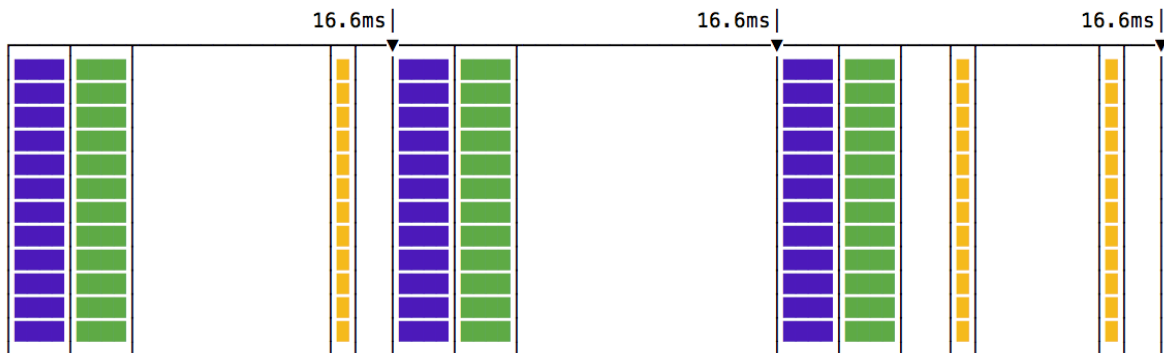
The illustration shows the perfect case. You have painting and rendering every 60ms, and your animation happens in between, perfectly regular.

If you used a higher frequency call for your animation function:

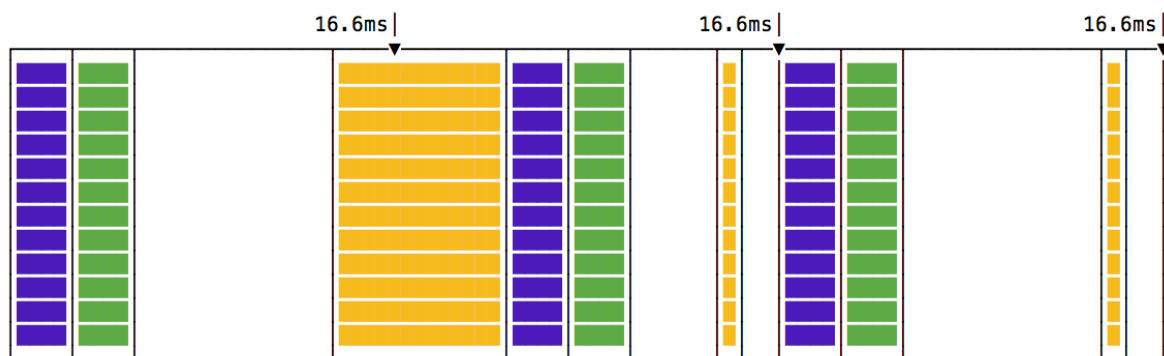


Notice how in each frame we call 4 animation steps, before any rendering happens, and this will make the animation feel very choppy.

What if `setTimeout` cannot run on time due to other code blocking the event loop? We end up with a missed frame:

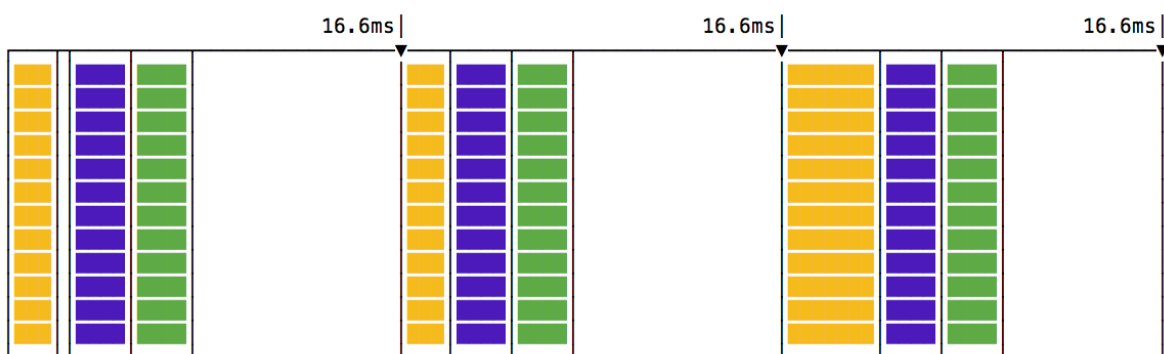


What if an animation step takes a little bit more than you anticipate?



the render and paint events will be delayed as well.

This is how `requestAnimationFrame()` works visually:



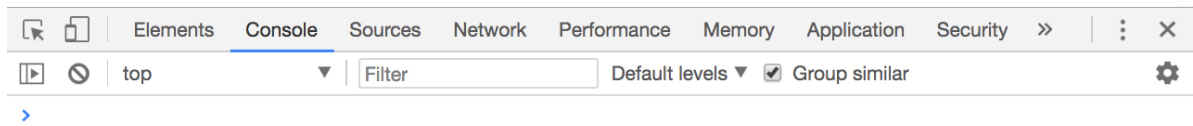
all the animation code runs **before** the rendering and painting events. This makes for a more predictable code, and there's a lot of time to do the animation without worrying about going past the 16ms time we have at our disposal.

THE CONSOLE API

Every browser exposes a console that lets you interact with the Web Platform APIs and also gives you an inside look at the code by printing messages that are generated by your JavaScript code running in the page



Every browser exposes a console that lets you interact with the Web Platform APIs and also gives you an inside look at the code by printing messages that are generated by your JavaScript code running in the page.

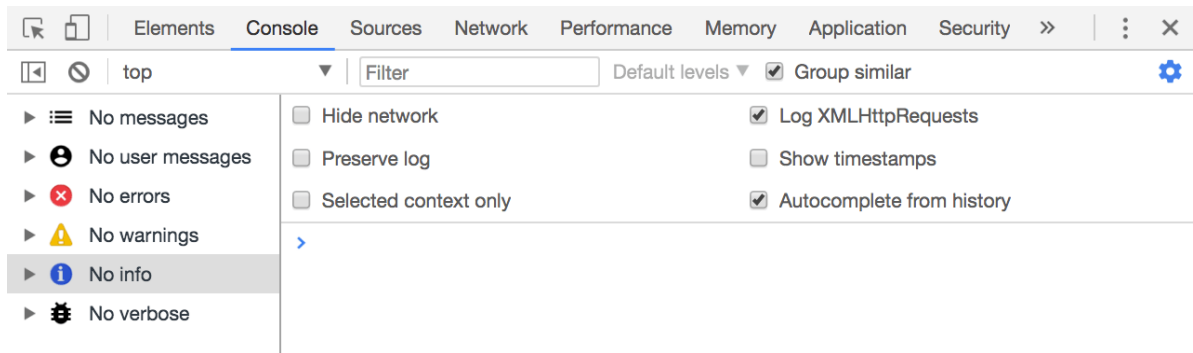


- Overview of the console
- Use console.log formatting
- Clear the console
- Counting elements
- Log more complex objects
- Logging different error levels
- Preserve logs during navigation
- Grouping console messages
- Print the stack trace
- Calculate the time spent
- Generate a CPU profile

Overview of the console

The console toolbar is simple. There's a button to clear the console messages, something you can also do by clicking `cmd-K` in macOS, or `ctrl-K` on Windows, a second button that activates a filtering sidebar, that lets you filter by text, or by type of message, for example error, warning, info, log, or debug messages.

You can also choose to hide network-generated messages, and just focus on the JavaScript log messages.



The console is not just a place where you can see messages, but also the best way to interact with JavaScript code, and many times the DOM. Or, just get information from the page.

Let's type our first message. Notice the `>`, let's click there and type

```
console.log('test')
```

The console acts as a **REPL**, which means read-eval-print loop. In short, it interprets our JavaScript code and prints something.

Use console.log formatting

As you see, `console.log('test')` prints 'test' in the Console.

Using `console.log` in your JavaScript code can help you debug for example by printing static strings, but you can also pass it a variable, which can be a JavaScript native type (for example an integer) or an object.

You can pass multiple variables to `console.log`, for example:

```
console.log('test1', 'test2')
```

We can also format pretty phrases by passing variables and a format specifier.

For example:

```
console.log("My %s has %d years", 'cat', 2)
```

- `%s` format a variable as a string
- `%d` or `%i` format a variable as an integer
- `%f` format a variable as a floating point number

- %o can be used to print a DOM Element
- %o used to print an object representation

Example:

```
console.log("%o, %o", document.body, document.body)
```

```
> console.log("%o, %o", document.body, document.body)
```

VM2945:1

```
▼ <body>
  ▶ <script>...</script>
  ▶ <div id="wrapper">...</div>
  ▶ <script data-no-instant>...</script>
  ▶ <script src="/livereload.js?port=1313&mindelay=10"></script>
  ▶ <iframe scrolling="no" frameborder="0" allowtransparency="true" src="https://platform.twitter.com/widgets/widget_iframe.73a792b...html?origin=http%3A%2F%2Flocalhost%3A1313" style="display: none;">...</iframe>
  </body>
,
▼ body ⓘ
  aLink: ""
  accessKey: ""
  assignedSlot: null
  ▶ attributes: NamedNodeMap {length: 0}
  background: ""
  baseURI: "http://localhost:1313/console-api/"
  bgColor: ""
  childElementCount: 5
  ▶ childNodes: NodeList(9) [text, script, text, div#wrapper, text, script, script, text, iframe]
  ▶ children: HTMLCollection(5) [script, div#wrapper, script, script, iframe, wrapper: div#wrapper]
  ▶ classList: DOMTokenList [value: ""]
  className: ""
  clientHeight: 496
```

Another useful format specifier is %c , which allows to pass CSS to format a string. For example:

```
console.log("%c My %s has %d years", "color: yellow; background:black; font-size: 16pt", "cat", 2)
```

```
> console.log("%c My %s has %d years", "color: yellow; background:black; font-size: 16pt", "cat", 2)
```

My cat has 2 years

VM3550:1

Clear the console

There are three ways to clear the console while working on it, with various input methods.

The first way is to click the **Clear Console Log** button on the console toolbar.

The second method is to type `console.clear()` inside the console, or in your a JavaScript function that runs in your app / site.

You can also just type `clear()` .

The third way is through a keyboard shortcut, and it's `cmd-k` (mac) or `ctrl + l` (Win)

Counting elements

`console.count()` is a handy method.

Take this code:

```
const x = 1
const y = 2
const z = 3
console.count("The value of x is " + x + " and has been checked .. how many times?")
console.count("The value of x is " + x + " and has been checked .. how many times?")
console.count("The value of y is " + y + " and has been checked .. how many times?")
```

What happens is that count will count the number of times a string is printed, and print the count next to it:

```
> const x = 1
   const y = 2
   const z = 3
   console.count("The value of x is " + x + " and has
   been checked .. how many times?")
   console.count("The value of x is " + x + " and has
   been checked .. how many times?")
   console.count("The value of y is " + y + " and has
   been checked .. how many times?")
```

```
The value of x is 1 and has been checked .. VM5252:4
how many times?: 1
```

```
The value of x is 1 and has been checked .. VM5252:5
how many times?: 2
```

```
The value of y is 2 and has been checked .. VM5252:6
how many times?: 1
```

You can just count apples and oranges:

```
const oranges = ['orange', 'orange']
const apples = ['just one apple']
oranges.forEach((fruit) => {
  console.count(fruit)
})
apples.forEach((fruit) => {
  console.count(fruit)
})
```

```
> const oranges = ['orange', 'orange']
  const apples = ['just one apple']
  oranges.forEach((fruit) => {
    console.count(fruit)
  })
  apples.forEach((fruit) => {
    console.count(fruit)
  })
```

orange: 1

VM5502:4

orange: 2

VM5502:4

just one apple: 1

VM5502:7

Log more complex objects

`console.log` is pretty amazing to inspect variables. You can pass it an object too, and it will do its best to print it to you in a readable way. Most of the times this means it prints a string representation of the object.

For example try

```
console.log([1, 2])
```

Another option to print objects is to use `console.dir`:

```
console.dir([1, 2])
```

As you can see this method prints the variable in a JSON-like representation, so you can inspect all its properties.

The same thing that `console.dir` outputs is achievable by doing

```
console.log("%0", [1,2])
```

```
> console.log([1, 2])
▼ (2) [1, 2] i VM6302:1
  0: 1
  1: 2
  length: 2
  ▶ __proto__: Array(0)
< undefined
> console.dir([1, 2])
▼ Array(2) i VM6308:1
  0: 1
  1: 2
  length: 2
  ▶ __proto__: Array(0)
< undefined
>
```

Which one to use depends on what you need to debug of course, and one of the two can do the best job for you.

Another function is `console.table()` which prints a nice table.

We just need to pass it an array of elements, and it will print each array item in a new row.

For example

```
console.table([[1,2], ['x', 'y']])
```

or you can also set column names, by passing instead of an array, an Object Literal, so it will use the object property as the column name

```
console.table([{'x': 1, 'y': 2, 'z': 3}, {'x': "First column", 'y': "Second column", 'z': null}])
```

```
> console.table([[1,2], ['x', 'y']])
```

VM6933:1

(index)	0	1
0	1	2
1	"x"	"y"

► Array(2)

< undefined

```
> console.table([x: 1, y: 2, z: 3], {x: "First column", y: "Second column", z: null})
```

VM6939:1

(index)	x	y	z
0	1	2	3
1	"First col..."	"Second co..."	null

► Array(2)

< undefined

>

`console.table` can also be more powerful and if you pass it an object literal that in turn contains an object, and you pass an array with the column names, it will print a table with the row indexes taken from the object literal. For example:

```
const shoppingCart = {}
shoppingCart.firstItem = {'color': 'black', 'size': 'L'}
shoppingCart.secondItem = {'color': 'red', 'size': 'L'}
shoppingCart.thirdItem = {'color': 'white', 'size': 'M'}
console.table(shoppingCart, ["color", "size"])
```

```
> const shoppingCart = {}
  shoppingCart.firstItem = {'color': 'black', 'size':
    'L'}
  shoppingCart.secondItem = {'color': 'red', 'size':
    'L'}
  shoppingCart.thirdItem = {'color': 'white', 'size':
    'M'}
  console.table(shoppingCart, ["color", "size"])
```

VM7490:5

(index)	color	size
firstItem	"black"	"L"
secondItem	"red"	"L"
thirdItem	"white"	"M"

► Object

< undefined

Logging different error levels

As we saw `console.log` is great for printing messages in the Console.

We'll now discover three more handy methods that will help us debug, because they implicitly indicate various levels of error.

First, **`console.info()`**

As you can see a little 'i' is printed beside it, making it clear the log message is just an information.

Second, **`console.warn()`**

prints a yellow exclamation point.

If you activate the Console filtering toolbar, you can see that the Console allows you to filter messages based on the type, so it's really convenient to differentiate messages because for example if we now click 'Warnings', all the printed messages that are not warnings will be hidden.

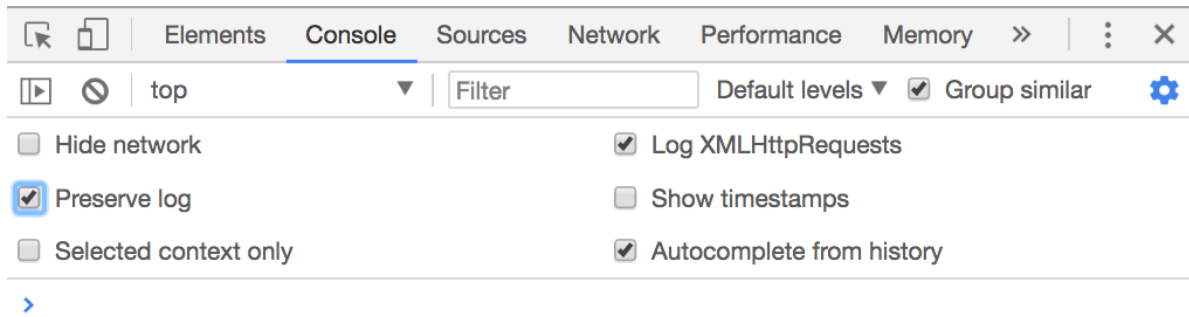
The third function is **`console.error()`**

this is a bit different than the others because in addition to printing a red X which clearly states there's an error, we have the full stack trace of the function that generated the error, so we can go and try to fix it.

```
> console.info('test')
test VM8466:1
< undefined
> console.warn('test')
⚠ ▶ test VM8479:1
< undefined
> console.error('test')
✖ ▼ test VM8489:1
  (anonymous) @ VM8489:1
< undefined
> |
```

Preserve logs during navigation

Console messages are cleared on every page navigation, unless you check the **Preserve log** in the console settings:



Grouping console messages

The Console messages can grow in size and the noise when you're trying to debug an error can be overwhelming.

To limit this problem the Console API offers a handy feature: Grouping the Console messages.

Let's do an example first.

```
console.group("Testing the location")
console.log("Location hash", location.hash)
console.log("Location hostname", location.hostname)
console.log("Location protocol", location.protocol)
console.groupEnd()
```

```
> console.group("Testing the location")
   console.log("Location hash", location.hash)
   console.log("Location hostname", location.hostname)
   console.log("Location protocol", location.protocol)
   console.groupEnd()
▼ Testing the location
  Location hash
  Location hostname localhost
  Location protocol http:
< undefined
```

As you can see the Console creates a group, and there we have the Log messages.

You can do the same, but output a collapsed message that you can open on demand, to further limit the noise:

```
console.groupCollapsed("Testing the location")
console.log("Location hash", location.hash)
console.log("Location hostname", location.hostname)
console.log("Location protocol", location.protocol)
console.groupEnd()
```

```
> console.groupCollapsed("Testing the location")
  console.log("Location hash", location.hash)
  console.log("Location hostname", location.hostname)
  console.log("Location protocol", location.protocol)
  console.groupEnd()
▶ Testing the location
< undefined
```

The nice thing is that those groups can be nested, so you can end up doing

```
console.group("Main")
console.log("Test")
console.group("1")
console.log("1 text")
console.group("1a")
console.log("1a text")
console.groupEnd()
console.groupCollapsed("1b")
console.log("1b text")
console.groupEnd()
console.groupEnd()
```

```
> console.group("Main")
  console.log("Test")
  console.group("1")
  console.log("1 text")
  console.group("1a")
  console.log("1a text")
  console.groupEnd()
  console.groupCollapsed("1b")
  console.log("1b text")
  console.groupEnd()
  console.groupEnd()
```

```
▼ Main
  |   Test
  |   ▼ 1
  |   |   1 text
  |   |   ▼ 1a
  |   |   |   1a text
  |   |   ▶ 1b
  |   < undefined
```

Print the stack trace

There might be cases where it's useful to print the call stack trace of a function, maybe to answer the question *how did you reach that part of code?*

You can do so using `console.trace()`:

```
const function2 = () => console.trace()
const function1 = () => function2()
function1()
```

```
> const function2 = () => console.trace()
  const function1 = () => function2()
  function1()
▼ console.trace
  function2 @ VM9861:1
  function1 @ VM9861:2
  (anonymous) @ VM9861:3
< undefined
```

Calculate the time spent

You can easily calculate how much time a function takes to run, using `time()` and `timeEnd()`

```
const doSomething = () => console.log('test')
const measureDoingSomething = () => {
  console.time('doSomething()')
  //do something, and measure the time it takes
  doSomething()
  console.timeEnd('doSomething()')
}
measureDoingSomething()
```

```
> const doSomething = () => console.log('test')
  const measureDoingSomething = () => {
    console.time('doSomething()')
    //do something, and measure the time it takes
    doSomething()
    console.timeEnd('doSomething()')
  }
  measureDoingSomething()
test
doSomething(): 0.530029296875ms
```

Generate a CPU profile

The DevTools allow you to analyze the CPU profile performance of any function.

You can start that manually, but the most accurate way to do so is to wrap what you want to monitor between the `profile()` and `profileEnd()` commands. They are similar to `time()` and `timeEnd()`, except they don't just measure time, but create a more detailed report.

```
const doSomething = () => console.log('test')
const measureDoingSomething = () => {
  console.profile("doSomething()")
  //do something, and measure its performance
  doSomething()
  console.profileEnd()
}
measureDoingSomething()
```

The screenshot shows the Chrome DevTools JavaScript Profiler. The top bar includes tabs for Elements, Console, Sources, Network, and JavaScript Profiler. The profiler is set to 'Heavy (Bottom Up)' view. The main table displays the performance profile for the function 'doSomething()'. The table has columns for 'Self Time', 'Total Time', and 'Function'. The 'Self Time' and 'Total Time' are both 110.3 ms, representing 100.00% of the total time. The 'Function' column shows the call stack: '(program)', '(anonymous) VM10940:1', and 'measureDoingSomething VM10940:2'. The 'doSomething()' function is highlighted in blue. Below the table, there are tabs for Console, Rendering, What's New, and Coverage. The Console tab is active, showing the output of the code: 'test', 'Profile 'doSomething()' started.', 'undefined', and 'Profile 'doSomething()' finished.'.

Profiles	Self Time	Total Time	Function
CPU PROFILES	110.3 ms 100.00...	110.3 ms 100.00...	(program)
	0 ms 0 %	0 ms 0 %	(anonymous) VM10940:1
doSomething() Save	0 ms 0 %	0 ms 0 %	▶ measureDoingSomething VM10940:2

```
> const doSomething = () => console.log('test')
const measureDoingSomething = () => {
  console.profile("doSomething()")
  //do something, and measure its performance
  doSomething()
  console.profileEnd()
}
measureDoingSomething()
test
Profile 'doSomething()' started.
undefined
Profile 'doSomething()' finished.
```

WEBSOCKETS

WebSockets are an alternative to HTTP communication in Web Applications. They offer a long lived, bidirectional communication channel between client and server. Learn how to use them to perform network interactions

WebSockets are an alternative to HTTP communication in Web Applications.

They offer a long lived, bidirectional communication channel between client and server.

Once established, the channel is kept open, offering a very fast connection with low latency and overhead.

Browser support for WebSockets

WebSockets are supported by all modern browsers.

IE	Edge [*]	Firefox	Chrome	Safari	Opera	iOS Safari [*]
		53	59	8	44	8.4
6		54	60	9	45	9.2
7	12	55	61	9.1	46	9.3
8	13	56	62	10	47	10.2
9	14	57	63	10.1	48	10.3
10	15	58	64	11	49	11.2
11	16	59	65	11.1	50	11.3
	17	60	66	TP	51	
	18	61	67		52	
			68			

How WebSockets differ from HTTP

HTTP is a very different protocol, and also a different way of communicate.

HTTP is a request/response protocol: the server returns some data when the client requests it.

With WebSockets:

- the **server can send a message to the client** without the client explicitly requesting something
- the client and the server can **talk to each other simultaneously**
- **very little data overhead** needs to be exchanged to send messages. This means a **low latency communication**.

WebSockets are great for **real-time** and **long-lived** communications.

HTTP is great for **occasional data exchange** and interactions initiated by the client.

HTTP is much simpler to implement, while WebSockets require a bit more overhead.

Secured WebSockets

Always use the secure, encrypted protocol for WebSockets, `wss://`.

`ws://` refers to the unsafe WebSockets version (the `http://` of WebSockets), and should be avoided for obvious reasons.

Create a new WebSockets connection

```
const url = 'wss://myserver.com/something'  
const connection = new WebSocket(url)
```

`connection` is a `WebSocket` (<https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>) object.

When the connection is successfully established, the `open` event is fired.

Listen for it by assigning a callback function to the `onopen` property of the `connection` object:

```
connection.onopen = () => {  
  //...  
}
```

If there's any error, the `onerror` function callback is fired:

```
connection.onerror = (error) => {  
  console.log(`WebSocket error: ${error}`)  
}
```

Sending data to the server using WebSockets

Once the connection is open, you can send data to the server.

You can do so conveniently inside the `onopen` callback function:

```
connection.onopen = () => {  
  connection.send('hey')  
}
```

Receiving data from the server using WebSockets

Listen with a callback function on `onmessage`, which is called when the `message` event is received:

```
connection.onmessage = (e) => {  
  console.log(e.data)  
}
```

Implement a server in Node.js

ws (<https://github.com/websockets/ws>) is a popular WebSockets library for Node.js.

We'll use it to build a WebSockets server. It can also be used to implement a client, and use WebSockets to communicate between two backend services.

Easily install it using

```
yarn init  
yarn add ws
```

The code you need to write is very little:

```
const WebSocket = require('ws')

const wss = new WebSocket.Server({ port: 8080 })

wss.on('connection', (ws) => {
  ws.on('message', (message) => {
    console.log(`Received message => ${message}`)
  })
  ws.send('ho!')
})
```

This code creates a new server on port 8080 (the default port for WebSockets), and adds a callback function when a connection is established, sending `ho!` to the client, and logging the messages it receives.

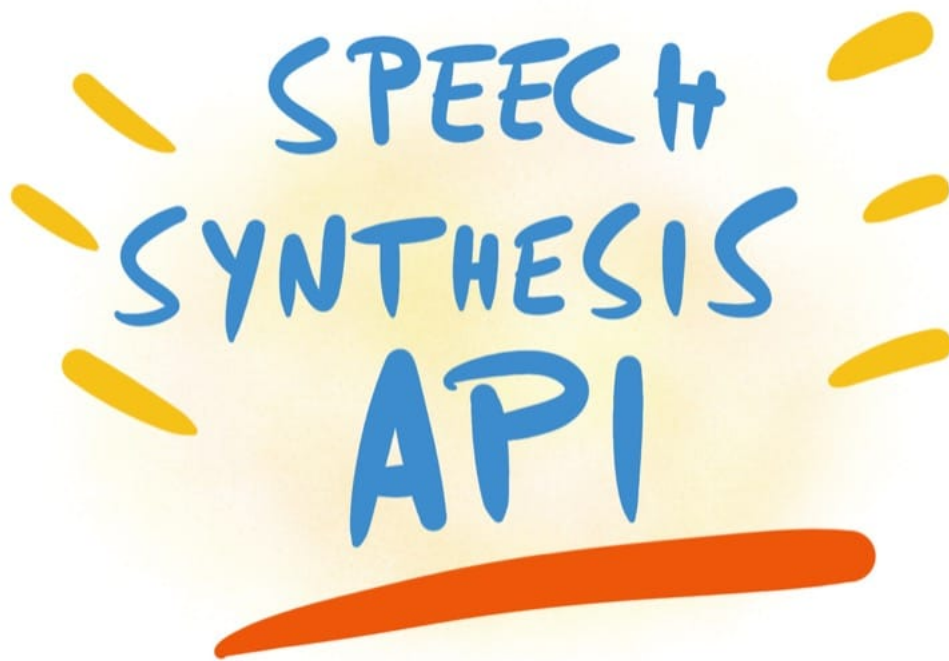
See a live example on Glitch

Here is a live example of a WebSockets server: <https://glitch.com/edit/#!/flavio-websockets-server-example>

Here is a WebSockets client that interacts with the server: <https://glitch.com/edit/#!/flavio-websockets-client-example>

THE SPEECH SYNTHESIS API

The Speech Synthesis API is an awesome API, great to experiment new kind of interfaces and let the browser talk to you



The Speech Synthesis API is an awesome tool provided by modern browsers. Introduced in 2014, it's now widely adopted (<https://caniuse.com/#feat=speech-synthesis>) and available in Chrome, Firefox, Safari and Edge. IE is not supported.

IE	Edge *	Firefox	Chrome	Safari	iOS Safari *
			² 63		
			² 64		10.3
	16	58	² 65	11	11.2
11	17	59	² 66	11.1	11.3
	18	60	² 67	TP	
		61	² 68		
			² 69		

It's part of the Web Speech API, along with the **Speech Recognition API**.

I used it recently to provide an alert on a page that monitored some parameters. When one of the numbers went up, I was alerted through the computer speakers.

Getting started

The most simple example of using the **Speech Synthesis API** stays on one line:

```
speechSynthesis.speak(new SpeechSynthesisUtterance('Hey'))
```

Copy and paste it in your browser console, and your computer should speak!

The API

The API exposes several objects to the `window` object.

SpeechSynthesisUtterance

`SpeechSynthesisUtterance` represents a speech request. In the example above we passed it a string. That's the message the browser should read aloud.

Once you got the utterance object, you can perform some tweaks to edit the speech properties:

```
const utterance = new SpeechSynthesisUtterance('Hey')
```

- `utterance.rate`: set the speed, accepts between [0.1 - 10], defaults to 1
- `utterance.pitch`: set the pitch, accepts between [0 - 2], defaults to 1
- `utterance.volume`: sets the volume, accepts between [0 - 1], defaults to 1
- `utterance.lang`: set the language (values use a BCP 47 language tag, like `en-US` or `it-IT`)
- `utterance.text`: instead of setting it in the constructor, you can pass it as a property. Text can be maximum 32767 characters
- `utterance.voice`: sets the voice (more on this below)

Example:

```
const utterance = new SpeechSynthesisUtterance('Hey')
utterance.pitch = 1.5
utterance.volume = 0.5
utterance.rate = 8
speechSynthesis.speak(utterance)
```

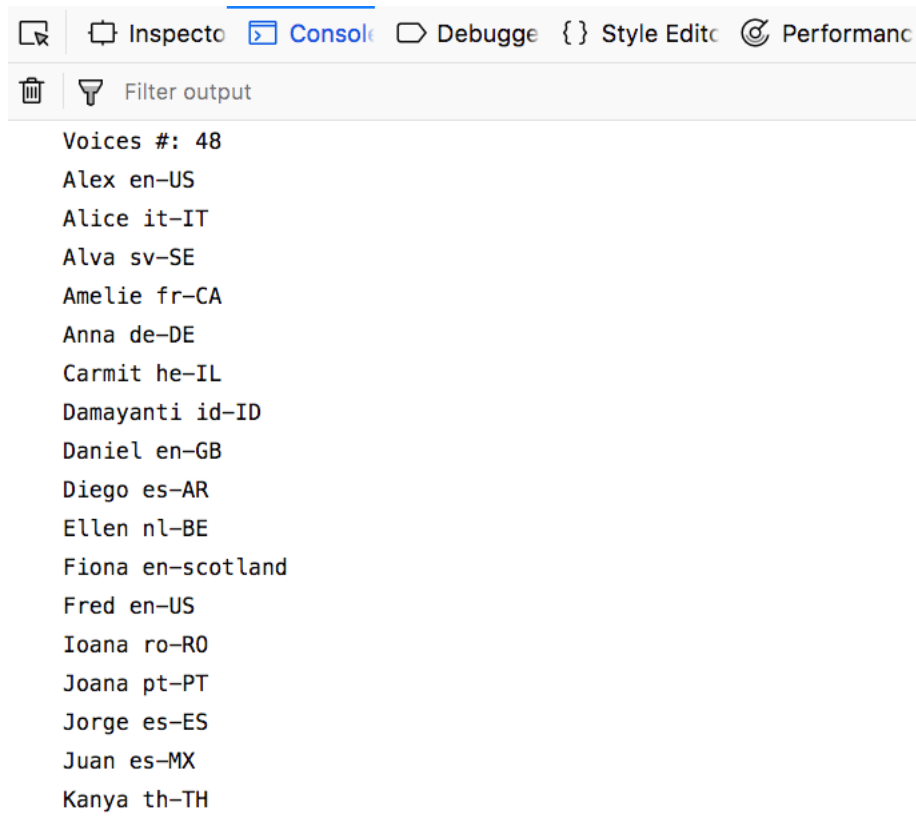
Set a voice

The browser has a different number of voices available.

To see the list, use this code:

```
console.log(`Voices #: ${speechSynthesis.getVoices().length}`)

speechSynthesis.getVoices().forEach((voice) => {
  console.log(voice.name, voice.lang)
})
```



Here is one of the cross browser issues. The above code works in Firefox, Safari (and possibly Edge but I didn't test it), but **does not work in Chrome**. Chrome requires the voices handling in a different way, and requires a callback that is called when the voices have been loaded:

```
const voiceschanged = () => {  
  console.log(`Voices #: ${speechSynthesis.getVoices().length}`)  
  speechSynthesis.getVoices().forEach((voice) => {  
    console.log(voice.name, voice.lang)  
  })  
}  
speechSynthesis.onvoiceschanged = voiceschanged
```

After the callback is called, we can access the list using `speechSynthesis.getVoices()`.

I believe this is because Chrome - if there is a network connection - checks additional languages from the Google servers:

Elements

Console

Sources

Network

Performance

Memory

top

Filter

Default levels

Zosia pl-PL

Zuzana cs-CZ

Google Deutsch de-DE

Google US English en-US

Google UK English Female en-GB

Google UK English Male en-GB

Google español es-ES

Google español de Estados Unidos es-US

Google français fr-FR

Google हिन्दी hi-IN

Google Bahasa Indonesia id-ID

Google italiano it-IT

Google 日本語 ja-JP

Google 한국의 ko-KR

Google Nederlands nl-NL

Google polski pl-PL

Google português do Brasil pt-BR

Google русский ru-RU

Google 普通话 (中国大陆) zh-CN

Google 粵語 (香港) zh-HK

Google 國語 (臺灣) zh-TW

If there is no network connection, the number of languages available is the same as Firefox and Safari. The additional languages are available where the network is enabled, but the API works offline as well.

Cross browser implementation to get the language

Since we have this difference, we need a way to abstract it to use the API. This example does this abstraction:

```
const getVoices = () => {
  return new Promise((resolve) => {
    let voices = speechSynthesis.getVoices()
    if (voices.length) {
      resolve(voices)
    }
    return
  })
  speechSynthesis.onvoiceschanged = () => {
    voices = speechSynthesis.getVoices()
    resolve(voices)
  }
}

const printVoicesList = async () => {
```

```

    (await getVoices()).forEach((voice) => {
      console.log(voice.name, voice.lang)
    })
  }

  printVoicesList()

```

See on Glitch (<https://glitch.com/edit/#!/flavio-get-list-voices-speech-api?path=script.js>)

Use a custom language

The default voice speaks in english.

You can use any language you want, by simply setting the utterance `lang` property:

```

let utterance = new SpeechSynthesisUtterance('Ciao')
utterance.lang = 'it-IT'
speechSynthesis.speak(utterance)

```

Use another voice

If there is more than one voice available, you might want to choose the other. For example the default italian voice is female, but maybe I want a male voice. That's the second one we get from the voices list.

```

const lang = 'it-IT'
const voiceIndex = 1

const speak = async (text) => {
  if (!speechSynthesis) { return }
  const message = new SpeechSynthesisUtterance(text)
  message.voice = await chooseVoice()
  speechSynthesis.speak(message)
}

const getVoices = () => {
  return new Promise((resolve) => {
    let voices = speechSynthesis.getVoices()
    if (voices.length) {
      resolve(voices)
      return
    }
    speechSynthesis.onvoiceschanged = () => {
      voices = speechSynthesis.getVoices();
      resolve(voices)
    }
  })
}

```

```
const chooseVoice = async () => {  
  const voices = (await getVoices()).filter((voice) => voice.lang == lang)  
  
  return new Promise((resolve) => {  
    resolve(voices[voiceIndex])  
  })  
}  
  
speak('Ciao')
```

See on Glitch (<https://glitch.com/edit/#!/flavio-speech-api-example?path=script.js>)

Values for the language

Those are some examples of the languages you can use:

- Arabic (Saudi Arabia) ➡ ar-SA
- Chinese (China) ➡ zh-CN
- Chinese (Hong Kong SAR China) ➡ zh-HK
- Chinese (Taiwan) ➡ zh-TW
- Czech (Czech Republic) ➡ cs-CZ
- Danish (Denmark) ➡ da-DK
- Dutch (Belgium) ➡ nl-BE
- Dutch (Netherlands) ➡ nl-NL
- English (Australia) ➡ en-AU
- English (Ireland) ➡ en-IE
- English (South Africa) ➡ en-ZA
- English (United Kingdom) ➡ en-GB
- English (United States) ➡ en-US
- Finnish (Finland) ➡ fi-FI
- French (Canada) ➡ fr-CA
- French (France) ➡ fr-FR
- German (Germany) ➡ de-DE
- Greek (Greece) ➡ el-GR
- Hindi (India) ➡ hi-IN
- Hungarian (Hungary) ➡ hu-HU
- Indonesian (Indonesia) ➡ id-ID
- Italian (Italy) ➡ it-IT
- Japanese (Japan) ➡ ja-JP
- Korean (South Korea) ➡ ko-KR

- Norwegian (Norway) ➡ no-NO
- Polish (Poland) ➡ pl-PL
- Portuguese (Brazil) ➡ pt-BR
- Portuguese (Portugal) ➡ pt-PT
- Romanian (Romania) ➡ ro-RO
- Russian (Russia) ➡ ru-RU
- Slovak (Slovakia) ➡ sk-SK
- Spanish (Mexico) ➡ es-MX
- Spanish (Spain) ➡ es-ES
- Swedish (Sweden) ➡ sv-SE
- Thai (Thailand) ➡ th-TH
- Turkish (Turkey) ➡ tr-TR

Mobile

On iOS the API works but must be triggered by a user action callback, like a response to a tap event, to provide a better experience to users and avoid unexpected sounds out of your phone.

You can't do like in the desktop where you can make your web pages speak something out of the blue.

